

TÓPICOS EM QUALIDADE E TESTE DE SOFTWARE



Organizador:
Orlando Saraiva Jr

ABNER DE SOUZA
BRUNO HENRIQUE GUINERIO
DARLAN DOS SANTOS OLIVEIRA JÚNIOR
GABRIEL DE OLIVEIRA SOUZA
GIOVANNY LOPES RIBEIRO
JAMILA MORAES CARDOSO
KALLIEL MARCOS PINHEIRO

LUIZ HENRIQUE SIMIONATO VICENTE
MARCELO FERREIRA MIRANDA
MARCO ANTONIO AMORIM FILHO
MATHEUS DE OLIVEIRA MATIAS SILVA
MIGUEL BARBIERI
STEPHAN MENDES DE OLIVEIRA
VICTOR MANOEL MARTINS
WILSON GERALDO DONIZETI PEREIRA JÚNIOR

ISBN: 978-65-02-20108-4

Um semestre de grande aprendizado...

Orlando Saraiva Júnior
orlandosaraivajr@gmail.com
orlando.nascimento@cps.sp.gov.br

Resumo

Este capítulo relata a experiência de ensino da disciplina "Qualidade e Testes de Software" na Fatec Araras durante o primeiro semestre de 2026. O objetivo é descrever a abordagem pedagógica adotada e discutir sua contribuição para os pilares de ensino, pesquisa e extensão. Trabalhamos conceitos de testes de software por meio de coding dojo remoto de TDD, dos Agile Testing Quadrants e da filosofia Unix de desenvolvimento. Como resultado, destaca-se a participação minha no evento Caipyra, com a palestra "TDD na Prática: Uma Demonstração com Django", evidenciando a integração entre sala de aula e comunidade técnica.

Palavras-chaves: Testes de Software; Test Driven Development; Ensino de Computação.

Introdução

Se você já trabalhou ou estagiou em alguma empresa de tecnologia, provavelmente já ouviu alguém falar "isso precisa ser testado antes de ir pra produção" ou "esse bug passou batido porque ninguém testou direito". Pois é, qualidade de software não é só um detalhe técnico chato que a gente empurra para o final do projeto: é uma das coisas que separa um sistema confiável de um sistema que vive dando dor de cabeça. E é justamente sobre isso que vamos conversar neste capítulo: como a disciplina "Qualidade e Testes de Software" tem sido conduzida no curso de Desenvolvimento em Software Multiplaraforma da Fatec Araras, contando um pouco do que vivemos em sala de aula durante o primeiro semestre de 2026.

Este capítulo não vai entrar em detalhes de uma ferramenta ou framework de testes específico. Não é sobre "como usar tal biblioteca" ou "como configurar tal pipeline". A ideia aqui é outra: contar como a disciplina foi pensada para gerar debate, reflexão e troca de ideias entre os alunos, sem amarrar o aprendizado a uma tecnologia só. Afinal, tecnologia muda rápido, mas a forma de pensar sobre qualidade, essa a gente quer que fique.

E qual é o objetivo deste relato? De forma geral, quero compartilhar como foi essa experiência de ensinar a disciplina ao longo do semestre e mostrar como ela contribui para a formação dos futuros profissionais da área. De forma mais específica, vamos descrever a

abordagem que usamos em sala de aula, contar quais foram os principais temas e atividades trabalhados com a turma, falar sobre os desafios e aprendizados que surgiram no caminho e, por fim, refletir sobre como essa disciplina se conecta com os três pilares que sustentam o ensino superior: ensino, pesquisa e extensão.

Desenvolvimento

Ao longo do semestre, fomos construindo com a turma uma base sólida sobre o que significa testar software de verdade, não só "rodar o programa e ver se não quebra", mas entender os diferentes tipos de teste, os níveis em que eles acontecem e por que cada um deles existe. A ideia é que o aluno saia da disciplina enxergando testes como parte do processo de desenvolvimento, e não como uma etapa burocrática que se faz depois que "o código já está pronto".

Uma das atividades que mais gerou engajamento foi o coding dojo remoto de TDD. Para quem não acompanhou ou já esqueceu, um coding dojo é basicamente um encontro em que o grupo resolve um problema de programação junto, geralmente alternando quem está no teclado, enquanto todo mundo discute as decisões em tempo real. No nosso caso, fizemos isso de forma remota e com foco em Test Driven Development: praticamos o ciclo de escrever o teste antes do código, ver o teste falhar, implementar o mínimo necessário para passar e só depois refatorar. Mais do que praticar a mecânica do "vermelho, verde, refatora", o dojo serviu para mostrar como o TDD muda a forma de pensar sobre o problema antes mesmo de começar a programar.

Também conversamos bastante sobre os Agile Testing Quadrants, aquele modelo que organiza os testes em quatro quadrantes, cruzando se eles apoiam o time ou criticam o produto, e se têm foco mais voltado para o negócio ou para a tecnologia. Esse mapa ajuda muito a entender que não existe "um tipo de teste certo": existem testes diferentes para perguntas diferentes, e parte da maturidade de um time é saber quando usar cada um deles, em vez de tentar resolver tudo com um único tipo de teste.

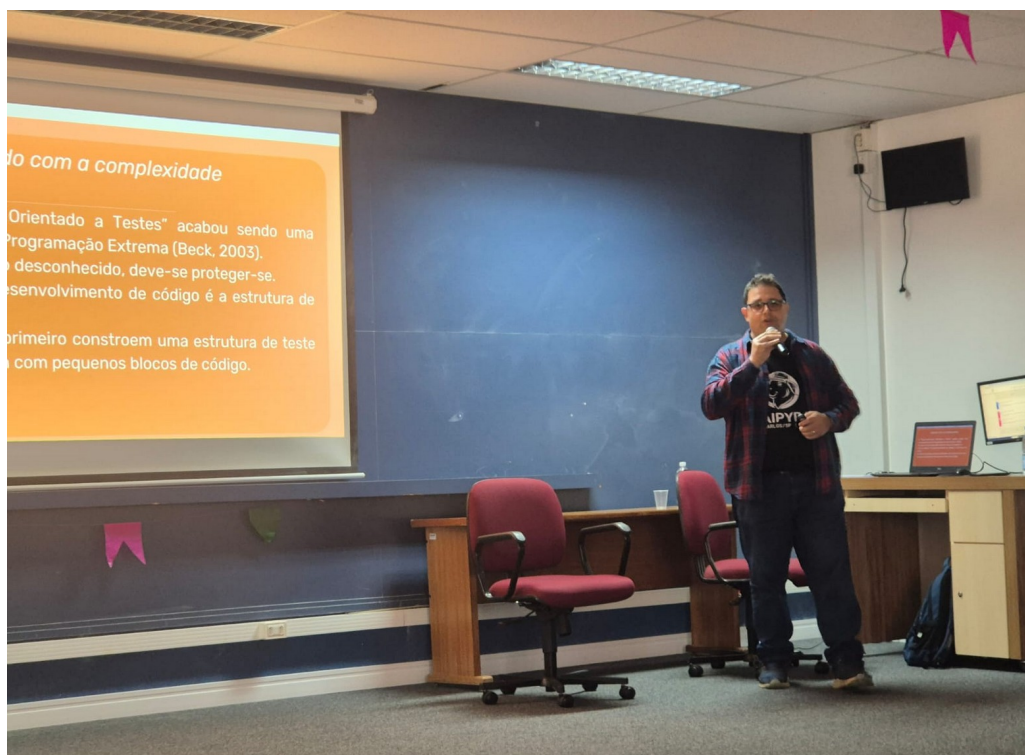
Outro assunto que entrou na roda de conversa foi a filosofia Unix de desenvolvimento. A ideia central é que cada programa deve fazer uma coisa só, e fazer bem feita, em vez de tentar resolver tudo de uma vez. Trouxemos essa filosofia para a discussão de testes porque ela se conecta diretamente com a testabilidade do código: componentes pequenos, com responsabilidade bem definida, são muito mais fáceis de testar do que módulos gigantes que fazem de tudo um pouco. Ou seja, escrever código "testável" também é, em certa medida, escrever código que segue esses princípios de simplicidade e responsabilidade única.

Vale destacar ainda que, durante o semestre, eu tive a oportunidade de levar parte dessa

discussão para fora da sala de aula, ao palestrar no evento Caipyra. O Caipyra é um evento sem fins lucrativos cujo objetivo é fomentar e compartilhar conhecimento sobre tecnologia, com foco especial em Python, no interior do Brasil.

A palestra apresentada teve como título "TDD na Prática: Uma Demonstração com Django" e partiu de uma premissa simples: existem duas formas principais de minimizar a complexidade no desenvolvimento de software, a modelagem e os testes. A partir dessa ideia, a apresentação explicou os fundamentos do Test Driven Development e realizei uma demonstração prática utilizando o framework Django, mostrando ao público como o ciclo de TDD se aplica no desenvolvimento real de uma aplicação web.

Figura 1 – Participação no evento Caypira em São Carlos



Fonte: próprio autor

Essa participação no Caipyra acabou se tornando uma extensão natural do que já vínhamos discutindo em sala de aula: os mesmos conceitos trabalhados com vocês durante o semestre, como TDD e a importância dos testes na redução da complexidade do software, foram apresentados para um público mais amplo, fora do ambiente acadêmico.

Isso reforça justamente o que comentamos na introdução deste capítulo: o ensino que acontece na Fatec Araras não fica restrito às paredes da sala de aula, ele se conecta com a pesquisa e com a extensão, alimentando e sendo alimentado por experiências como essa.

Considerações Finais

Ao longo deste capítulo, relatamos como a disciplina "Qualidade e Testes de Software" foi conduzida na Fatec Araras durante o primeiro semestre de 2026, buscando proporcionar um ambiente de debate e construção coletiva do conhecimento, sem prender o aprendizado a uma tecnologia específica. Atividades como o coding dojo remoto de TDD e as discussões sobre os Agile Testing Quadrants e a filosofia Unix mostraram que é possível, e necessário, formar profissionais que pensem criticamente sobre qualidade de software, em vez de apenas memorizar ferramentas que mudam rapidamente.

A participação do professor no evento Caipyra, reforça que o que se discute em sala de aula não precisa, e talvez não deva, ficar restrito a ela. Essa experiência evidencia como ensino, pesquisa e extensão se entrelaçam na prática docente: o conteúdo trabalhado com os alunos alimentou uma apresentação para a comunidade técnica, e essa vivência externa, por sua vez, retorna para enriquecer as próximas turmas.

Bibliografia

CAIPYRA. Caipyra 2026: evento de Python em São Carlos (SP). São Carlos, 2026. Disponível em: <https://2026.caipyra.python.org.br/>. Acesso em: 16 jun. 2026.

RAYMOND, Eric S. **The art of Unix programming**. Boston: Addison-Wesley, 2003.

BECK, Kent. **Test-driven development: by example**. Boston: Addison-Wesley, 2002.

CRISPIN, Lisa; GREGORY, Janet. **Agile testing: a practical guide for testers and agile teams**. Boston: Addison-Wesley, 2009.

Shift-left testing em projetos ágeis: a importância da antecipação dos testes para a qualidade de software

Jamila Moraes Cardoso

jmc18.ads@gmail.com

jamila.cardoso@aluno.cps.sp.gov.br

Resumo

A crescente demanda por sistemas confiáveis e de alta qualidade tem levado organizações a adotarem práticas que reduzam falhas durante o desenvolvimento de software. Nesse contexto, o Shift-Left Testing destaca-se como uma estratégia que antecipa as atividades de teste para as fases iniciais do ciclo de desenvolvimento, promovendo a identificação precoce de defeitos e a melhoria contínua da qualidade. Este trabalho tem como objetivo analisar a contribuição dessa abordagem em projetos desenvolvidos com metodologias ágeis. A pesquisa foi realizada por meio de revisão bibliográfica qualitativa, utilizando livros, artigos científicos, normas e materiais especializados em engenharia de software e testes. Os resultados demonstram que a antecipação dos testes reduz custos de correção, diminui retrabalho, aumenta a colaboração entre equipes e melhora a confiabilidade dos produtos desenvolvidos. Além disso, foi elaborado um estudo de caso demonstrando a aplicação prática da abordagem no desenvolvimento de uma calculadora simples. Conclui-se que o Shift-Left Testing representa uma importante evolução nas práticas de garantia da qualidade, contribuindo significativamente para a eficiência dos processos de desenvolvimento ágil.

Palavras-chave: Shift-Left Testing; Qualidade de Software; Testes de Software; Metodologias Ágeis; Engenharia de Software.

Introdução

A transformação digital e a crescente dependência de sistemas computacionais têm elevado a necessidade de garantir altos níveis de qualidade durante o desenvolvimento de software. Organizações dos mais diversos segmentos dependem de aplicações confiáveis para executar processos críticos, atender clientes e sustentar suas operações. Nesse cenário, falhas de software podem resultar em prejuízos financeiros, perda de credibilidade e comprometimento da experiência

do usuário.

Com o avanço das metodologias ágeis, especialmente Scrum e Kanban, as equipes passaram a trabalhar com ciclos curtos de desenvolvimento e entregas frequentes. Embora tais abordagens proporcionem maior flexibilidade e rapidez na entrega de valor, também exigem mecanismos eficientes para garantir a qualidade do produto em todas as etapas do processo.

Tradicionalmente, os testes eram realizados apenas nas fases finais do desenvolvimento, o que frequentemente resultava na descoberta tardia de defeitos. Segundo Pressman e Maxim (2021), quanto mais tarde um erro é identificado, maior é o custo necessário para sua correção. Dessa forma, surgiu a necessidade de incorporar práticas que permitam detectar falhas o mais cedo possível.

Nesse contexto, destaca-se o Shift-Left Testing, abordagem que propõe antecipar as atividades de teste para as fases iniciais do ciclo de desenvolvimento. Ao promover a colaboração entre desenvolvedores, analistas e testadores desde a definição dos requisitos, essa estratégia contribui para a construção de produtos mais confiáveis e alinhados às expectativas dos usuários.

O presente trabalho tem como objetivo analisar os benefícios e desafios da implementação do Shift-Left Testing em projetos ágeis, bem como apresentar um estudo de caso que demonstre sua aplicação prática.

Metodologia

Este trabalho caracteriza-se como uma pesquisa bibliográfica de natureza qualitativa. A investigação foi conduzida por meio da análise de livros, artigos científicos, publicações acadêmicas e materiais técnicos relacionados à qualidade de software, testes de software e metodologias ágeis.

Foram utilizadas obras clássicas da Engenharia de Software, materiais disponibilizados pelo International Software Testing Qualifications Board (ISTQB) e documentos relacionados às metodologias Scrum e Kanban. A pesquisa buscou identificar conceitos fundamentais, benefícios, limitações e boas práticas relacionadas à adoção do Shift-Left Testing.

Desenvolvimento

Além da revisão bibliográfica, foi desenvolvido um estudo de caso hipotético baseado em situações comuns encontradas em equipes ágeis de desenvolvimento de software.

Qualidade de Software

A qualidade de software pode ser definida como a capacidade de um sistema atender aos requisitos funcionais e não funcionais estabelecidos, satisfazendo as necessidades dos usuários e das organizações.

De acordo com Pressman e Maxim (2021), a qualidade deve ser incorporada durante todo o ciclo de vida do software, abrangendo atividades de planejamento, desenvolvimento, testes e manutenção. A busca pela qualidade envolve ações de garantia da qualidade (Quality Assurance – QA) e controle da qualidade (Quality Control – QC), cujo objetivo é prevenir defeitos e assegurar a conformidade do produto com padrões previamente definidos.

Entre os principais atributos de qualidade destacam-se:

Confiabilidade;

Eficiência;

Segurança;

Usabilidade;

Manutenibilidade;

Portabilidade.

Esses atributos influenciam diretamente a aceitação e o sucesso de um sistema no ambiente organizacional.

Testes de Software

Os testes de software representam uma das principais estratégias para identificar defeitos antes que o sistema seja disponibilizado aos usuários finais.

Segundo o ISTQB (2023), o processo de teste tem como objetivo avaliar a qualidade do software e reduzir os riscos associados à sua utilização.

Os principais níveis de teste são:

Testes Unitários

Verificam componentes individuais do sistema, geralmente desenvolvidos e executados pelos próprios programadores.

Testes de Integração

Avaliam a comunicação entre módulos e componentes distintos.

Testes de Sistema

Validam o comportamento do sistema como um todo, considerando requisitos funcionais e não funcionais.

Testes de Aceitação

Confirmam se o software atende às necessidades e expectativas do cliente ou usuário final.

A aplicação adequada desses testes contribui significativamente para a redução de falhas em produção.

Metodologias Ágeis

As metodologias ágeis surgiram como resposta às limitações dos modelos tradicionais de desenvolvimento, especialmente em ambientes sujeitos a mudanças frequentes.

O Manifesto Ágil, publicado em 2001, estabeleceu princípios voltados à colaboração, adaptação e entrega contínua de valor.

Entre os frameworks mais utilizados destacam-se:

Scrum

Baseado em sprints, reuniões periódicas e entregas incrementais de software.

Kanban

Focado na visualização do fluxo de trabalho e na melhoria contínua dos processos.

Nessas abordagens, a qualidade passa a ser responsabilidade de toda a equipe, reforçando a necessidade de integração entre desenvolvimento e testes.

Shift-Left Testing

O Shift-Left Testing consiste na antecipação das atividades de teste para as fases iniciais do ciclo de desenvolvimento.

O termo "Left" refere-se ao deslocamento dos testes para o lado esquerdo do fluxo tradicional de desenvolvimento, abrangendo atividades como:

Revisão de requisitos;
Definição de critérios de aceitação;
Planejamento de testes;
Desenvolvimento orientado a testes (TDD);
Automação contínua de testes.

Essa prática permite identificar problemas ainda durante a fase de análise e projeto, reduzindo significativamente os custos de correção.

Benefícios do Shift-Left Testing

A adoção da abordagem proporciona diversas vantagens:

Redução de Custos

Falhas identificadas precocemente demandam menos esforço para correção.

Menor Retrabalho

Problemas são corrigidos antes de impactar outras funcionalidades.

Maior Cobertura de Testes

A automação possibilita a execução frequente de cenários de validação.

Feedback Rápido

As equipes recebem informações contínuas sobre a qualidade do sistema.

Melhoria da Qualidade

A identificação precoce de defeitos reduz a ocorrência de falhas em produção.

Integração entre Equipes

Promove colaboração entre desenvolvedores, testadores e analistas.

Desafios da Implementação

Apesar dos benefícios, algumas dificuldades podem surgir durante a adoção:

Resistência à mudança cultural;

Necessidade de capacitação técnica;

Investimento em ferramentas de automação;

Adequação dos processos organizacionais;

Maior envolvimento dos profissionais desde as fases iniciais.

O sucesso da implementação depende do comprometimento da organização e do apoio da gestão.

Comparativo dos Processos

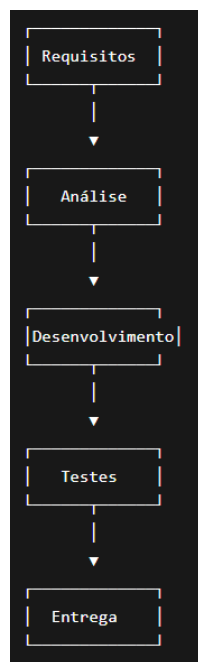


Figura 1 – Processo Tradicional de Testes

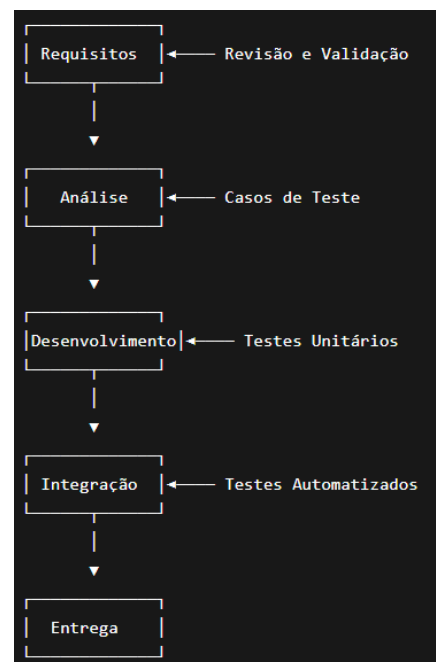


Figura 2 – Processo com Shift-Left Testing

A Figura 1 apresenta o fluxo tradicional de desenvolvimento, no qual os testes são executados apenas após a implementação das funcionalidades. Já a Figura 2 ilustra a abordagem Shift-Left Testing, caracterizada pela antecipação das atividades de validação e verificação ao longo de todo o ciclo de desenvolvimento.

Estudo de Caso

Aplicação do Shift-Left Testing no Desenvolvimento de uma Calculadora Simples

Para demonstrar a aplicação prática da abordagem Shift-Left Testing, foi considerado o desenvolvimento de uma calculadora simples capaz de realizar as quatro operações matemáticas básicas: adição, subtração, multiplicação e divisão.

Cenário Inicial

Foi realizado o desenvolvimento de uma calculadora para uso educacional. Inicialmente, esforços estavam voltados na implementação das funcionalidades, deixando a execução dos testes para o final da sprint.

Após a conclusão do desenvolvimento, foram identificados diversos problemas durante a fase de testes:

- Resultado incorreto em algumas operações de multiplicação;

- Falha ao dividir um número por zero;

- Ausência de validação para entrada de caracteres não numéricos;

- Mensagens de erro pouco informativas para o usuário;

- Inconsistências entre os requisitos definidos e o comportamento implementado.

Esses problemas exigiram retrabalho significativo, aumentando o tempo necessário para entrega do sistema.

Aplicação do Shift-Left Testing

Com o objetivo de reduzir defeitos e melhorar a qualidade do software, foi decidido aplicar os princípios do Shift-Left Testing desde o início do projeto. As seguintes ações foram adotadas:

Revisão Antecipada dos Requisitos

Antes da implementação, foram analisados os requisitos do sistema. Foram definidos critérios claros, como:

- A calculadora deve aceitar apenas valores numéricos;

- A divisão por zero deve gerar uma mensagem de erro apropriada;

Todas as operações devem retornar resultados corretos com números inteiros e decimais.

Definição Prévia dos Casos de Teste

Os casos de teste foram elaborados antes da codificação. Exemplos:

Caso de Teste	Entrada	Resultado Esperado
Soma simples	$5 + 3$	8
Subtração simples	$10 - 4$	6
Multiplicação	7×6	42
Divisão	$20 \div 5$	4
Divisão por zero	$10 \div 0$	Mensagem de erro
Entrada inválida	$A + 5$	Mensagem de erro

Desenvolvimento Orientado por Testes (TDD)

Os testes unitários foram criados antes da implementação das funcionalidades. Por exemplo, antes de desenvolver a função de divisão, foi criado um teste verificando o comportamento esperado quando o divisor fosse igual a zero.

Somente após a aprovação dos testes a funcionalidade era considerada concluída.

Exemplos de Testes Unitários

Durante a aplicação do Shift-Left Testing, os casos de teste foram definidos antes da implementação das funcionalidades da calculadora. Essa prática permitiu validar os requisitos desde as fases iniciais do desenvolvimento.

A seguir são apresentados alguns exemplos de testes unitários utilizando a linguagem Java e o framework JUnit.

Teste de Soma

@Test

```
public void deveSomarDoisNumeros() {  
    Calculadora calc = new Calculadora();
```

```
double resultado = calc.somar(5, 3);  
assertEquals(8, resultado);  
}
```

Esse teste verifica se a operação de soma retorna corretamente o valor esperado.

Teste de Subtração

```
@Test  
public void deveSubtrairDoisNumeros() {  
    Calculadora calc = new Calculadora();  
    double resultado = calc.subtrair(10, 4);  
    assertEquals(6, resultado);  
}
```

O objetivo é validar o comportamento da operação de subtração.

Teste de Multiplicação

```
@Test  
public void deveMultiplicarDoisNumeros() {  
    Calculadora calc = new Calculadora();  
    double resultado = calc.multiplicar(7, 6);  
    assertEquals(42, resultado);  
}
```

Esse teste garante que a multiplicação seja realizada corretamente.

Teste de Divisão

```
@Test  
public void deveDividirDoisNumeros() {  
    Calculadora calc = new Calculadora();  
    double resultado = calc.dividir(20, 5);  
    assertEquals(4, resultado);  
}
```

O teste valida a funcionalidade de divisão para valores válidos.

Teste de Divisão por Zero

```
@Test
```

```
public void deveLancarExcecaoAoDividirPorZero() {  
    Calculadora calc = new Calculadora();  
    assertThrows(  
        ArithmeticException.class,  
        () -> calc.dividir(10, 0)  
    );  
}
```

Esse teste verifica se o sistema trata adequadamente uma situação de erro, impedindo divisões por zero.

Teste de Entrada Inválida

```
@Test  
public void deveRetornarErroParaEntradaInvalida() {  
    Calculadora calc = new Calculadora();  
    assertThrows(  
        NumberFormatException.class,  
        () -> calc.converterNumero("ABC")  
    );  
}
```

O objetivo é garantir que entradas não numéricas sejam identificadas e tratadas corretamente.

Relação com o Shift-Left Testing

A criação desses testes antes da implementação das funcionalidades permitiu identificar problemas ainda nas fases iniciais do projeto. Dessa forma, a equipe conseguiu validar requisitos, reduzir retrabalho e aumentar a confiabilidade da calculadora antes mesmo da fase final de homologação, demonstrando na prática os benefícios da abordagem Shift-Left Testing.

Automação dos Testes

Todos os testes unitários foram integrados ao processo de desenvolvimento, sendo executados automaticamente sempre que uma alteração fosse realizada no código.

Resultados Obtidos

Após a adoção do Shift-Left Testing, os resultados observados foram:

Indicador	Antes	Depois	Variação
Defeitos encontrados	12	4	-66,7%
Retrabalho	10h	3h	-70%
Cobertura de testes	20%	85%	+65 p.p.
Testes automatizados	0	15	+15

Redução significativa dos defeitos encontrados na fase final do projeto;

Identificação precoce de problemas relacionados aos requisitos;

Menor necessidade de retrabalho;

Maior confiabilidade das operações matemáticas implementadas;

Melhoria na comunicação entre desenvolvedores e responsáveis pelos testes;

Maior velocidade na entrega da versão final da calculadora.

Além disso, falhas críticas, como a divisão por zero e a validação de entradas inválidas, foram detectadas ainda durante as fases iniciais do desenvolvimento, evitando correções tardias.

Discussão dos Resultados

Os resultados obtidos corroboram os estudos de Pressman e Maxim (2021), que destacam que a identificação precoce de defeitos reduz significativamente os custos de manutenção. Da mesma forma, as recomendações do ISTQB (2023) evidenciam que a execução contínua de testes aumenta a confiabilidade e a qualidade do software entregue. No estudo de caso apresentado, a redução de defeitos e o aumento da cobertura de testes demonstram na prática os benefícios descritos pela literatura.

Considerações Finais

O Shift-Left Testing representa uma evolução importante nas práticas de garantia da qualidade de software, especialmente em ambientes que adotam metodologias ágeis. Ao antecipar as atividades de teste para as fases iniciais do desenvolvimento, a abordagem permite identificar defeitos mais cedo, reduzir custos de correção e aumentar a confiabilidade dos sistemas produzidos.

A pesquisa bibliográfica demonstrou que a integração entre testes, desenvolvimento e automação constitui um fator determinante para o sucesso dos projetos modernos de software. Além disso, o estudo de caso evidenciou que a aplicação prática da estratégia gera benefícios concretos, como redução de retrabalho, aumento da cobertura de testes e melhoria da comunicação entre equipes.

Embora existam desafios relacionados à mudança cultural, capacitação profissional e investimento em ferramentas, os ganhos obtidos justificam sua adoção em organizações que buscam maior eficiência e qualidade em seus processos de desenvolvimento.

Como trabalhos futuros, recomenda-se investigar a aplicação do Shift-Left Testing em ambientes DevOps e em projetos baseados em microsserviços, avaliando seus impactos em contextos de maior complexidade tecnológica.

Bibliografia

BECK, Kent. *Test Driven Development: By Example*. Boston: Addison-Wesley, 2003.

INTERNATIONAL SOFTWARE TESTING QUALIFICATIONS BOARD (ISTQB). *Certified Tester Foundation Level Syllabus*. Version 4.0. 2023.

PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de Software: Uma Abordagem Profissional*. 9. ed. Porto Alegre: AMGH, 2021.

SCHWABER, Ken; SUTHERLAND, Jeff. *The Scrum Guide*. 2020. Disponível em: <https://scrumguides.org>. Acesso em: 08 jun. 2026.

SOMMERVILLE, Ian. *Engenharia de Software*. 10. ed. São Paulo: Pearson, 2019.

KANBAN UNIVERSITY. *Kanban Guide*. Disponível em: <https://kanban.university>. Acesso em: 08 jun. 2026.

MYERS, Glenford J.; SANDLER, Corey; BADGETT, Tom. *The Art of Software Testing*. 3. ed. Hoboken: John Wiley & Sons, 2011.

A ferramenta Cypress

Stephan Mendes de Oliveira
stephan.mendesnew@gmail.com
stephan.oliveira @aluno.cps.sp.gov.br

Resumo

Este capítulo apresenta um guia prático sobre o Cypress, uma ferramenta de código aberto que revolucionou a automação de testes para aplicações web. Através de uma abordagem direta, o texto aborda desde os fundamentos teóricos e pilares da ferramenta até a sua aplicação no mundo real. É apresentado o passo a passo para a preparação do ambiente utilizando Node.js e JavaScript, seguido pela exploração da estrutura padrão de pastas do framework. Por fim, são demonstrados exemplos práticos de script automatizados cobrindo testes de interface visual (E2E) e validações de API, consolidando o Cypress como uma solução indispensável para assegurar a qualidade e a agilidade no desenvolvimento de software.

Palavras-chaves: Cypress. Automação de testes. JavaScript. Testes de API. Qualidade de software

Introdução

A velocidade com que os sistemas web evoluem exige que as estratégias de garantia de qualidade acompanhem o mesmo ritmo. Em metodologias ágeis, os testes manuais repetitivos tornam-se gargalos, abrindo espaço para a necessidade de automação. É nesse cenário que o Cypress se consolidou como uma das ferramentas mais influentes do mercado.

Criado para resolver dores crônicas de frameworks mais antigos (que sofriam com lentidão e testes que quebravam sem motivo aparente), o Cypress propõe uma experiência de desenvolvimento simplificada e altamente confiável. Ele permite que desenvolvedores e analistas qualidade (QAs) simulem o comportamento exato de um usuário real na tela, reduzindo drasticamente a ocorrência de falhas que chegam até o cliente final.

A escolha deste framework justifica-se por sua capacidade de unificar testes de ponta a ponta e testes de serviços sob uma mesma tecnologia, otimizando o tempo das equipes e garantindo entregas de software muito mais robustas.

O que é o Cypress?

O Cypress é um framework de automação de testes de próxima geração, construindo especificamente para a web moderna. O ecossistema é dividido em duas partes principais:

- Cypress App: A ferramenta de execução local, que instalamos no projeto. Ela é 100% gratuita e distribuída sob a licença open-source MIT.
- Cypress Cloud: Uma plataforma complementar paga (antigamente chamada de Dashboard) voltada para a nuvem, utilizada por grandes equipes para gerenciar relatórios, analisar testes intermitentes (flaky tests) e rodar testes em paralelo na esteira de CI/CD.

Os Pilares da Ferramenta

A alta adoção do Cypress pelo mercado se apoia em recursos nativos inovadores que facilitam o dia a dia de quem escreve os testes:

- Viagem no Tempo (Time Traveling): O Cypress tira fotos instantâneas (snapshots) de cada comando executado. Ao visualizar o painel do teste, basta passar o mouse sobre o comando (como um clique ou digitação) para ver exatamente como a tela do sistema estava naquele milissegundo.
- Depuração Direta (Debugging): Como o ambiente de testes roda acoplado ao navegador, mensagens de erro claras são exibidas na tela. Além disso, é possível abrir o console do próprio navegador (F12) para inspecionar falhas e respostas de rede em tempo real.
- Suporte Multi-Navegadores (Cross-Browser); O Cypress oferece suporte nativo para rodar os testes em diferentes motores de navegação, incluindo a família Chromium (Google Chrome, Microsoft Edge, Electron) e também Firefox e WebKit (o motor do Safari).
- Esperas Automáticas (Automatic Waiting): Esqueça comandos manuais para fazer o teste “esperar a página carregar”. O Cypress aguarda automaticamente que os elementos fiquem visíveis e que as requisições terminem antes de avançar para o próximo passo.

Desenvolvimento

Abaixo, detalhamos o fluxo prático para instalar, estruturar e executar seus primeiros testes com JavaScript.

Pré-requisitos do Ambiente

Para que o Cypress funcione, é necessário ter o Node.js (ambiente de execução JavaScript) instalado na máquina. O Node.js inclui o npm (gerenciador de pacotes), que utilizaremos para baixar a ferramenta.

Passo 1: Instalar o Node.js:

- Acesse o site oficial: <http://nodejs.org>
- Baixe a versão LTS – Long Term Support
- Siga as instruções do instalador para concluir a instalação

Passo 2: Verificar se o Node.js foi instalado corretamente:

Abra o terminal do seu sistema e execute o comando como na figura abaixo para verificar se o Node.js está instalado corretamente.

Figura 1 – verificando versão instalada

```
steph@DESKTOP-7QOUGOF MINGW64 ~  
$ node -v  
v22.14.0
```

Fonte: Próprio autor.

Passo 3: Iniciando o projeto Node:

Crie uma pasta para o seu projeto no computador. Abra o terminal do seu sistema dentro dessa pasta e execute o comando abaixo para iniciar um projeto Node padrão:

Figura 2 – Iniciando projeto Node

```
steph@DESKTOP-7QOUGOF MINGW64 ~/Documents/projectNode  
$ npm init -y  
Wrote to C:\Users\steph\Documents\projectNode\package.json:  
  
{  
  "name": "projectnode",  
  "version": "1.0.0",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "description": ""  
}
```

Fonte: Próprio autor.

Passo 4: Instalação do Cypress como uma dependência:

Instale o framework Cypress como uma dependência de desenvolvimento no seu projeto.

Figura 3 – Instalando o Cypress no projeto

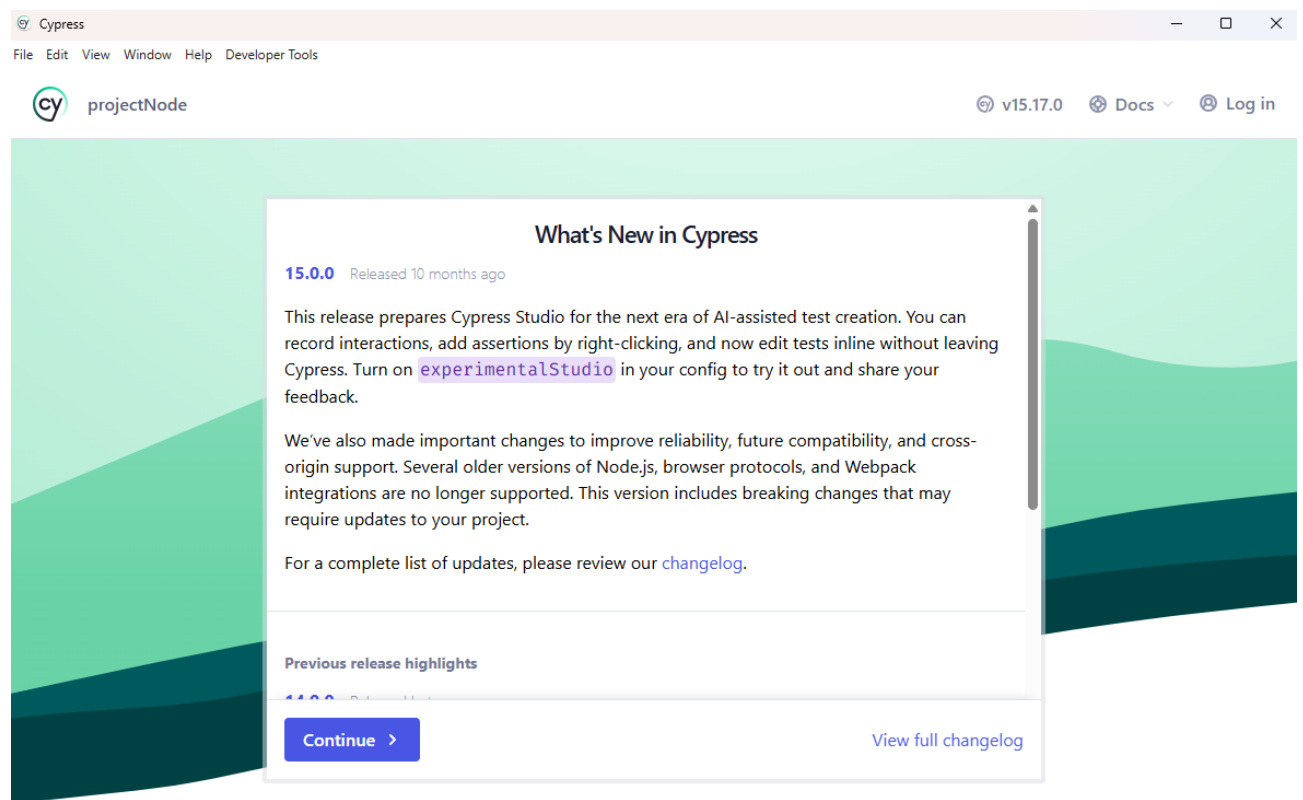
```
steph@DESKTOP-7QOUGOF MINGW64 ~/Documents/projectNode  
$ npm install cypress --save-dev  
  
added 169 packages, and audited 170 packages in 50s  
  
53 packages are looking for funding  
  run `npm fund` for details  
  
found 0 vulnerabilities
```

Se tudo ocorrer como o planejado, o terminal retornara informações dos pacotes que ele baixou e instalou o Cypress e todas as dependências que ele precisa para funcionar, além de se a instalação foi limpa e segura, demonstrando se houve alertas de segurança ou não.

Passo 5: Abrindo o Cypress:

Com o comando abaixo abra a interface interativa do Cypress para que ele crie a estrutura de arquivos padrão automaticamente:

Figura 4 – Interface interativa exibe boas-vindas do Cypress na versão v15.17.0, fornecendo as opções de configuração automática para testes ponta a ponta (E2E)

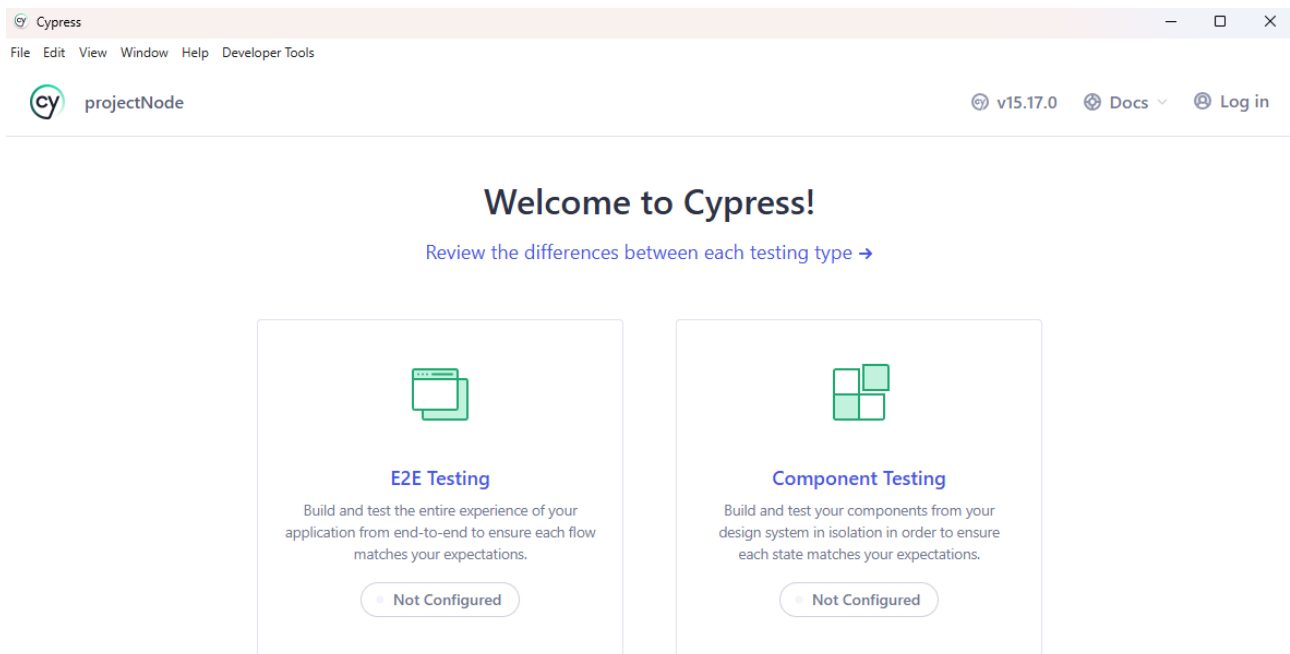


Passo 5: Configuração Assistida na Interface Gráfica:

Para que a estrutura de testes seja gerada automaticamente na raiz do seu projeto, siga as etapas do assistente visual do Cypress:

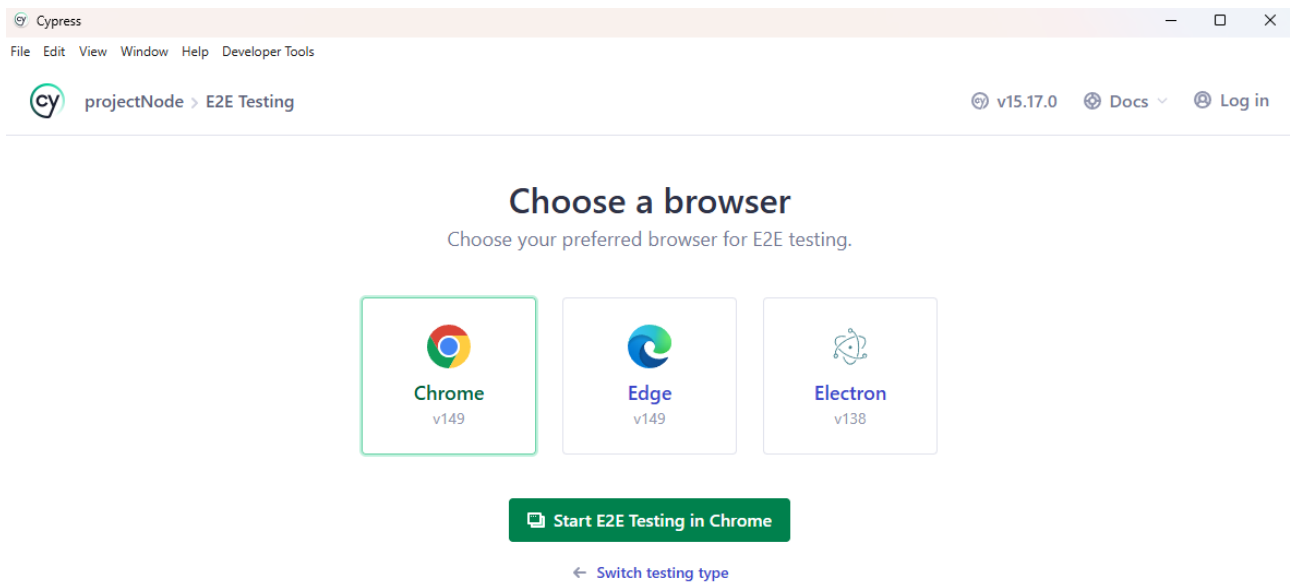
1. Na janela gráfica inicial (figura 4), clique em “Continue”.
2. Na tela de seleção do tipo de teste, clique no bloco que corresponde ao tipo de testes (E2E Testing ou Component Testing), no caso do exemplo, vamos clicar em E2E Testing.

Figura 5 – Após clicar em continue a interface limpa fornecendo as opções de configuração do Cypress



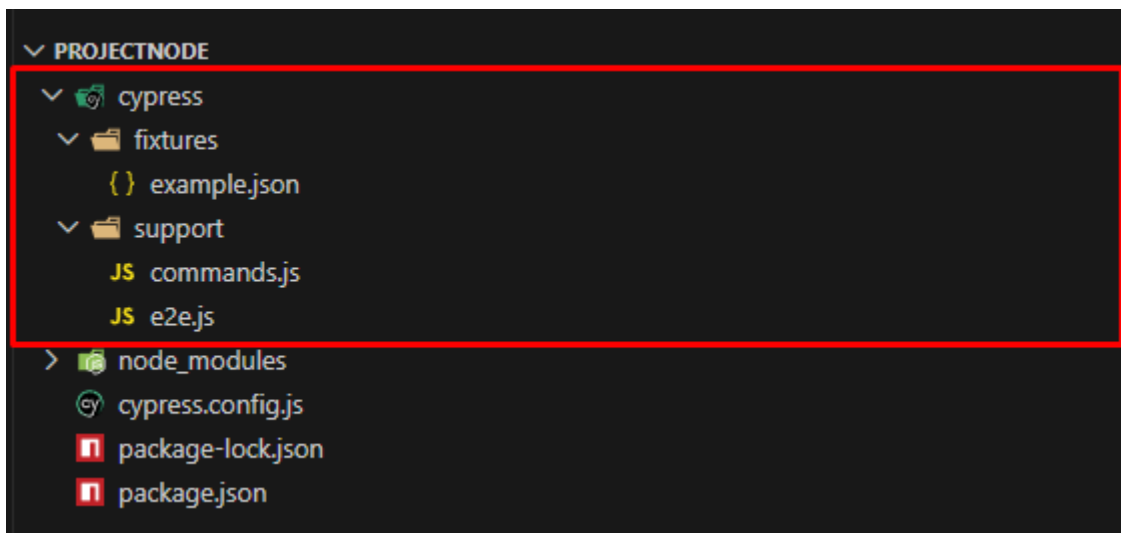
3. O painel exibirá uma lista com os novos arquivos de configuração que serão implantados (incluindo o *cypress.config.js*). Clique em “Continue”.
4. Na janela de escolha de navegadores, selecione o navegador de sua preferência (como o Google Chrome) e clique no botão verde “Start E2E Testing”.

Figura 6 – Opções de navegadores disponíveis em sua máquina.



Assim que o navegador controlado for inicializado, o setup inicial estará concluído e a pasta principal de testes será criada na raiz do seu projeto.

Figura 7 – Diretório cypress criado na raiz do projeto:



Criando o primeiro Teste

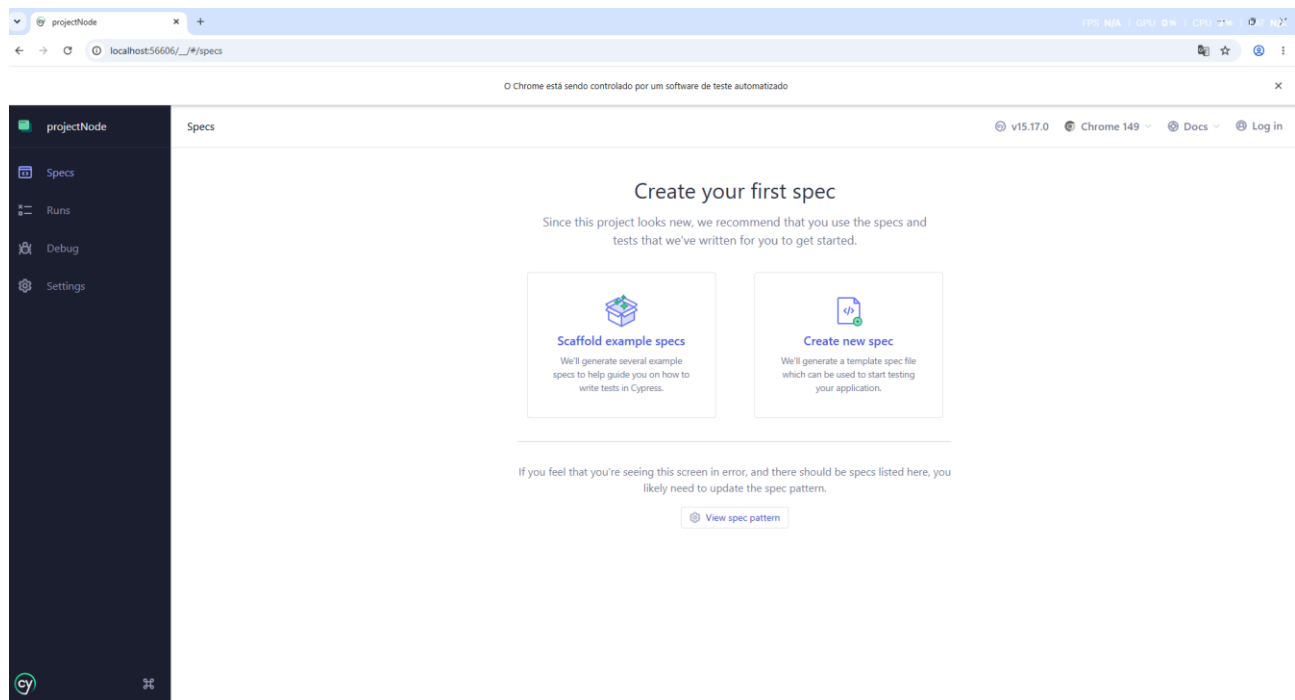
Após concluir a configuração assistida na interface gráfica, o framework organiza seus

componentes em uma árvore de diretórios modularizada na raiz do editor de código (como o VS Code), sob uma pasta principal chamada *cypress/*.

Para criar o primeiro arquivo de testes de forma automatizada e já estruturar a subpasta recomendada para os testes de ponta a ponta (e2e), utiliza-se o próprio assistente visual do Cypress que ficou aberto no navegador:

1. Na tela principal do assistente, clique no botão “Create new spec”.

Figura 8 – Tela de criação do primeiro spec



Fonte: Próprio autor.

2. No campo principal de texto que solicita o caminho do arquivo, substitua o nome padrão e defina o nome do arquivo de sua preferência (no exemplo a seguir escolhi *jornada_testes.cy.js*).

Figura 9 – Definição do nome do arquivo

Enter the path for your new spec ×

 cypress\e2e\jornada_testes.cy.js

Create spec

Back

Fonte: Próprio autor.

3. Confirme a criação no botão de salvamento

Figura 10 – Confirmação do sucesso na criação do spec

Great! The spec was successfully added ×

 cypress\e2e\jornada_testes.cy.js ^

```
1 describe('template spec', () => {
2   it('passes', () => {
3     cy.visit('https://example.cypress.io')
4   })
5 })
```

 Okay, run the spec

 Create another spec

Fonte: Próprio autor.

Com essa ação realizada por meio da interface gráfica, o Cypress cria automaticamente a pasta *e2e* dentro de *cypress/* e adiciona o arquivo *jornada_teses.cy.js* com um código de exemplo padrão (template). A arquitetura final de pastas na raiz do seu projeto passará a exibir a seguinte

divisão:

- *cypress/e2e/*: O repositório central onde as especificações técnicas de teste devem residir.
Regra obrigatória: Todo script de teste precisa utilizar o sufixo *.cy.js* para que o motor de varredura do Cypress o reconheça como um arquivo de automação válido.
- *cypress/fixtures/*: Pasta reservada para armazenar arquivos de massas de dados estáticos (normalmente estruturados em formato *.json*), simulando dados vindos de fontes externas.
- *cypress/support/*: Diretório que abriga configurações globais e comportamentos pré-execução, como o arquivo *commands.js*, usado para criar comandos personalizados.
- *cypress.config.js*: Arquivo centralizado de configuração do projeto, onde são definidos parâmetros globais como URLs base e tempos de limite (timeouts).

Para demonstrar a versatilidade do Cypress em cobrir diferentes camadas de uma aplicação utilizando JavaScript puro, o código padrão gerado automaticamente na etapa anterior foi substituído.

Figura 11 – Código proposto para explicação de maneira didática:

```
cypress > e2e > jornada_testes.cy.js > describe('Testes de Autenticação e APIs') callback
1  describe('Testes de Autenticação e APIs', () => {
2    // Cenário de Interface (UI)
3    it('Login com sucesso preenchendo credenciais válidas', () => {
4      cy.visit('https://example.cypress.io/commands/actions');
5      // Digita o e-mail no campo e valida se o texto entrou certo
6      cy.get('.action-email')
7        .type('usuario.correto@email.com')
8        .should('have.value', 'usuario.correto@email.com');
9    });
10   // Cenário de Integração (API)
11   it('Validar retorno dos dados do usuário via API', () => {
12     // Faz o GET direto no endpoint de usuários
13     cy.request('GET', 'https://jsonplaceholder.typicode.com/users/1')
14     .then((response) => {
15       // Valida o status code 200 (OK)
16       expect(response.status).to.eq(200);
17
18       // Garante que os campos obrigatórios estão voltando no body
19       expect(response.body).to.have.property('name');
20       expect(response.body).to.have.property('email');
21     });
22   });
23 });
```

Fonte: Próprio autor.

O script utiliza funções globais do ecossistema Mocha (nativo no Cypress) para estruturar a

suíte de testes:

- *describe*: Atua como um bloco agrupador principal. Ele é utilizado para dar um título à suíte de testes (neste caso, focado em autenticação e validação de serviços), organizando a exibição no relatório visual.
- *it*: Define um cenário de teste individual dentro do bloco delimitado pelo *describe*. Cada função *it* representa um caso de teste isolado que passará ou falhará de forma independente.

Análise do Cenário Interface Visual: Este cenário simula o comportamento de um usuário real interagindo com os elementos visuais renderizados na tela da aplicação (frontend):

- *cy.visit()*: Comando de navegação que instrui o navegador controlado a abrir uma URL específica de forma direta.
- *cy.get(' .action-email')*: Realiza uma varredura completa na estrutura do DOM(página HTML) para capturar um elemento. Neste exemplo, localiza o campo de texto por meio do seu seletor de classe CSS(*.action-email*).
- *.type('usuario.correto@email.com')*: Acoplado ao seletor anterior, este método simula a digitação física dos caracteres do e-mail dentro da caixa de texto mapeada.
- *.should('have.value', ...)*: Representa uma asserção (validação de segurança). O comando verifica se a propriedade interna do input realmente contém o texto digitado, garantindo que o comportamento lógico da interface está correto.

Análise do Cenário Validação de Serviços (API/Backend): Este cenário testa a camada interna do sistema (*backend*) por meio de requisições diretas de dados, sem carregar elementos visuais na tela:

- *cy.request('GET',...)*: Dispara uma requisição HTTP nativa para buscar os dados dos usuários no servidor, sem precisar de bibliotecas externas.
- *.then((response)=> {...})*: Captura a resposta enviada pelo servidor para que o script possa analisar suas propriedades.
- *expect(response.status).to.eq(200)*: Valida o código de rede; o valor 200 confirma internacionalmente que a comunicação com o servidor foi bem-sucedida.
- *Expect(response.body).to.have.property()*: Inspecciona o corpo de dados recebido (payload JSON) e garante que chaves obrigatórias como *name* e *email* estão presentes no contrato.

Considerações Finais

A implementação de testes automatizados com o Cypress demonstra como a engenharia de qualidade pode ser integrada de forma ágil ao desenvolvimento de software. A capacidade de validar, em um único ecossistema, o comportamento visual da interface (frontend) e a integridade dos serviços de dados (backend) confere uma flexibilidade indispensável para equipes que buscam alta cobertura de testes e entregas rápidas.

Como recomendação técnica, sugere-se a incorporação de validações negativas e o uso de massas de dados externas (fixtures) para criar cenários mais robustos. Para trabalhos futuros, propõe-se a integração desta suíte de testes a um pipeline de Integração Contínua (CI/CD) utilizando o GitHub Actions. Isso permitirá que os roteiros híbridos de interface e API sejam disparados de forma automática a cada alteração de código, consolidando uma cultura de qualidade contínua e prevenindo a ocorrência de regressões no sistema antes que elas cheguem ao ambiente de produção.

Bibliografia

Automação com Cypress. QA Mind7 – (Thairam Michael). Disponível em:

<https://www.youtube.com/watch?v=VyIFjUloYM4&list=PLBHHiNoJsoNxnovuGjHsC0oZpN99PMhT9> . Acesso em: 03 jun. 2026.

CYPRESS. Cypress Documentation. Disponível em: <https://docs.cypress.io/>. Acesso em: 04 jun. 2026.

SARAIVA JUNIOR, Orlando. ebook_qualidade_teste_software. GitHub. Disponível em:

https://github.com/orlandosaraivajr/ebook_qualidade_teste_software/blob/master/978-65-01-55343-6_Tópicos_em_Qualidade_e_Testes_de_Software_Primeiro_semestre_2025.pdf . Acesso em: 06 jun. 2026.

Resumo

Durante o desenvolvimento de um software, é comum que novas funcionalidades sejam adicionadas, erros sejam corrigidos e partes do código sejam modificadas constantemente. Em meio a tantas mudanças, garantir que tudo continue funcionando corretamente pode se tornar uma tarefa complicada. É nesse cenário que entram os testes automatizados, que ajudam os desenvolvedores a verificar rapidamente se o sistema continua se comportando da forma esperada.

Entre as ferramentas disponíveis para Ruby, o RSpec se destaca por sua popularidade e facilidade de uso. Sua proposta é permitir a criação de testes de forma simples e legível, tornando mais fácil identificar problemas antes que eles cheguem ao usuário final. Além disso, os testes escritos com RSpec servem como uma espécie de documentação do sistema, mostrando de maneira clara qual comportamento cada funcionalidade deve apresentar.

Este material tem como objetivo apresentar os principais conceitos relacionados ao RSpec, explicando seu funcionamento, suas vantagens e sua importância dentro do processo de desenvolvimento de software. Também serão abordadas situações práticas em que a ferramenta pode ser utilizada para melhorar a qualidade do código, reduzir a ocorrência de bugs e facilitar a manutenção de projetos desenvolvidos em Ruby.

Palavras-chaves: Rspec, Ruby, Rails, OOP

Introdução

O RSpec surgiu em 2005, criado por Steven Baker, David Chelimsky e Aslak Hellesoy, com a proposta de tornar os testes em Ruby mais legíveis e próximos da linguagem natural. Na época, muitas ferramentas de teste exigiam uma abordagem mais técnica e menos intuitiva, enquanto o RSpec buscava descrever o comportamento esperado do sistema de forma clara, seguindo conceitos do Behavior-Driven Development (BDD). Com o passar dos anos, a ferramenta se tornou uma das mais utilizadas pela comunidade Ruby e passou a fazer parte do dia a dia de muitos desenvolvedores e projetos profissionais.

Meu primeiro contato com o RSpec aconteceu durante os estudos no The Odin Project, plataforma bastante conhecida por ensinar desenvolvimento web através de projetos práticos. Até aquele momento, eu já tinha escrito bastante código em Ruby, mas ainda não tinha parado para pensar seriamente em testes automatizados. A princípio, a ideia

parecia simples: escrever alguns testes e verificar se o programa funcionava. Na prática, porém, a experiência foi bem diferente.

Durante o desenvolvimento dos meus projetos pessoais, comecei a perceber a quantidade absurda de erros que passavam despercebidos e eram quase “irrelevantes na perspectiva de quem só queria passar o software e pronto” quando eu testava tudo manualmente. Muitas vezes uma alteração aparentemente pequena quebrava alguma funcionalidade antiga, gerando bugs difíceis de identificar. Em diversos momentos eu passava horas procurando problemas no código até descobrir que um simples teste automatizado teria apontado exatamente onde estava a falha.

Foi justamente enfrentando esses problemas que comecei a precisar de ferramentas de teste e comecei a usar o RSpec. Mais do que apenas verificar se um método retorna o valor correto, a ferramenta ajuda a criar uma rede de segurança para o projeto. Hoje em dia, trabalho em uma empresa de Desenvolvimento de software logístico e a utilidade do RSpec no nosso dia a dia é absurdamente comum

Ao longo deste material serão apresentados os principais conceitos do RSpec, seu funcionamento, suas vantagens e exemplos de utilização. Além dos aspectos técnicos da ferramenta, também será abordada sua importância na redução de bugs, na manutenção do código e no desenvolvimento de aplicações mais confiáveis.

Desenvolvimento

Como o RSpec Funciona

Antes de entender a sintaxe do RSpec, é importante entender a ideia por trás da ferramenta. Muita gente pensa que um teste automatizado serve apenas para verificar se um pedaço de código funciona, mas na prática ele funciona mais como uma forma de descrever o comportamento esperado de um sistema.

Imagine que você tem um cachorro. Você sabe que, quando joga uma bolinha, o comportamento esperado é que ele corra atrás dela. Se o cachorro simplesmente sentar e começar a olhar para o céu, algo está errado. O RSpec segue uma lógica parecida: você descreve qual comportamento espera observar e depois verifica se ele realmente acontece.

Por exemplo, imagine uma classe chamada Cachorro. Se ela possui um método chamado **latir**, espera-se que, ao executar esse método, o resultado seja um latido. O teste não está preocupado em saber exatamente como o método foi construído internamente. Ele apenas verifica se o comportamento observado corresponde ao comportamento esperado.

```
class Cachorro
  def latir
    "Au Au!"
  end
end

class Pato
  def grasnar
    "Quack!"
  end
end
```

Table 1 - Classe Cachorro

A mesma ideia pode ser aplicada a outros animais. Se tivermos uma classe representando um pato, podemos esperar que ele grasne. Se tivermos uma classe representando um peixe, podemos esperar que ele nade. O objetivo do teste é confirmar que cada objeto está se comportando da maneira que deveria.

É justamente por isso que o RSpec é frequentemente associado ao conceito de Behavior-Driven Development (BDD), ou Desenvolvimento Orientado por Comportamento. Em vez de focar apenas nos detalhes técnicos da implementação, o desenvolvedor passa a pensar no comportamento esperado do sistema. A pergunta deixa de ser "como isso foi programado?" e passa a ser "o que isso deveria fazer?".

Essa mudança de perspectiva traz algumas vantagens. Quando um teste falha, normalmente fica muito mais fácil entender o problema, porque a mensagem de erro mostra exatamente qual comportamento não ocorreu como esperado. Além disso, os próprios testes acabam funcionando como uma forma de documentação, permitindo que outros desenvolvedores entendam rapidamente como determinada funcionalidade deveria se comportar.

```
require_relative "animal"

RSpec.describe Cachorro do
  it "late quando o método latir é chamado" do
    cachorro = Cachorro.new

    expect(cachorro.latir).to eq("Ai Au!")
  end
end

RSpec.describe Pato do
  it "grasna quando o método grasnar é chamado" do
    pato = Pato.new

    expect(pato.grasnar).to eq("Quack!")
  end
end
```

Table 2 - Chamada de metodo inesperada

```
jrdotan@Jrdotan ~/D/ebook> nvim tecnica.py
jrdotan@Jrdotan ~/D/ebook> nvim animal.rb
jrdotan@Jrdotan ~/D/ebook> nvim animal_spec.rb
jrdotan@Jrdotan ~/D/ebook> rspec animal_spec.rb
F.

Failures:

  1) Cachorro late quando o método latir é chamado
     Failure/Error: expect(cachorro.latir).to eq("Ai Au!")

       expected: "Ai Au!"
       got: "Au Au!"

       (compared using ==)
       # ./animal_spec.rb:7:in `block (2 levels) in <top (required)>'

Finished in 0.00975 seconds (files took 0.06469 seconds to load)
2 examples, 1 failure

Failed examples:

rspec ./animal_spec.rb:4 # Cachorro late quando o método latir é chamado
jrdotan@Jrdotan ~/D/ebook [1]> |
```

Table 3 - Mensagem de erro

De forma simples, o RSpec funciona como um fiscal que observa o comportamento do sistema e compara esse comportamento com aquilo que foi definido pelo desenvolvedor.

Se tudo acontecer conforme esperado, o teste passa. Caso contrário, ele falha e aponta que existe algum problema que precisa ser investigado.

Principais Ferramentas e Recursos do RSpec

Depois de entender a ideia geral por trás dos testes automatizados, é importante conhecer algumas das ferramentas que o RSpec oferece para organizar e escrever os testes. Embora existam muitos recursos avançados, alguns elementos aparecem praticamente em qualquer projeto e servem como base para o restante da ferramenta.

Um dos componentes mais utilizados é o **describe**. Ele serve para agrupar testes relacionados a uma mesma classe, método ou funcionalidade. Pense nele como uma categoria que informa ao RSpec qual parte do sistema está sendo analisada. Se estivermos testando uma classe chamada Cachorro, por exemplo, faz sentido criar um bloco **describe** para reunir todos os testes relacionados a ela.

```
class Cachorro
  def initialize(com_fome = false)
    @com_fome = com_fome
  end

  def latir
    "Au Au!"
  end

  def pedir_comida
    @com_fome
  end
end

RSpec.describe Cachorro do
```

Table 4 - Metodo describe

Outro recurso muito comum é o **it**. Enquanto o **describe** informa o que está sendo testado, o **it** descreve qual comportamento específico é esperado. É nele que normalmente escrevemos frases como "late quando o método latir é chamado" ou "retorna verdadeiro quando o animal está acordado". Essa abordagem torna os testes mais fáceis de ler e entender.

Talvez o elemento mais importante do RSpec seja o **expect**, responsável por realizar a verificação propriamente dita. É através dele que o desenvolvedor informa qual resultado espera receber. Quando o resultado obtido corresponde ao esperado, o teste passa. Caso contrário, ele falha e o RSpec exibe uma mensagem detalhando o problema encontrado.

Associado ao **expect**, existem os chamados **matchers**. Eles funcionam como operadores de comparação especializados. O matcher **eq**, por exemplo, verifica se dois valores são iguais. Outros matchers bastante utilizados são **be**, **include**, **start_with**,

end_with, **be_nil** e **be_empty**. Esses recursos tornam os testes mais expressivos e facilitam a leitura do que está sendo validado.

```
# before
before do
  @cachorro = Cachorro.new
end

# it + expect + eq
it "late corretamente" do
  expect(@cachorro.latir).to eq("Au Au!")
end

# matcher be_a
it "é uma instância de Cachorro" do
  expect(@cachorro).to be_a(Cachorro)
end

# matcher start_with
it "começa o latido com Au" do
  expect(@cachorro.latir).to start_with("Au")
end

# matcher end_with
it "termina o latido com !" do
  expect(@cachorro.latir).to end_with("!")
end
```

Table 5 - Ferramentas Rspec

Outra ferramenta interessante é o **context**. Ele permite separar cenários diferentes dentro do mesmo conjunto de testes. Imagine um sistema de alimentação para animais. Um cachorro pode se comportar de uma forma quando está com fome e de outra quando já foi alimentado. Em vez de misturar tudo no mesmo lugar, é possível criar contextos diferentes para cada situação, deixando os testes mais organizados.

O RSpec também oferece recursos para preparação de dados antes da execução dos testes. Um dos mais conhecidos é o **before**, utilizado quando algum objeto precisa ser criado repetidamente em vários testes. Em vez de escrever o mesmo código diversas vezes, o desenvolvedor pode colocá-lo em um bloco de preparação, tornando os testes mais limpos e fáceis de manter.

Essas ferramentas formam a base da maior parte dos testes escritos com RSpec. Embora existam recursos mais avançados, como mocks, doubles, stubs e testes de integração, compreender bem o funcionamento de **describe**, **it**, **expect**, **matchers**, **context** e **before** já permite criar uma quantidade significativa de testes úteis para projetos pessoais e profissionais.


```

# context
context "quando está com fome" do
  before do
    @cachorro = Cachorro.new(true)
  end

  it "pede comida" do
    expect(@cachorro.pedir_comida).to eq(true)
  end
end

context "quando não está com fome" do
  before do
    @cachorro = Cachorro.new(false)
  end

  it "não pede comida" do
    expect(@cachorro.pedir_comida).to eq(false)
  end
end

# include
it "está presente na lista de animais" do
  animais = ["Cachorro", "Gato", "Pato"]

  expect(animais).to include("Cachorro")
end

# be_nil
it "retorna nil para variável vazia" do
  osso = nil

  expect(osso).to be_nil
end
end

```

Table 6 - Context e outras ferramentas rspec

Vantagens e Desvantagens do RSpec

Assim como qualquer ferramenta utilizada no desenvolvimento de software, o RSpec possui pontos fortes e limitações. Apesar de ser uma das bibliotecas de testes mais populares do Ruby, isso não significa que ele seja perfeito ou que se encaixe em todos os cenários. Conhecer seus prós e contras é importante para entender por que tantas equipes o utilizam e quais desafios podem surgir durante seu uso.

Uma das maiores vantagens do RSpec é a legibilidade. Sua sintaxe foi projetada para se aproximar da linguagem natural, tornando os testes mais fáceis de entender, eu sei que muita gente vai dizer o mesmo sobre qualquer ferramenta de teste, mas qual é? a gente ta falando de Ruby, a linguagem conhecida por literalmente simplificar o processo ao ponto de nem mesmo requerir parenteses na hora de declarar uma function. Mesmo

alguém que não participou do desenvolvimento pode ler um teste e compreender qual comportamento está sendo validado. Isso faz com que os testes também funcionem como uma forma de documentação do sistema.

Outro ponto positivo é a facilidade para encontrar bugs. Durante meus estudos com Ruby e no desenvolvimento de projetos pessoais, percebi que vários erros passavam despercebidos quando eu fazia apenas testes manuais. Com o RSpec, muitos desses problemas eram identificados rapidamente, geralmente apontando exatamente qual comportamento havia falhado. Isso reduz bastante o tempo gasto procurando a origem de um bug.

O RSpec também aumenta a confiança durante alterações no código. Em projetos que estão em constante evolução, é comum precisar modificar funcionalidades já existentes. Sem testes, cada alteração pode gerar receio de quebrar alguma coisa sem perceber. Com uma boa suíte de testes, é possível validar rapidamente se as mudanças afetaram outras partes do sistema.

Além disso, a ferramenta incentiva boas práticas de programação. Como o código precisa ser testável, o desenvolvedor tende a criar métodos menores, responsabilidades mais bem definidas e uma estrutura mais organizada. Isso normalmente resulta em projetos mais fáceis de manter ao longo do tempo.

Por outro lado, o RSpec também apresenta algumas desvantagens. A primeira delas é a curva de aprendizado. Para quem está começando, conceitos como matchers, doubles, mocks, stubs e hooks podem parecer confusos. No início, é comum gastar mais tempo tentando entender o teste do que a própria funcionalidade que está sendo desenvolvida.

Outro ponto negativo é que escrever testes exige tempo. Em projetos pequenos ou protótipos simples, pode parecer que o desenvolvedor está fazendo o trabalho em dobro, já que precisa criar a funcionalidade e depois criar testes para validá-la. Embora esse investimento geralmente compense no longo prazo, nem sempre essa vantagem é percebida imediatamente.

Também existe o risco de criar testes ruins, mas sinceramente neste caso o problema está mais entre a cadeira e o computador.

Por fim, em projetos muito grandes, uma suíte extensa de testes pode aumentar o tempo necessário para executar todas as verificações. Embora existam estratégias para minimizar esse problema, ele ainda pode impactar a produtividade em determinadas situações.

No geral, as vantagens do RSpec costumam superar suas desvantagens, especialmente em projetos que possuem manutenção contínua e crescimento constante. Apesar do esforço inicial necessário para aprender a ferramenta e escrever os testes, os benefícios relacionados à qualidade do software, redução de bugs e segurança durante o desenvolvimento geralmente compensam esse investimento.

Considerações Finais

Ao longo deste material foi possível compreender a importância dos testes automatizados no desenvolvimento de software e como o RSpec se tornou uma das

principais ferramentas para esse propósito dentro do ecossistema Ruby. Sua proposta de descrever comportamentos de forma clara e legível facilita tanto a criação quanto a manutenção dos testes, contribuindo para a qualidade do código e para a redução de erros durante o desenvolvimento.

Além dos aspectos teóricos, foram apresentados exemplos práticos que demonstram como o RSpec pode ser utilizado para validar o funcionamento de classes, métodos e diferentes cenários dentro de uma aplicação. Também foram discutidas algumas de suas principais ferramentas, como **describe**, **it**, **expect**, **context** e os **matchers**, que formam a base da maioria dos testes escritos com a biblioteca.

Durante minha jornada de aprendizado em Ruby, especialmente através do The Odin Project e do desenvolvimento de projetos pessoais, pude perceber na prática a importância dos testes automatizados. Em diversos momentos, erros que seriam difíceis de encontrar manualmente foram identificados rapidamente pelos testes, mostrando que o tempo investido na sua criação pode evitar muitas horas de depuração no futuro.

Apesar de possuir algumas limitações e exigir um período de adaptação para quem está começando, o RSpec continua sendo uma ferramenta extremamente relevante para desenvolvedores Ruby. Seu uso contribui para aplicações mais confiáveis, facilita a manutenção de sistemas e oferece maior segurança na implementação de novas funcionalidades. Dessa forma, seu estudo se mostra fundamental para qualquer pessoa interessada em desenvolver software de forma mais profissional e sustentável.

Bibliografia

FATEC ARARAS. Faculdade de Tecnologia de Araras. Disponível em: <https://fatecararas.cps.sp.gov.br/>. Acesso em: 09 jun. 2026.

SARAIVA JUNIOR, Orlando. *ebook_qualidade_teste_software*. GitHub. Disponível em: https://github.com/orlandosaraivajr/ebook_qualidade_teste_software. Acesso em: 09 jun. 2026.

CHELIMSKY, David; HASHIMOTO, Yuki. *The RSpec Book: Behaviour Driven Development with RSpec, Cucumber, and Friends*. Dallas: Pragmatic Bookshelf, 2010.

THOMAS, Dave; HUNT, Andy. *Programming Ruby 3.3: The Pragmatic Programmers' Guide*. Raleigh: Pragmatic Bookshelf, 2024.

RSpec Team. *RSpec Documentation*. Disponível em: <https://rspec.info>. Acesso em: 09 jun. 2026.

Ruby Community. *Ruby Documentation*. Disponível em: <https://www.ruby-lang.org>. Acesso em: 09 jun. 2026.

The Odin Project. *Ruby Course*. Disponível em: <https://www.theodinproject.com/>. Acesso em: 09 jun. 2026.

A ferramenta XYZ

Abner de Souza
abnersouza26@gmail.com
abner.souza@fatec.sp.gov.br

Resumo

A garantia de qualidade de software e a manutenção de uma base de código legível e segura são etapas indispensáveis no ciclo de vida de aplicações modernas. Este trabalho define como tema central a análise estática de código utilizando a ferramenta PHPStan, com foco delimitado na sua aplicação e configuração dentro do projeto ABN-Horse.

O objetivo geral é avaliar a eficácia do PHPStan como um verificador de tipos e identificador de potenciais bugs no código antes mesmo da execução, garantindo maior estabilidade de ponta a ponta. Os objetivos específicos incluem a análise da sua configuração atual, a avaliação do nível de rigor aplicado (Level 8) e o impacto dessa tecnologia na experiência de desenvolvimento e segurança do sistema. A justificativa baseia-se na necessidade de modernizar e otimizar rotinas de engenharia no PHP.

A metodologia aplicada envolveu o estudo do arquivo de configuração, a delimitação dos diretórios críticos do sistema e a execução de análises comparativas de níveis de tipagem.

Palavras-chaves: PHPStan. Análise Estática de Código. Qualidade de Software. PHP. ABN-Horse

Introdução

A garantia de qualidade de software e a manutenção de uma base de código legível e segura são etapas indispensáveis no ciclo de vida de aplicações modernas.

Na introdução deste trabalho, define-se como tema central a análise estática de código utilizando a ferramenta PHPStan, com foco delimitado na sua aplicação e configuração dentro do projeto ABN-Horse.

O objetivo geral é avaliar a eficácia do PHPStan como um verificador de tipos e identificador de potenciais bugs no código antes mesmo da execução, garantindo maior estabilidade de ponta a ponta.

Os objetivos específicos incluem a análise da sua configuração atual, a avaliação do nível de rigor aplicado (Level 8) e o impacto dessa tecnologia na experiência de desenvolvimento e segurança do sistema.

A justificativa para a escolha do tema baseia-se na necessidade de modernizar e otimizar

rotinas de engenharia no PHP, garantindo entregas ágeis e evitando regressões no código de forma proativa.

A metodologia aplicada envolveu o estudo do arquivo de configuração (phpstan.neon), a delimitação dos diretórios críticos do sistema e a execução de análises comparativas de níveis de tipagem.

O trabalho está organizado na seguinte estrutura: introdução, desenvolvimento detalhando as características de configuração e regras de negócio avaliadas, e considerações finais

Desenvolvimento

Configuração Inicial

O PHPStan foi introduzido no projeto ABN-Horse como dependência de desenvolvimento. A configuração centralizada ocorre através do arquivo phpstan.neon, que dita o escopo e as regras que a ferramenta deve seguir ao avaliar o código.

Existe um script personalizado no projeto configurado para gerar um relatório de fácil leitura e formatado, além de permitir a comparação de relatórios e totais de erros.

Estrutura de Análise e Escopo

Para garantir que a análise seja eficiente e direcionada onde há maior necessidade de controle de regras de negócio, o escopo foi delimitado para focar em três pilares principais da aplicação ABN-Horse:

- app/Nucleo/ (Core e regras centrais)
- app/Modelos/ (Representação dos dados e persistência)
- app/Controladores/ (Lógica de requisições e respostas)

Esses diretórios são o coração do sistema ABN-Horse, e garantir que não haja erros de tipagem neles previne falhas críticas em tempo de execução.

Nível de Rigor e Boas Práticas (Level 8)

A ferramenta opera em diferentes níveis de rigor (do 0 ao 9). No projeto ABN-Horse, o PHPStan foi configurado no Nível 8.

Este é um dos níveis mais altos e estritos da ferramenta, focando ativamente em verificação profunda em tipos iteráveis (arrays e coleções), garantia de que propriedades e métodos sejam chamados corretamente, e exigência de retornos documentados ou tipados nativamente.

Além disso, a configuração foi ajustada para obrigar que tipos garantidos pelo DocBlock não sejam cegamente aceitos sem verificação em tempo de execução, aumentando a confiabilidade da integração com bancos de dados e APIs externas.

Bootstrap e Exceções Controladas

Antes de cada análise estática, o PHPStan carrega o arquivo de inicialização. Isso permite que constantes, variáveis globais do framework ou autoloader personalizados sejam carregados para o contexto do PHPStan não emitir falsos positivos.

Como parte de um processo de migração e reestruturação, algumas integrações incompletas ou não utilizadas no momento (como integrações de gateway de pagamento) foram temporariamente isoladas da validação. Isso permite manter a fluidez de entrega contínua sem poluir o relatório de erros com arquivos ainda em desenvolvimento inicial

Execução da Análise e Relatório Customizado

A execução no dia a dia do desenvolvedor se dá pelo acionamento de um script próprio criado em PHP para organizar a saída e torná-la mais amigável. A ferramenta percorre todos os arquivos configurados, analisa o nível 8 e agrupa as ocorrências por tipo de erro.

Conforme evidenciado pelas análises, o script consolida problemas recorrentes. Abaixo estão as representações da execução do comando agrupando ocorrências:

Figura 1 – Visão geral e problemas de tipagem. A análise começa trazendo um resumo e alertando problemas como incompatibilidade de argumentos (argument.type), fundamental para impedir falhas de lógicas.

```
PS C:\Users\Abner Souza\Documents\GitHub\ABN-Horse Orlando> php tools\phpstan-relatorio.php
>>
PHPStan - relatório legível (nível 8, escopo em phpstan.neon)
=====
Total de avisos: 163

▸ Tipo: argument.type
  O que é: Tipo de argumento incompatível com o esperado.
  Ocorrências: 16

  Arquivo: app/Controladores/ControladorPagamento.php:102
  Mensagem: Parameter #1 $json of function json_decode expects string, string|false given.
```

Fonte: próprio autor

Figura 2 – Mapeamento de ausência de tipos. O relatório mapeia minuciosamente onde a base de código falha em declarar os retornos corretamente (missingType.return), uma regra estrita do nível 8.


```
-----
> Tipo: missingType.iterableValue
O que é: Array/iterável sem tipo dos elementos: documento com PHPDoc (ex.: list<Tipo> ou array<string, mixed>) em vez de só array.
Ocorrências: 66

Arquivo: app/Controladores/ControladorAdmin.php:569
Mensagem: Method ControladorAdmin::bannersSiteVazios() return type has no value type specified in iterable type array.
Dica: See: https://phpstan.org/blog/solving-phpstan-no-value-type-specified-in-iterable-type
```

Fonte: próprio autor

Figura 3 – Controle rígido de Variáveis Indefinidas. O rigor é tamanho que o PHPStan acusa até variáveis que, dependendo de desvios do fluxo, podem chegar em uma instrução sem estarem declaradas.

```
Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\Abner Souza\Documents\GitHub\ABN-Horse Orlando> php tools\phpstan-relatorio.php
>>

> Tipo: variable.undefined
O que é: Variável possivelmente indefinida – em algum fluxo ela pode ser lida sem ter sido atribuída.
Ocorrências: 2

Arquivo: app/Controladores/ControladorAdmin.php:182
Mensagem: Variable $bannerAtual might not be defined.

Arquivo: app/Controladores/ControladorAdmin.php:196
Mensagem: Variable $bannerAtual might not be defined.
```

Fonte: próprio autor

Análise de Resultados

A análise foca na leitura inteligente do relatório agrupado. Ele aponta com exatidão a regra de qualidade que foi violada, o nome e o caminho do arquivo afetado, juntamente com a linha exata e a mensagem original da ferramenta orientando o ajuste.

Isso cria um ciclo de feedback rápido onde o desenvolvedor corrige os problemas estruturais de forma cirúrgica.

Considerações Finais

A estruturação do PHPStan no ABN-Horse, aliada a um formatador de relatórios próprio, evidenciou que a integração de ferramentas de análise estática avançada transforma a gestão de qualidade no PHP. Ao focar em diretórios chave (Núcleo, Modelos e Controladores) no Nível 8, o projeto demonstra uma maturidade técnica robusta.

Como principais conclusões práticas, destaca-se a precisão do diagnóstico sem a necessidade de simular interações complexas de interface. O agrupamento de erros facilita para a equipe entender onde estão os gargalos do projeto (como a forte ausência de tipos de retorno observada nas evidências).

Para os próximos trabalhos futuros, recomenda-se a redução drástica das ocorrências listadas, aplicando as tipagens estritas recomendadas pela ferramenta, visando atingir o número zero de avisos no escopo configurado.

Por fim, conclui-se que o PHPStan atua como um revisor automático rigoroso, agregando confiança imediata para as entregas e a estabilidade da aplicação em todas as suas fases.

Bibliografia

PHPSTAN. Getting Started - PHPStan Documentation. Disponível em: <https://phpstan.org/user-guide/getting-started>. Acesso em: 09 jun. 2026.

PHPSTAN. Rule Levels. Disponível em: <https://phpstan.org/user-guide/rule-levels>. Acesso em: 09 jun. 2026.

PHPSTAN. Config Reference. Disponível em: <https://phpstan.org/config-reference>. Acesso em: 09 jun. 2026.

PHPSTAN. Ignoring Errors. Disponível em: <https://phpstan.org/user-guide/ignoring-errors>. Acesso em: 09 jun. 2026.

PHPSTAN. Discovering Symbols. Disponível em: <https://phpstan.org/user-guide/discoveringsymbols>. Acesso em: 09 jun. 2026.

Jest e Supertest

Bruno Henrique Guinério
brunoguinerio1204@gmail.com
bruno.guinerio@aluno.cps.sp.gov.br

Resumo

Os testes automatizados são uma prática importante no desenvolvimento de software, pois ajudam a identificar falhas e garantir o correto funcionamento das aplicações. Este capítulo apresenta o Jest e o Supertest, duas ferramentas amplamente utilizadas para a criação e execução de testes em aplicações desenvolvidas com Node.js e NestJS. Inicialmente, são abordados os conceitos básicos e o funcionamento dessas ferramentas, destacando como podem ser utilizadas em conjunto para validar endpoints e regras de negócio. Em seguida, são apresentados exemplos práticos de execução de testes End-to-End, demonstrando a validação de funcionalidades da aplicação. Também é abordada a geração de relatórios de cobertura de código por meio do Jest, permitindo analisar quais partes do sistema estão sendo efetivamente testadas. Os resultados demonstram que a utilização dessas ferramentas contribui para a criação de aplicações mais confiáveis, auxiliando na identificação de erros e na melhoria da qualidade do software durante o processo de desenvolvimento.

Palavras-chaves: Jest. Supertest. Testes automatizados. NestJS. Cobertura de código. API.

Introdução

Os testes automatizados são uma parte importante do desenvolvimento de software, pois ajudam a identificar falhas, validar funcionalidades e garantir que alterações no código não afetem comportamentos já existentes. Dessa forma, eles contribuem para a criação de aplicações mais confiáveis e de fácil manutenção.

No desenvolvimento de aplicações com NestJS, duas ferramentas bastante utilizadas para esse propósito são o Jest e o Supertest. O Jest é responsável pela criação e execução dos testes, enquanto o Supertest permite realizar requisições HTTP para validar o funcionamento de APIs. A combinação dessas ferramentas possibilita a realização de testes completos, simulando situações reais de uso da aplicação.

Este capítulo tem como objetivo apresentar os conceitos básicos do Jest e do Supertest, destacando seu funcionamento, benefícios e aplicação prática em uma API desenvolvida com

NestJS. Também será abordada a geração de relatórios de cobertura de código utilizando os recursos disponibilizados pelo Jest, demonstrando como essas ferramentas podem auxiliar na melhoria da qualidade do software.

Desenvolvimento

Jest e Supertest

O Jest é um framework de testes utilizado em aplicações JavaScript e TypeScript. Sua principal função é permitir a criação e execução de testes automatizados, auxiliando na validação do comportamento da aplicação.

Já o Supertest é uma biblioteca utilizada para realizar requisições HTTP durante os testes. Com ela é possível testar endpoints da API de forma semelhante ao que acontece quando um usuário utiliza o sistema.

Quando utilizadas em conjunto, essas ferramentas permitem criar testes completos para validar regras de negócio, autenticação, autorização e operações realizadas pela aplicação.

Funcionamento

O Jest executa os arquivos de teste e verifica se os resultados obtidos correspondem aos resultados esperados.

O Supertest atua enviando requisições HTTP para a aplicação e capturando as respostas retornadas pelos endpoints.

O fluxo de execução ocorre da seguinte forma:

1. O Jest inicia o ambiente de testes;
2. O NestJS cria uma instância da aplicação;
3. O Supertest envia requisições para os endpoints;
4. O Jest valida as respostas recebidas;
5. Os resultados são exibidos no terminal.

Adicionando as dependências

Para utilizar o Supertest em projetos NestJS é necessário instalar a biblioteca através do npm. Os comandos necessários para instalar as dependências são:

- `npm install --save-dev supertest;`
- `npm install --save-dev @types/supertest.`

Executando os testes

Após criar os arquivos de teste, o Jest pode executá-los através do seguinte comando:

- `npm run test:e2e.`

Ao final da execução será exibido um relatório semelhante ao exemplo abaixo:

```
PASS test/task.e2e-spec.ts (9.047 s)
PASS test/user.e2e-spec.ts (9.283 s)

Test Suites: 2 passed, 2 total
Tests:       16 passed, 16 total
Snapshots:   0 total
Time:        11.089 s
Ran all test suites.
```

Fonte: próprio autor

Entendendo o resultado dos testes

Ao executar os testes, o Jest fornece algumas informações importantes:

- Test Suites: Quantidade de arquivos de testes executados;
- Tests: Quantidade total de cenários testados;
- Snapshots: Indica a quantidade de snapshots utilizados durante os testes. Como essa funcionalidade é normalmente aplicada em interfaces front-end, seu uso não foi necessário nos testes de API desenvolvidos neste trabalho;
- Time: Tempo total de execução dos testes.

Cobertura de código

Além da execução dos testes, o Jest também possui suporte nativo para geração de relatórios de cobertura. Para gerar o relatório basta executar:

- `npm run test:e2e -- --coverage`.

Após a execução do comando, o terminal exibirá um resumo semelhante ao seguinte:

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	85.71	64.16	97.82	86.32	
src	100	100	100	100	
app.module.ts	100	100	100	100	
src/common/db/prisma	100	75	100	100	
prisma.module.ts	100	100	100	100	
prisma.service.ts	100	75	100	100	13
src/common/functions	100	100	100	100	
make-hash.ts	100	100	100	100	
src/modules/auth	95.83	72.72	88.88	95.23	
auth.controller.ts	90.9	75	66.66	88.88	22
auth.module.ts	100	50	100	100	19
auth.service.ts	95.83	75	100	95.45	26
src/modules/auth/strategies	92.3	68.75	100	95.23	
jwt.strategy.ts	92.3	70	100	90.9	16
local.strategy.ts	92.3	66.66	100	100	10-16
src/modules/task	75.67	53.57	100	76.19	
task.controller.ts	100	75	100	100	15-40
task.module.ts	100	100	100	100	
task.service.ts	63.26	37.5	100	64.28	30-31,41-42,55-57,69-71,83-84,96-98
src/modules/task/dto	100	75	100	100	
create-task.dto.ts	100	75	100	100	12
update-task.dto.ts	100	75	100	100	12
src/modules/user	81.81	59.37	100	81.15	
user.controller.ts	100	75	100	100	17-52
user.module.ts	100	100	100	100	
user.service.ts	72	50	100	71.73	40-41,55-56,69-70,88-93,106-107
src/modules/user/dto	79.31	70	100	85.18	
create-user.dto.ts	100	100	100	100	
update-user.dto.ts	100	100	100	100	
user-response.dto.ts	68.42	70	100	76.47	13-17

Fonte: próprio autor

O Jest disponibiliza algumas métricas para a análise:

- Statements: Indica a porcentagem de instruções executadas;
- Branches: Indica a porcentagem de estruturas condicionais executadas;
- Functions: Representa a porcentagem de funções executadas;
- Lines: Representa a porcentagem de linhas executadas;
- Uncovered lines: Representa as linhas que não foram executadas.

Além das informações exibidas no terminal, o Jest gera automaticamente um relatório HTML localizado em `coverage/lcov-report/index.html`. Ao abrir esse arquivo o navegador exibirá as informações detalhadas de testes sobre cada arquivo da aplicação.

All files

85.71% Statements 252/294 64.16% Branches 77/128 97.82% Functions 45/46 86.32% Lines 221/256

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
src	100%	9/9	100%	7/7
src/common/db/prisma	100%	18/18	75%	14/14
src/common/functions	100%	3/3	100%	3/3
src/modules/auth	95.83%	46/48	72.72%	40/42
src/modules/auth/strategies	92.3%	24/26	68.75%	20/21
src/modules/task	75.67%	56/74	53.57%	48/63
src/modules/task/dto	100%	10/10	75%	10/10
src/modules/user	81.81%	63/77	59.37%	56/69
src/modules/user/dto	79.31%	23/29	70%	23/27

Fonte: próprio autor

Exemplo prático

A seguir será apresentado um exemplo de teste utilizado para validar a criação de usuários.

```
it("should create user", async () => {
  const res = await request(app.getHttpServer())
    .post("/user")
    .send(userData)
    .expect(201);

  expect(res.body.user).toMatchObject({
    name: userData.name,
    id: expect.any(String),
  });

  expect(res.body.user.email).not.toBe(userData.email);
});
```

Fonte: próprio autor

O teste envia uma requisição para o endpoint de cadastro e verifica se o usuário foi criado corretamente.

Considerações Finais

Neste capítulo foram apresentados os principais conceitos sobre o Jest e o Supertest, duas ferramentas bastante utilizadas para a realização de testes automatizados em aplicações desenvolvidas com Node.js e NestJS.

Também foi possível compreender como essas ferramentas funcionam em conjunto, desde a execução dos testes até a geração dos relatórios de cobertura de código. Além disso, foram apresentados exemplos práticos que demonstram como validar o funcionamento de endpoints e regras de negócio da aplicação.

Os relatórios de cobertura mostraram a importância de analisar quais partes do sistema estão sendo testadas, ajudando a identificar pontos que podem necessitar de novos testes.

Dessa forma, o uso do Jest e do Supertest contribui para o desenvolvimento de aplicações mais confiáveis, facilitando a identificação de erros e aumentando a qualidade do software ao longo do processo de desenvolvimento.

Bibliografia

JEST. Getting Started. Disponível em: <https://jestjs.io/docs/getting-started/>. Acesso em: 2 jun. 2026.

FORWARDEMAIL. Supertest. Disponível em: <https://github.com/forwardemail/supertest/>. Acesso em: 2 jun. 2026.

NestJS. End to End testing. Disponível em: <https://docs.nestjs.com/fundamentals/testing#end-to-end-testing/>. Acesso em: 3 jun. 2026.

TestSprite

Luiz Henrique Simionato Vicente

luizsimi455@gmail.com

luiz.vicente@aluno.cps.sp.gov.br

Resumo

Este documento apresenta um guia abrangente sobre a implementação e automação de testes de interface e experiência do usuário (UI/UX) utilizando a inteligência artificial autônoma do TestSprite. O escopo baseia-se no ciclo completo de desenvolvimento do projeto MinhaApp, construído com o ecossistema React e Vite. A documentação abrange desde a configuração inicial do ambiente e estruturação de páginas fundamentais como dashboards analíticos e formulários operacionais, até a aplicação de mecanismos de prevenção de erros (Poka-Yoke) para garantir a integridade da entrada de dados.

A metodologia detalha o bootstrap do TestSprite, a varredura automatizada do código-fonte e a geração de um plano com 10 cenários de testes prioritários. Os resultados evidenciam a eficácia da integração da IA, que executou os testes de forma autônoma e atingiu uma taxa de sucesso de 100%, validando fluxos de autenticação, responsividade visual e a resiliência dos componentes de interface. Conclui-se que a automação baseada em IA atua como um catalisador de qualidade de software, eliminando a sobrecarga de validações manuais e garantindo estabilidade e escalabilidade para o produto final.

Palavras-chaves: Ferramenta TestSprite. Testes Automatizados. CI/CD. Inteligência Artificial.

Introdução

A garantia de qualidade de software é uma etapa indispensável no ciclo de vida de aplicações modernas. Na introdução deste trabalho, define-se como tema central a automação de testes utilizando Inteligência Artificial, com foco delimitado nas capacidades e funcionalidades da ferramenta TestSprite. O objetivo geral é avaliar a eficácia do TestSprite como um agente autônomo (via protocolo MCP) capaz de gerenciar testes de ponta a ponta. Os objetivos específicos incluem a análise da sua integração contínua, a avaliação do recurso de autorrecuperação (*self-healing*) de testes falhos e o impacto dessa tecnologia na experiência do usuário (UX). A justificativa para a escolha do tema baseia-se na necessidade crescente de otimizar rotinas de engenharia, garantindo entregas ágeis e seguras sem onerar as equipes com testes repetitivos. A metodologia aplicada envolveu a pesquisa das funcionalidades da plataforma, a análise de sua capacidade de inferir intenções de projeto a partir do código e simulações de testes de regressão. O trabalho está organizado na seguinte estrutura: introdução, desenvolvimento detalhando as características e a jornada prática de aprendizado, e considerações finais.

Desenvolvimento

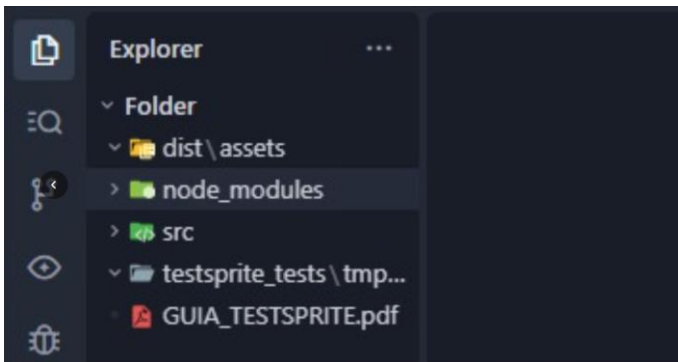
Fase 1: Configuração Inicial 1.1 Verificar Estrutura do Diretório Comando executado:

Comando executado:

```
cd c:\Users\Usuario\Desktop\projeto_qualidade  
ls -la
```

Resultado esperado: Diretório vazio ou com estrutura inicial Preparado para criar o projeto

Fase 1.1 - Diretório Inicial Descrição: Terminal mostrando o diretório projeto_qualidade vazio, pronto para iniciar o desenvolvimento.



Fase 1.2 - Verificar Node.js e npm

```
node --version  
npm --version
```

2. Fase 2: Criação do Projeto

2.1 Criar Projeto React com Vite: A execução do comando `npm create vite@latest ----template react-force` realiza o download e instalação do Vite, clona o template React, instala as dependências automaticamente e inicia o servidor de desenvolvimento.

```
31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
|
◇ Starting dev server...

> projeto_qualidade@0.0.0 dev
> vite

VITE v8.0.16 ready in 571 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

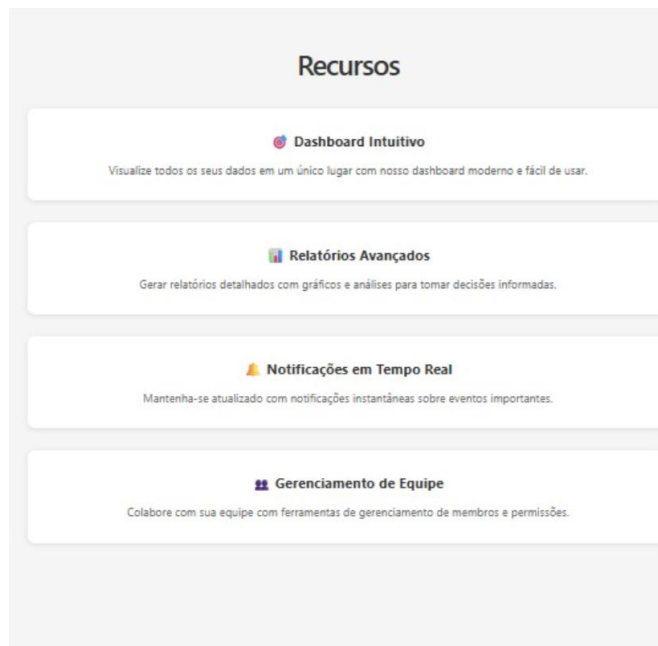
2.2 Estrutura Inicial do Projeto: O comando `tree -L 2` exibe a estrutura base criada, contendo os diretórios `public/`, `src/` e arquivos fundamentais como `vite.config.js` e `App.jsx`. (Referência: *Screenshot 4 - Estrutura de arquivos*).

2.3 Instalação: Finalizar a base rodando `npm install` para as dependências adicionais. (Referência: *Screenshot 5 - Instalação de dependências*).

3. Fase 3: Desenvolvimento das Páginas e UI/UX

O arquivo `src/App.jsx` concentra 5 páginas principais desenvolvidas:

- **Home Page:** Construída com hero section, gradiente de fundo, call-to-action buttons, cards de recursos em destaque e footer.
- **Página de Login:** Inclui formulário com campos de username e password, validação de credenciais, redirecionamento automático para o Dashboard após o login e link de "Esqueceu a senha?".
- **Página de Contato e Formulários Operacionais:**
 - Formulário estruturado com campos de Nome, Email e Mensagem.
 - Sistema de validação em tempo real com exibição imediata de mensagens de erro.
 - Implementação de máscaras de entrada rigorosas (como formatos numéricos específicos para marcadores de "KM" e dados formatados).
 - Limpeza automática dos dados (reset de formulário) imediatamente após a confirmação de envio para minimizar o risco de erro humano e envios duplicados.
 - Confirmação visual de envio e informações de contato na barra lateral.
- **Dashboard:** Ambiente protegido exibindo 4 cards de estatísticas (Usuários, Receita, Pedidos, Satisfação), indicadores de variação (positivo/negativo), navbar adaptada para usuário autenticado e botão de logout.
- **Estilização CSS (src/App.css):** Focada em design responsivo (mobile-first), gradientes modernos, animações, sistema de cores consistente e tipografia legível.



4. Fase 4: Configuração e Bootstrap do TestSprite

4.1 Estrutura de Configuração: O TestSprite, ao ser inicializado, gera a pasta `testsprite_tests/`. Esta pasta contém o `config.json`, o arquivo `code_summary.yaml` e a subpasta `prd_files/` com a documentação do produto e o PRD padronizado. As configurações base incluem `localPort: 4173`, `type: frontend` e `testScope: codebase`.

4.2 Análise Automática de Código: A ferramenta analisa todo o código fonte e documenta no `code_summary.yaml` a stack tecnológica utilizada (JavaScript ES6+, React 18, Vite, CSS3, HTML5), as rotas da aplicação (ex: `/`, `/features`, `/login`) e as features implementadas.

5. Fase 5: Geração do Plano de Teste (Foco em UI/UX)

Com base na análise do PRD e do código fonte, o TestSprite gera casos de teste priorizados dentro do arquivo `testsprite_frontend_test_plan.json`.

Casos de Teste Estruturados:

TC001 a TC004 (Alta Prioridade): Validação de "User Login" (acesso ao dashboard com credenciais válidas), "Home Page and Navigation", "Public Navigation" e funcionalidade de logout no "Protected Dashboard View".

TC005 (Alta Prioridade): Submissão de um formulário de contato válido.

TC006 (Alta Prioridade): Visualização correta dos cards de estatísticas no dashboard após login.

TC007 a TC009 (Média Prioridade): Tentativa de submissão de formulário com dados inválidos, tentativa de acesso ao dashboard sem login prévio e verificação de visibilidade de todos os elementos da Home Page.

TC010 (Baixa Prioridade): Redimensionamento do navegador para testar a responsividade da navegação.

TC011 (Alta Prioridade - Adicionado): Verificação da aplicação em tempo real de máscaras de entrada durante a digitação do usuário (ex: campos de quilometragem).

TC012 (Alta Prioridade - Adicionado): Validação do esvaziamento imediato dos campos

do formulário após submissão bem-sucedida, garantindo o reset do estado visual.

6. Fase 6: Execução dos Testes

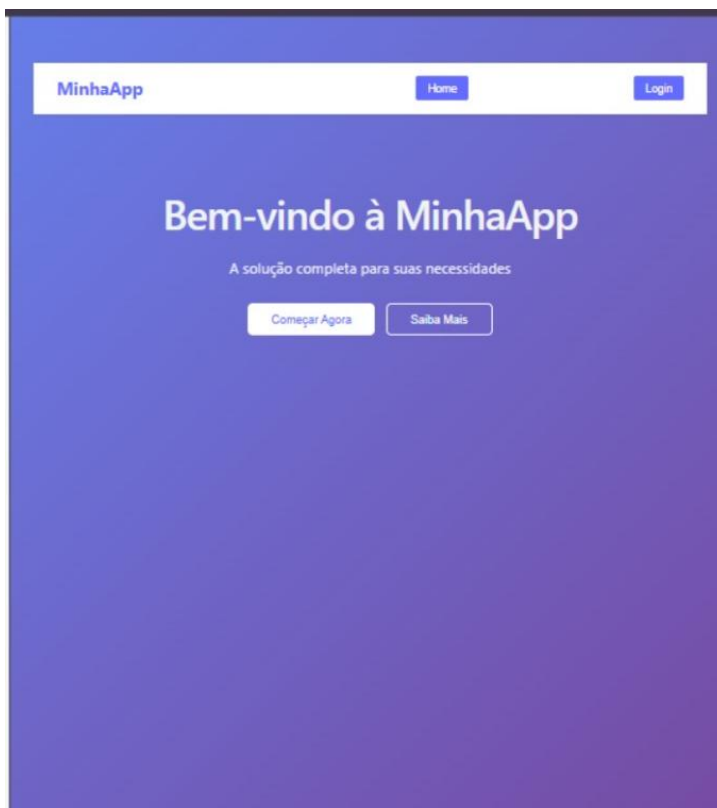
6.1 Build e Preview: O build de produção é gerado (gerando arquivos otimizados na pasta dist/) via `npm run build`. Em seguida, o servidor de preview é iniciado com `npm run preview`, disponibilizando a aplicação na porta 4173.

6.2 Execução Automatizada: O TestSprite lê o plano gerado, cria o código automatizado, executa a aplicação em um navegador headless, captura os logs e gera o relatório de resultados dinamicamente.

7. Fase 7: Análise de Resultados

7.1 Relatório de Testes: O arquivo `testsprite_tests/tmp/raw_report.md` é gerado contendo o resumo da execução, o status individual de cada caso de teste (Pass/Fail), logs de erro e métricas de cobertura.

7.2 Dashboard e Evidências: O Dashboard do TestSprite consolida a taxa de sucesso (ex: 100% de aprovação). As screenshots automáticas comprovam visualmente estados críticos como a página de login preenchida, mensagens de sucesso nos formulários e a visibilidade correta dos cards no Dashboard.



Considerações Finais

A estruturação e execução deste guia evidenciaram que a integração de agentes autônomos de Inteligência Artificial, como o TestSprite, transforma a gestão da qualidade de software, tornando-a proativa e altamente escalável. A ferramenta demonstrou ser um recurso estratégico ao ser incorporada aos pipelines de automação, eliminando os gargalos tradicionais dos testes manuais. Isso permite que a equipe de desenvolvimento direcione seu foco para a resolução de problemas arquiteturais complexos e para o aprimoramento contínuo da experiência do usuário.

Como principais conclusões práticas, destaca-se a alta precisão dos diagnósticos de falha e a resiliência das validações automatizadas de interface, especialmente em cenários rigorosos que exigem a aplicação de máscaras de entrada dinâmicas e o controle seguro de estados e limpeza de formulários.

Para os próximos passos e trabalhos futuros, recomenda-se a expansão da aplicação prática do TestSprite em ambientes operacionais mais robustos. Isso inclui a validação de módulos voltados para engenharia de campo e controle tecnológico avaliando o comportamento da interface frente à persistência offline de dados bem como testes de estresse visual e funcional em dashboards analíticos complexos.

Por fim, conclui-se que o uso de automação baseada em IA não substitui o rigor das boas práticas de engenharia de software e a necessidade de uma coordenação técnica atenta e bem fundamentada. Pelo contrário, a ferramenta atua como um acelerador indispensável para equipes que buscam garantir um padrão de excelência técnica, estabilidade e confiabilidade em suas entregas.

Bibliografia

FATEC ARARAS. Faculdade de Tecnologia de Araras. Disponível em:

<https://fatecararas.cps.sp.gov.br/>. Acesso em: 29 maio. 2026.

SARAIVA JUNIOR, Orlando. ebook_qualidade_teste_software. GitHub. Disponível em:

https://github.com/orlandosaraivajr/ebook_qualidade_teste_software. Acesso em: 29 maio. 2026.

TESTSPRITE. Documentation Overview. Disponível em: <https://docs.testsprite.com/>. Acesso em: 29 maio. 2026.

TESTSPRITE. AI Software Testing Tool. Disponível em: <https://www.testsprite.com/>. Acesso em: 29 maio. 2026.

Aplicação de Testes Automatizados com Pytest e Selenium

*Giovanny Lopes Ribeiro
Glopesribeiro04@gmail.com
Giovanny.ribeiro@aluno.cps.sp.gov.br*

Resumo

Este capítulo apresenta a aplicação das ferramentas Pytest e Selenium no projeto Cookie Clicker. O objetivo foi desenvolver testes automatizados para validar funcionalidades da aplicação web por meio da automação de interações no navegador. A metodologia consistiu no estudo das ferramentas e na implementação prática de casos de teste em Python. Foram realizados testes para verificar o carregamento da página, a interação com elementos e a execução de ações automatizadas. Os resultados demonstraram o funcionamento correto da aplicação e a eficiência das ferramentas na validação de sistemas web, contribuindo para o aprendizado sobre automação de testes e qualidade de software.

Palavras-chave: Pytest. Selenium. Automação de Testes. Python. Cookie Clicker.

Introdução

A automação de testes é uma prática importante para garantir a qualidade e o funcionamento correto de aplicações. Neste capítulo, será apresentada a utilização das ferramentas Pytest e Selenium por meio da experiência adquirida no desenvolvimento de testes automatizados para o projeto Cookie Clicker.

O objetivo foi compreender como criar e executar testes capazes de validar funcionalidades da aplicação de forma automática, simulando ações realizadas pelos usuários no navegador. Para isso, foram realizados estudos sobre as ferramentas e implementados testes práticos utilizando Python, Pytest e Selenium. A experiência permitiu desenvolver conhecimentos sobre automação, identificação de falhas e validação de sistemas web, destacando a importância dos testes no desenvolvimento de software..

Desenvolvimento

O projeto Cookie Clicker foi desenvolvido com o objetivo de aplicar conceitos de automação de testes utilizando Python, Pytest e Selenium. A aplicação consiste na automação de ações dentro do jogo

Cookie Clicker, simulando a interação de um usuário com a página web por meio do navegador.

Para a execução do projeto, foi necessário instalar algumas dependências. Primeiramente, deve-se instalar o Python em sua versão mais recente. Em seguida, as bibliotecas utilizadas podem ser instaladas através do comando:

```
C:\Users\Giovanny\Desktop\Cookie_Clicker>pip install selenium pytest webdriver-manager  
Collecting selenium
```

(instalação das dependências)

Após a instalação das dependências, o projeto pode ser executado utilizando os comandos do Pytest para validar o funcionamento dos testes automatizados.

```
3. Rodar o projeto  
python main.py  
Estrutura do projeto  
Cookie_Clicker/  
|  
├── main.py  
├── bot.py  
├── README.md  
└── requirements.txt (opcional)
```

(Estrutura do projeto)

Conforme observado na Figura 1, o projeto foi organizado de forma a separar os arquivos de teste e os recursos necessários para a automação. Essa organização facilita a manutenção do código e a criação de novos cenários de teste.

Na automação desenvolvida, o Selenium foi utilizado para controlar o navegador, acessar o jogo Cookie Clicker e realizar ações como clicar automaticamente no cookie principal e interagir com elementos da página. Já o Pytest foi responsável pela execução e validação dos testes, permitindo verificar se o comportamento esperado estava sendo alcançado.

Durante o desenvolvimento do projeto, foi possível compreender conceitos importantes relacionados à automação de testes web, localização de elementos por XPath e ID, sincronização de ações e estruturação de casos de teste. Além disso, a experiência proporcionou maior familiaridade com o uso do Selenium e do Pytest em aplicações reais, demonstrando como essas ferramentas podem auxiliar na garantia da qualidade de software.



(Rodando projeto)

A estrutura de testes do projeto é composta por quatro casos de teste principais: `test_abre_site`, responsável por verificar se o site Cookie Clicker é carregado corretamente; `test_cookie_existe`, que valida a presença do cookie principal na página; `test_clicar_cookie`, que testa o funcionamento dos cliques automáticos e a atualização do contador de cookies; e `test_comprar_upgrade`, que verifica a tentativa de compra de melhorias após a obtenção de uma quantidade mínima de cookies. Além disso, a `fixture bot` é utilizada para inicializar e encerrar o navegador automaticamente, garantindo que cada teste seja executado em um ambiente limpo e controlado. Essa organização permitiu validar as funcionalidades mais importantes da automação de forma estruturada e eficiente.

```
import pytest
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from cookie_clicker import CookieClicker

@pytest.fixture
def bot():
    bot = CookieClicker()
    yield bot
    bot.fechar()

def test_abre_site(bot):
    bot.abrir_site()

    WebDriverWait(bot.driver, 10).until(
        lambda d: "Cookie Clicker" in d.title
    )
    assert "Cookie Clicker" in bot.driver.title
```

(testes)

Após a implementação dos testes, foi realizada sua execução por meio do comando `pytest -v`, responsável por executar todos os casos de teste do projeto e exibir informações detalhadas sobre os resultados. Conforme apresentado na Figura 4, os quatro testes desenvolvidos foram executados com sucesso, incluindo a validação da abertura do site, a verificação da existência do cookie principal, a execução dos cliques automáticos e a tentativa de compra de upgrades. O resultado "PASSED" em

todos os cenários demonstrou que as funcionalidades implementadas estavam operando conforme o esperado, evidenciando a eficiência da automação de testes na validação e garantia.

```
tests\test_cookie_clicker.py::test_abre_site PASSED [100%]  
tests\test_cookie_clicker.py::test_cookie_existe PASSED [100%]  
tests\test_cookie_clicker.py::test_clicar_cookie PASSED [100%]  
tests\test_cookie_clicker.py::test_comprar_upgrade PASSED [100%]
```

(testes que passaram nos parâmetros)

Considerações Finais

O desenvolvimento do projeto Cookie Clicker possibilitou a aplicação prática dos conceitos de automação de testes utilizando Pytest e Selenium. Durante sua execução, foi possível compreender a importância dos testes automatizados na validação de funcionalidades, além de adquirir conhecimentos sobre interação com elementos web, estruturação de testes e utilização de boas práticas no desenvolvimento de software.

Os resultados obtidos demonstraram a eficiência das ferramentas estudadas, com todos os testes sendo executados com sucesso. Como continuidade do projeto, podem ser implementados novos cenários de teste, relatórios automatizados e melhorias na organização do código, tornando a automação mais robusta e próxima das soluções utilizadas no mercado de tecnologia.

Bibliografia

PYTEST. Documentation. Disponível em: <https://docs.pytest.org/en/stable/>. Acesso em: 09 jun. 2026.

[Selenium Documentation](#). Acesso em: 09 jun. 2026.

[Cookie Clicker Project Repository](#). Acesso em: 09 jun. 2026.

[Python Documentation](#). Acesso em: 09 jun. 2026.

FATEC ARARAS. Faculdade de Tecnologia de Araras. Disponível em: <https://fatecararas.cps.sp.gov.br/>. Acesso em: 09 jun. 2026.

JUnit e Mockito

Marcelo Ferreira Miranda
marcelofmiranda23@gmail.com
marcelo.miranda@aluno.cps.sp.gov.br

Resumo

Os testes automatizados são uma prática importante no desenvolvimento de software, pois ajudam a identificar falhas e garantir o correto funcionamento das aplicações. Este capítulo apresenta o JUnit e o Mockito, duas ferramentas amplamente utilizadas para a criação e execução de testes em aplicações desenvolvidas com Java e Spring Boot. Inicialmente, são abordados os conceitos básicos e o funcionamento dessas ferramentas, destacando como podem ser utilizadas em conjunto para validar regras de negócio e comportamentos esperados da aplicação. Em seguida, são apresentados exemplos práticos de execução de testes unitários, demonstrando a validação de funcionalidades de forma isolada. Os resultados demonstram que a utilização dessas ferramentas contribui para a criação de aplicações mais confiáveis, auxiliando na identificação de erros e na melhoria da qualidade do software durante o processo de desenvolvimento.

Palavras-chaves: JUnit. Mockito. Testes automatizados. Spring Boot. Testes unitários.

Introdução

Os testes automatizados são uma parte importante do desenvolvimento de software, pois ajudam a identificar falhas, validar funcionalidades e garantir que alterações no código não afetem comportamentos já existentes. Dessa forma, eles contribuem para a criação de aplicações mais confiáveis e de fácil manutenção.

No desenvolvimento de aplicações com Java e Spring Boot, duas ferramentas bastante utilizadas para esse propósito são o JUnit e o Mockito. O JUnit é responsável pela criação e execução dos testes, enquanto o Mockito permite simular dependências externas, possibilitando testar unidades de código de forma isolada. A combinação dessas ferramentas possibilita a realização de testes completos, simulando situações

controladas de uso da aplicação sem depender de banco de dados ou serviços externos.

Este capítulo tem como objetivo apresentar os conceitos básicos do JUnit e do Mockito, destacando seu funcionamento, benefícios e aplicação prática em uma API desenvolvida com Spring Boot. Também será abordado o posicionamento dessas ferramentas dentro dos Quadrantes de Testes Ágeis, demonstrando como essas ferramentas podem auxiliar na melhoria da qualidade do software.

Desenvolvimento

O JUnit é um framework de testes utilizado em aplicações Java e Spring Boot. Sua principal função é permitir a criação e execução de testes automatizados, auxiliando na validação do comportamento da aplicação.

Já o Mockito é uma biblioteca utilizada para criar objetos simulados, conhecidos como mocks. Com ela é possível substituir dependências reais por versões controladas durante os testes, permitindo isolar o comportamento da classe que está sendo testada.

Quando utilizadas em conjunto, essas ferramentas permitem criar testes completos para validar regras de negócio, fluxos de criação e atualização de entidades, além de cenários de erro esperados pela aplicação.

Os Quadrantes de Testes Ágeis

Os Quadrantes de Testes Ágeis, propostos por Brian Marick e popularizados por Lisa Crispin e Janet Gregory, organizam os diferentes tipos de testes em quatro quadrantes de acordo com dois eixos: o foco do teste (voltado ao negócio ou voltado à tecnologia) e o objetivo (apoiar a equipe ou criticar o produto).

O JUnit e o Mockito se posicionam no Quadrante 1 (Q1), que agrupa testes voltados à tecnologia e com foco em apoiar a equipe de desenvolvimento. São testes executados com frequência, de forma automatizada, e que fornecem feedback rápido sobre a qualidade interna do código. Enquanto o Q2 abriga testes funcionais voltados ao negócio, o Q3 contempla testes exploratórios conduzidos por pessoas, e o Q4 engloba testes de desempenho e segurança, o Q1 é onde vivem os testes unitários e de integração que garantem que cada peça do código funciona corretamente de forma isolada.

Funcionamento

O JUnit executa os métodos anotados com `@Test` e verifica se os resultados

obtidos correspondem aos resultados esperados, utilizando métodos de asserção como `assertEquals`, `assertThat` e `assertThatThrownBy`.

O Mockito atua criando objetos simulados das dependências da classe testada. Por meio das anotações `@Mock` e `@InjectMocks`, é possível injetar os mocks automaticamente na classe de serviço, controlando o comportamento esperado com o método `when(...).thenReturn(...)`.

O fluxo de execução ocorre da seguinte forma:

1. O JUnit inicializa o contexto de testes com a extensão `@ExtendWith(MockitoExtension.class)`;
2. O Mockito cria os objetos simulados anotados com `@Mock`;
3. Os mocks são injetados na classe de serviço anotada com `@InjectMocks`;
4. O comportamento dos mocks é configurado com `when(...).thenReturn(...)`;
5. O método a ser testado é executado;
6. O JUnit valida os resultados por meio das asserções;
7. Os resultados são exibidos no terminal.

Adicionando as dependências

Para utilizar o JUnit e o Mockito em projetos Spring Boot, basta adicionar a dependência de testes no arquivo `pom.xml`. O Spring Boot já inclui ambas as ferramentas por meio do starter de testes:

- `mvn test` (para executar os testes via Maven);
- `./gradlew test` (para executar os testes via Gradle).

Executando os testes

Após criar os arquivos de teste, o Maven pode executá-los através do seguinte comando:

- `mvn test`

Ao final da execução será exibido um relatório semelhante ao exemplo abaixo:

```
[INFO] -----  
[INFO]  T E S T S  
[INFO] -----  
[INFO] Running com.Soucelo.service.UserServiceTest  
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0  
[INFO]  
[INFO] Results:  
[INFO]
```

```
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
```

Fonte: próprio autor

Entendendo o resultado dos testes

Ao executar os testes, o Maven fornece algumas informações importantes:

- **Tests run:** Tests run: Quantidade total de testes executados;
- **Failures:** Failures: Quantidade de testes que falharam por asserção incorreta;
- **Errors:** Errors: Quantidade de testes que lançaram exceções inesperadas durante a execução;
- **Skipped:** Skipped: Quantidade de testes ignorados com a anotação `@Disabled`;
- **BUILD SUCCESS / FAILURE:** BUILD SUCCESS / FAILURE: Indica se a execução foi bem-sucedida ou não.

Exemplo prático

A seguir será apresentado um exemplo de teste utilizado para validar a criação de usuários.

```
@Test
void createUser_shouldCreateUser_whenHasValidData() {
    when(repository.existsByEmail("joao@gmail.com"))
        .thenReturn(false);
    when(repository.existsByCpf("123.567.789-00"))
        .thenReturn(false);

    service.createUser(userValidRequest());

    verify(repository, times(1))
        .createUser(any(User.class));
}
```

Fonte: próprio autor

O teste configura os mocks para retornar false nas verificações de e-mail e CPF já existentes, executa o método de criação e verifica se o repositório foi chamado exatamente uma vez para persistir o usuário. Também é possível validar cenários de erro:

```
@Test
void createUser_shouldThrowException_whenEmailAlreadyExists() {
    when(repository.existsByEmail("joao@gmail.com"))
        .thenReturn(true);

    assertThatThrownBy(() -> service.createUser(userValidRequest()))
        .assertInstanceOf(RuntimeException.class)
}
```

```
.hasMessage("Email already exists");

verify(repository, never()).createUser(any());
}
```

Fonte: próprio autor

Nesse caso, o mock é configurado para retornar true na verificação de e-mail, simulando que o endereço já está cadastrado. O teste garante que uma exceção do tipo `RuntimeException` é lançada com a mensagem correta e que o repositório nunca é chamado para persistir o usuário.

Limitações do JUnit e do Mockito

Apesar de serem ferramentas poderosas para a validação de regras de negócio, o JUnit e o Mockito possuem limitações importantes que devem ser consideradas durante a definição da estratégia de testes de uma aplicação.

Por serem ferramentas voltadas ao Quadrante 1, seu foco está na validação de unidades isoladas de código. Isso significa que elas não são capazes de validar, por exemplo, a integração real com o banco de dados, o comportamento de APIs externas, fluxos completos da aplicação ou questões relacionadas à performance e segurança. Como os mocks substituem as dependências reais, eventuais problemas de integração entre os componentes da aplicação podem passar despercebidos durante os testes unitários.

Essas lacunas são justamente cobertas pelos demais quadrantes de testes. O Quadrante 2 abriga testes funcionais e de aceitação, que validam o comportamento da aplicação do ponto de vista do negócio, utilizando ferramentas como Selenium e Cypress. O Quadrante 3 contempla testes exploratórios e de usabilidade, conduzidos por pessoas, que identificam problemas que os testes automatizados não conseguem detectar. Já o Quadrante 4 engloba testes de desempenho, carga e segurança, realizados com ferramentas como JMeter, que garantem que a aplicação se comporta adequadamente sob condições adversas.

Dessa forma, o JUnit e o Mockito não devem ser vistos como a única solução de testes de uma aplicação, mas sim como uma parte fundamental de uma estratégia mais ampla, onde cada quadrante contribui com um tipo diferente de validação, tornando o software mais confiável em todas as suas dimensões.

Considerações Finais

Neste capítulo foram apresentados os principais conceitos sobre o JUnit e o

Mockito, duas ferramentas bastante utilizadas para a realização de testes automatizados em aplicações desenvolvidas com Java e Spring Boot.

Também foi possível compreender como essas ferramentas funcionam em conjunto, desde a execução dos testes até a validação dos resultados, além de seu posicionamento dentro dos Quadrantes de Testes Ágeis, especificamente no Quadrante 1, voltado à tecnologia e com foco em apoiar a equipe de desenvolvimento. Além disso, foram apresentados exemplos práticos que demonstram como validar o funcionamento de regras de negócio e cenários de erro da aplicação.

Dessa forma, o uso do JUnit e do Mockito contribui para o desenvolvimento de aplicações mais confiáveis, facilitando a identificação de erros e aumentando a qualidade do software ao longo do processo de desenvolvimento.

Bibliografia

JUNIT. JUnit 5 User Guide. Disponível em: <https://junit.org/junit5/docs/current/user-guide/>. Acesso em: 2 jun. 2026.

MOCKITO. Mockito Framework Site. Disponível em: <https://site.mockito.org/>. Acesso em: 2 jun. 2026.

SPRING. Testing – Spring Framework Documentation. Disponível em: <https://docs.spring.io/spring-framework/reference/testing.html>. Acesso em: 3 jun. 2026.

CRISPIN, Lisa; **GREGORY,** Janet. Agile Testing: A Practical Guide for Testers and Agile Teams. Addison-Wesley, 2009.

JUnit e Mockito

Marcelo Ferreira Miranda
marcelofmiranda23@gmail.com
marcelo.miranda@aluno.cps.sp.gov.br

Resumo

Os testes automatizados são uma prática importante no desenvolvimento de software, pois ajudam a identificar falhas e garantir o correto funcionamento das aplicações. Este capítulo apresenta o JUnit e o Mockito, duas ferramentas amplamente utilizadas para a criação e execução de testes em aplicações desenvolvidas com Java e Spring Boot. Inicialmente, são abordados os conceitos básicos e o funcionamento dessas ferramentas, destacando como podem ser utilizadas em conjunto para validar regras de negócio e comportamentos esperados da aplicação. Em seguida, são apresentados exemplos práticos de execução de testes unitários, demonstrando a validação de funcionalidades de forma isolada. Os resultados demonstram que a utilização dessas ferramentas contribui para a criação de aplicações mais confiáveis, auxiliando na identificação de erros e na melhoria da qualidade do software durante o processo de desenvolvimento.

Palavras-chaves: JUnit. Mockito. Testes automatizados. Spring Boot. Testes unitários.

Introdução

Os testes automatizados são uma parte importante do desenvolvimento de software, pois ajudam a identificar falhas, validar funcionalidades e garantir que alterações no código não afetem comportamentos já existentes. Dessa forma, eles contribuem para a criação de aplicações mais confiáveis e de fácil manutenção.

No desenvolvimento de aplicações com Java e Spring Boot, duas ferramentas bastante utilizadas para esse propósito são o JUnit e o Mockito. O JUnit é responsável pela criação e execução dos testes, enquanto o Mockito permite simular dependências externas, possibilitando testar unidades de código de forma isolada. A combinação dessas ferramentas possibilita a realização de testes completos, simulando situações

controladas de uso da aplicação sem depender de banco de dados ou serviços externos.

Este capítulo tem como objetivo apresentar os conceitos básicos do JUnit e do Mockito, destacando seu funcionamento, benefícios e aplicação prática em uma API desenvolvida com Spring Boot. Também será abordado o posicionamento dessas ferramentas dentro dos Quadrantes de Testes Ágeis, demonstrando como essas ferramentas podem auxiliar na melhoria da qualidade do software.

Desenvolvimento

O JUnit é um framework de testes utilizado em aplicações Java e Spring Boot. Sua principal função é permitir a criação e execução de testes automatizados, auxiliando na validação do comportamento da aplicação.

Já o Mockito é uma biblioteca utilizada para criar objetos simulados, conhecidos como mocks. Com ela é possível substituir dependências reais por versões controladas durante os testes, permitindo isolar o comportamento da classe que está sendo testada.

Quando utilizadas em conjunto, essas ferramentas permitem criar testes completos para validar regras de negócio, fluxos de criação e atualização de entidades, além de cenários de erro esperados pela aplicação.

Os Quadrantes de Testes Ágeis

Os Quadrantes de Testes Ágeis, propostos por Brian Marick e popularizados por Lisa Crispin e Janet Gregory, organizam os diferentes tipos de testes em quatro quadrantes de acordo com dois eixos: o foco do teste (voltado ao negócio ou voltado à tecnologia) e o objetivo (apoiar a equipe ou criticar o produto).

O JUnit e o Mockito se posicionam no Quadrante 1 (Q1), que agrupa testes voltados à tecnologia e com foco em apoiar a equipe de desenvolvimento. São testes executados com frequência, de forma automatizada, e que fornecem feedback rápido sobre a qualidade interna do código. Enquanto o Q2 abriga testes funcionais voltados ao negócio, o Q3 contempla testes exploratórios conduzidos por pessoas, e o Q4 engloba testes de desempenho e segurança, o Q1 é onde vivem os testes unitários e de integração que garantem que cada peça do código funciona corretamente de forma isolada.

Funcionamento

O JUnit executa os métodos anotados com `@Test` e verifica se os resultados

obtidos correspondem aos resultados esperados, utilizando métodos de asserção como `assertEquals`, `assertThat` e `assertThatThrownBy`.

O Mockito atua criando objetos simulados das dependências da classe testada. Por meio das anotações `@Mock` e `@InjectMocks`, é possível injetar os mocks automaticamente na classe de serviço, controlando o comportamento esperado com o método `when(...).thenReturn(...)`.

O fluxo de execução ocorre da seguinte forma:

1. O JUnit inicializa o contexto de testes com a extensão `@ExtendWith(MockitoExtension.class)`;
2. O Mockito cria os objetos simulados anotados com `@Mock`;
3. Os mocks são injetados na classe de serviço anotada com `@InjectMocks`;
4. O comportamento dos mocks é configurado com `when(...).thenReturn(...)`;
5. O método a ser testado é executado;
6. O JUnit valida os resultados por meio das asserções;
7. Os resultados são exibidos no terminal.

Adicionando as dependências

Para utilizar o JUnit e o Mockito em projetos Spring Boot, basta adicionar a dependência de testes no arquivo `pom.xml`. O Spring Boot já inclui ambas as ferramentas por meio do starter de testes:

- `mvn test` (para executar os testes via Maven);
- `./gradlew test` (para executar os testes via Gradle).

Executando os testes

Após criar os arquivos de teste, o Maven pode executá-los através do seguinte comando:

- `mvn test`

Ao final da execução será exibido um relatório semelhante ao exemplo abaixo:

```
[INFO] -----  
[INFO]  T E S T S  
[INFO] -----  
[INFO] Running com.Soucelo.service.UserServiceTest  
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0  
[INFO]  
[INFO] Results:  
[INFO]
```

```
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
```

Fonte: próprio autor

Entendendo o resultado dos testes

Ao executar os testes, o Maven fornece algumas informações importantes:

- **Tests run:** Tests run: Quantidade total de testes executados;
- **Failures:** Failures: Quantidade de testes que falharam por asserção incorreta;
- **Errors:** Errors: Quantidade de testes que lançaram exceções inesperadas durante a execução;
- **Skipped:** Skipped: Quantidade de testes ignorados com a anotação `@Disabled`;
- **BUILD SUCCESS / FAILURE:** BUILD SUCCESS / FAILURE: Indica se a execução foi bem-sucedida ou não.

Exemplo prático

A seguir será apresentado um exemplo de teste utilizado para validar a criação de usuários.

```
@Test
void createUser_shouldCreateUser_whenHasValidData() {
    when(repository.existsByEmail("joao@gmail.com"))
        .thenReturn(false);
    when(repository.existsByCpf("123.567.789-00"))
        .thenReturn(false);

    service.createUser(userValidRequest());

    verify(repository, times(1))
        .createUser(any(User.class));
}
```

Fonte: próprio autor

O teste configura os mocks para retornar false nas verificações de e-mail e CPF já existentes, executa o método de criação e verifica se o repositório foi chamado exatamente uma vez para persistir o usuário. Também é possível validar cenários de erro:

```
@Test
void createUser_shouldThrowException_whenEmailAlreadyExists() {
    when(repository.existsByEmail("joao@gmail.com"))
        .thenReturn(true);

    assertThatThrownBy(() -> service.createUser(userValidRequest()))
        .assertInstanceOf(RuntimeException.class)
```

```
.hasMessage("Email already exists");

verify(repository, never()).createUser(any());
}
```

Fonte: próprio autor

Nesse caso, o mock é configurado para retornar true na verificação de e-mail, simulando que o endereço já está cadastrado. O teste garante que uma exceção do tipo `RuntimeException` é lançada com a mensagem correta e que o repositório nunca é chamado para persistir o usuário.

Limitações do JUnit e do Mockito

Apesar de serem ferramentas poderosas para a validação de regras de negócio, o JUnit e o Mockito possuem limitações importantes que devem ser consideradas durante a definição da estratégia de testes de uma aplicação.

Por serem ferramentas voltadas ao Quadrante 1, seu foco está na validação de unidades isoladas de código. Isso significa que elas não são capazes de validar, por exemplo, a integração real com o banco de dados, o comportamento de APIs externas, fluxos completos da aplicação ou questões relacionadas à performance e segurança. Como os mocks substituem as dependências reais, eventuais problemas de integração entre os componentes da aplicação podem passar despercebidos durante os testes unitários.

Essas lacunas são justamente cobertas pelos demais quadrantes de testes. O Quadrante 2 abriga testes funcionais e de aceitação, que validam o comportamento da aplicação do ponto de vista do negócio, utilizando ferramentas como Selenium e Cypress. O Quadrante 3 contempla testes exploratórios e de usabilidade, conduzidos por pessoas, que identificam problemas que os testes automatizados não conseguem detectar. Já o Quadrante 4 engloba testes de desempenho, carga e segurança, realizados com ferramentas como JMeter, que garantem que a aplicação se comporta adequadamente sob condições adversas.

Dessa forma, o JUnit e o Mockito não devem ser vistos como a única solução de testes de uma aplicação, mas sim como uma parte fundamental de uma estratégia mais ampla, onde cada quadrante contribui com um tipo diferente de validação, tornando o software mais confiável em todas as suas dimensões.

Considerações Finais

Neste capítulo foram apresentados os principais conceitos sobre o JUnit e o

Mockito, duas ferramentas bastante utilizadas para a realização de testes automatizados em aplicações desenvolvidas com Java e Spring Boot.

Também foi possível compreender como essas ferramentas funcionam em conjunto, desde a execução dos testes até a validação dos resultados, além de seu posicionamento dentro dos Quadrantes de Testes Ágeis, especificamente no Quadrante 1, voltado à tecnologia e com foco em apoiar a equipe de desenvolvimento. Além disso, foram apresentados exemplos práticos que demonstram como validar o funcionamento de regras de negócio e cenários de erro da aplicação.

Dessa forma, o uso do JUnit e do Mockito contribui para o desenvolvimento de aplicações mais confiáveis, facilitando a identificação de erros e aumentando a qualidade do software ao longo do processo de desenvolvimento.

Bibliografia

JUNIT. JUnit 5 User Guide. Disponível em: <https://junit.org/junit5/docs/current/user-guide/>. Acesso em: 2 jun. 2026.

MOCKITO. Mockito Framework Site. Disponível em: <https://site.mockito.org/>. Acesso em: 2 jun. 2026.

SPRING. Testing – Spring Framework Documentation. Disponível em: <https://docs.spring.io/spring-framework/reference/testing.html>. Acesso em: 3 jun. 2026.

CRISPIN, Lisa; **GREGORY,** Janet. Agile Testing: A Practical Guide for Testers and Agile Teams. Addison-Wesley, 2009.

A ferramenta Flutter Test

Miguel Barbieri

barbierimiguel055@gmail.com

miguel.barbieri@aluno.cps.sp.gov.br

Resumo

O presente trabalho apresenta a ferramenta Flutter Test, utilizada para a realização de testes automatizados em aplicações desenvolvidas com Flutter. O objetivo deste estudo foi compreender a importância dos testes de software no desenvolvimento de aplicações modernas, além de demonstrar como a ferramenta pode auxiliar na identificação de falhas, aumento da confiabilidade e melhoria da qualidade do sistema. A metodologia utilizada baseou-se em pesquisas bibliográficas, análise da documentação oficial do Flutter e desenvolvimento de testes em um projeto prático de cadastro de usuários. Durante o desenvolvimento, foram aplicados testes unitários para validar funcionalidades do repositório responsável pelo gerenciamento dos dados dos usuários. Os resultados obtidos demonstraram que a utilização do Flutter Test contribui significativamente para a redução de erros, manutenção do código e segurança no desenvolvimento de aplicações.

Palavras-chave: Flutter Test. Flutter. Testes automatizados. Qualidade de software.

Introdução

O desenvolvimento de software exige cada vez mais qualidade, segurança e confiabilidade, principalmente em aplicações modernas que precisam atender às necessidades dos usuários de maneira eficiente. Nesse contexto, os testes automatizados tornaram-se fundamentais para garantir o correto funcionamento dos sistemas e evitar falhas durante o desenvolvimento.

A ferramenta Flutter Test é utilizada para realizar testes em aplicações desenvolvidas com Flutter, permitindo validar funcionalidades do sistema de forma automatizada. Sua utilização auxilia os desenvolvedores na identificação de erros, garantindo maior estabilidade e qualidade do software.

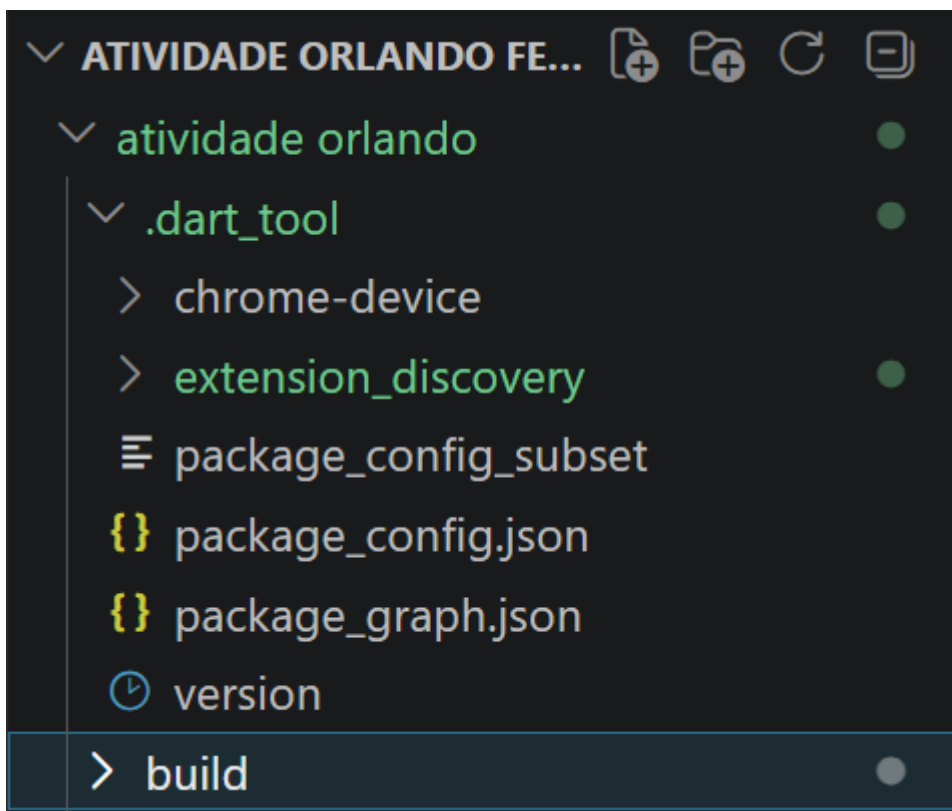
O objetivo deste trabalho é apresentar a ferramenta Flutter Test, demonstrando suas principais características, funcionalidades e vantagens no desenvolvimento de aplicações. Além disso, será apresentado um exemplo prático utilizando testes automatizados em um sistema de cadastro de usuários desenvolvido em Flutter.

A metodologia utilizada consistiu em pesquisas bibliográficas, análise da documentação oficial do Flutter e realização de testes em um projeto prático. O trabalho está organizado em seções que abordam os conceitos da ferramenta, o desenvolvimento do projeto e as considerações finais.

Desenvolvimento

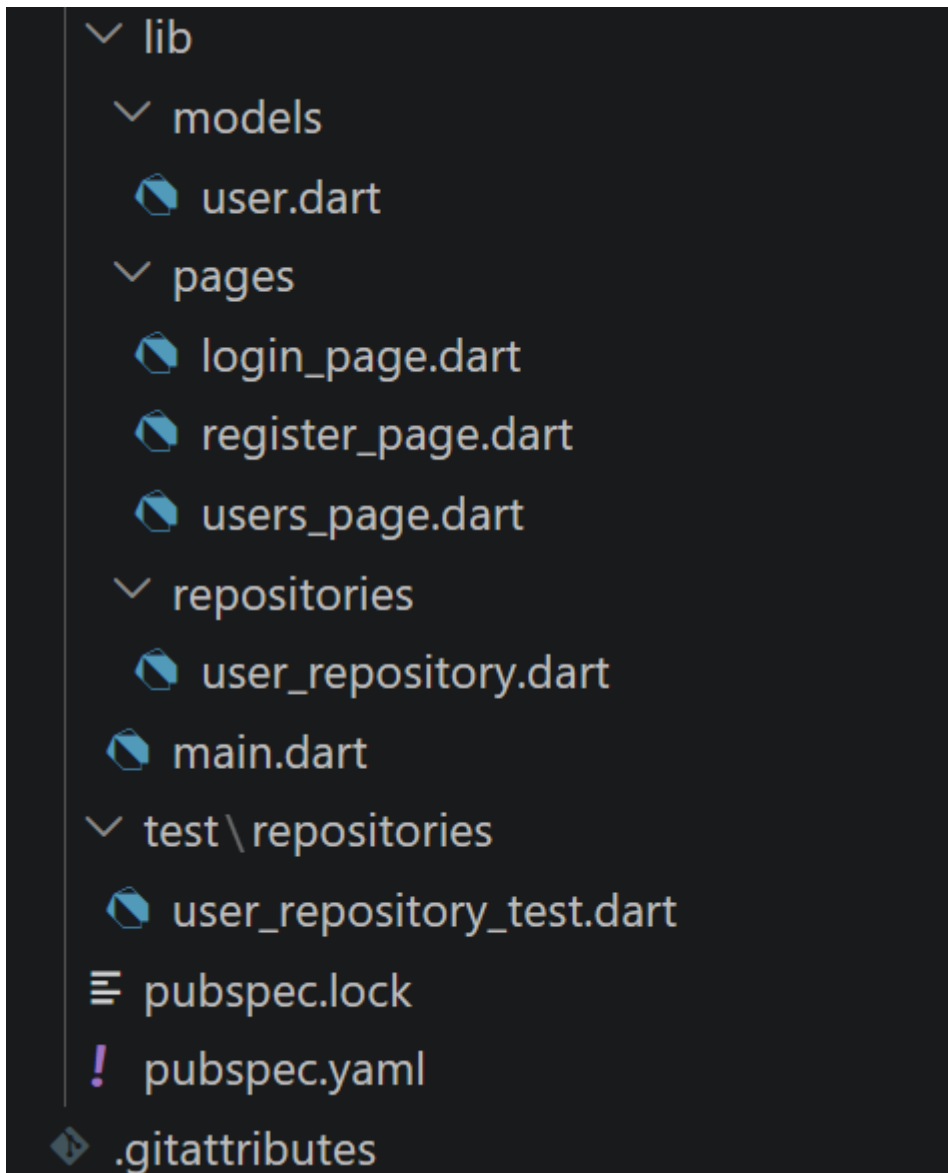
Estrutura do Projeto

Figura 1 – Estrutura do projeto



Fonte: próprio autor

Figura 1.1 – Estrutura do projeto



Fonte: próprio autor

A seguir, a explicação da estrutura:

- **.dart_tool/**: diretório gerado automaticamente pelo Flutter para armazenar configurações e informações internas do projeto.
- **chrome-device/**: contém configurações relacionadas à execução da aplicação no navegador Google Chrome.
- **extension_discovery/**: armazena arquivos internos utilizados pelo Flutter para descoberta de extensões e ferramentas.
- **package_config_subset**: arquivo auxiliar utilizado na configuração dos pacotes do projeto.
- **package_config.json**: define os pacotes e dependências utilizados pela aplicação.
- **package_graph.json**: representa a estrutura de dependências existentes no projeto.
- **version**: arquivo responsável por armazenar informações sobre a versão do SDK utilizado.
- **build/**: diretório gerado automaticamente durante a compilação, contendo os arquivos

produzidos para execução da aplicação.

- **lib/**: pasta principal do projeto, onde se encontra o código-fonte desenvolvido.
- **models/**: contém as classes responsáveis pela modelagem e representação dos dados da aplicação.
- **pages/**: armazena as telas e interfaces gráficas utilizadas pelo usuário.
- **repositories/**: responsável pela lógica de negócio e manipulação dos dados da aplicação.
- **main.dart**: arquivo principal responsável por iniciar a execução da aplicação Flutter.
- **test/**: diretório destinado aos testes automatizados desenvolvidos com a ferramenta Flutter Test.
- **pubspec.lock**: registra as versões exatas das dependências instaladas no projeto.
- **pubspec.yaml**: arquivo principal de configuração do Flutter, contendo dependências, recursos e informações gerais da aplicação.
- **.gitattributes**: arquivo de configuração utilizado pelo Git para controle de versão e gerenciamento de arquivos do repositório.

Essa estrutura permitiu organizar adequadamente os componentes do sistema, facilitando o desenvolvimento, a manutenção e a implementação dos testes automatizados utilizando o Flutter Test.

Testes Realizados

Figura 2 – Teste de adição de usuário

```
test('Adicionar usuário', () {  
  final repo = UserRepository();  
  
  repo.addUser(User(email: 'a@a.com', password: '1234'));  
  
  expect(repo.countUsers(), 1);  
});
```

Fonte: próprio autor

Este teste tem como objetivo verificar se a funcionalidade de cadastro de usuários está funcionando corretamente. Inicialmente, é criada uma instância da classe `UserRepository`, responsável por armazenar os usuários do sistema. Em seguida, é adicionado um novo usuário com o e-mail `a@a.com` e a senha `1234` utilizando o método `addUser()`. Por fim, o teste utiliza a função `expect()` para confirmar que a quantidade de usuários cadastrados é igual a 1. Caso o valor retornado pelo método `countUsers()` seja diferente de 1, o teste falhará, indicando que ocorreu algum problema no processo de cadastro.

Figura 2.1 – Teste de verificação de email existente

```
test('Verificar email existente', () {  
    final repo = UserRepository();  
  
    repo.addUser(User(email: 'a@a.com', password: '1234'));  
  
    expect(repo.emailExists('a@a.com'), true);  
});
```

Fonte: próprio autor

Este teste tem como objetivo verificar se o sistema consegue identificar corretamente um e-mail já cadastrado. Inicialmente, é criada uma instância da classe UserRepository, responsável pelo armazenamento e gerenciamento dos usuários. Em seguida, um usuário é cadastrado com o e-mail a@a.com e a senha 1234. Após o cadastro, o método emailExists() é utilizado para verificar se o e-mail informado está presente no repositório. O teste utiliza a função expect() para confirmar que o resultado retornado é true, indicando que o e-mail foi encontrado com sucesso.

Figura 2.2 – Teste de validação

```
test('Login válido', () {  
    final repo = UserRepository();  
  
    repo.addUser(User(email: 'a@a.com', password: '1234'));  
  
    expect(repo.login('a@a.com', '1234'), true);  
});
```

Fonte: próprio autor

Este teste tem como objetivo validar o processo de autenticação de usuários no sistema. Inicialmente, é criada uma instância da classe UserRepository, responsável pelo gerenciamento dos usuários cadastrados. Em seguida, um usuário é adicionado ao repositório com o e-mail a@a.com e a senha 1234. Após o cadastro, o método login() é executado utilizando as mesmas credenciais. Por fim, a função expect() verifica se o resultado retornado é true, indicando que o login foi realizado com sucesso.

Figura 2.3 – Teste de tentativa de login inválida

```
test('Login inválido', () {  
    final repo = UserRepository();  
  
    expect(repo.login('a@a.com', '1234'), false);  
});
```

Fonte: próprio autor

Este teste tem como objetivo verificar se o sistema bloqueia tentativas de login utilizando credenciais inexistentes ou inválidas. Inicialmente, é criada uma instância da classe UserRepository, porém nenhum usuário é cadastrado no repositório. Em seguida, o método login() é executado utilizando o e-mail a@a.com e a senha 1234. Como não existe nenhum usuário registrado com essas credenciais, espera-se que o método retorne false. A função expect() é utilizada para validar esse comportamento, garantindo que o sistema não permita acessos indevidos.

Figura 2.4 – Teste de remoção de usuários

```
test('Deve remover usuários selecionados', () {  
    final repo = UserRepository();  
  
    // Adiciona usuários  
    repo.addUser(User(email: 'a@a.com', password: '123'));  
    repo.addUser(User(email: 'b@b.com', password: '123'));  
    repo.addUser(User(email: 'c@c.com', password: '123'));  
  
    // Remove dois usuários  
    repo.removeUsers(['a@a.com', 'b@b.com']);  
  
    // Verifica se sobrou apenas 1  
    expect(repo.countUsers(), 1);  
  
    // Verifica se o usuário restante é o correto  
    expect(repo.emailExists('c@c.com'), true);  
});
```

Fonte: próprio autor

Este teste tem como objetivo verificar se o sistema consegue remover corretamente usuários selecionados do repositório. Inicialmente, é criada uma instância da classe UserRepository e são cadastrados três usuários diferentes. Em seguida, o método removeUsers() é utilizado para remover dois deles, identificados pelos seus respectivos e-mails. Após a remoção, o teste valida se apenas

um usuário permanece cadastrado no sistema e se o usuário restante é exatamente o esperado. Dessa forma, é possível garantir que a funcionalidade de exclusão múltipla está operando corretamente.

Figura 2.5 – Teste de remoção seletiva

```
test('Não deve remover usuários não selecionados', () {  
  final repo = UserRepository();  
  
  repo.addUser(User(email: 'a@a.com', password: '123'));  
  repo.addUser(User(email: 'b@b.com', password: '123'));  
  
  // Remove só um  
  repo.removeUsers(['a@a.com']);  
  
  expect(repo.emailExists('a@a.com'), false);  
  expect(repo.emailExists('b@b.com'), true);  
});  
  
});  
}
```

Fonte: próprio autor

Este teste tem como objetivo verificar se o sistema remove apenas os usuários selecionados, preservando os demais registros cadastrados. Inicialmente, é criada uma instância da classe `UserRepository` e são adicionados dois usuários ao repositório. Em seguida, o método `removeUsers()` é utilizado para remover apenas o usuário com o e-mail `a@a.com`. Após a remoção, o teste valida se esse usuário realmente foi excluído e se o usuário `b@b.com` continua armazenado no sistema. Dessa forma, garante-se que a operação de remoção afete somente os registros especificados.

Considerações Finais

O estudo da ferramenta Flutter Test permitiu compreender a importância dos testes automatizados no desenvolvimento de aplicações Flutter. Por meio da implementação de testes voltados ao cadastro de usuários, autenticação de login, validação de e-mails e remoção de registros, foi possível verificar como a ferramenta auxilia na identificação de falhas e na validação do comportamento esperado do sistema. Os resultados obtidos demonstraram que a utilização de testes automatizados contribui para o aumento da qualidade, confiabilidade e manutenção do software.

Além dos benefícios observados durante o desenvolvimento do projeto, o uso do Flutter Test mostrou-se uma prática essencial para reduzir erros e garantir maior segurança em futuras atualizações da aplicação. Como proposta para trabalhos futuros, recomenda-se a ampliação da cobertura de testes por meio da implementação de testes de widgets e testes de integração, permitindo validar não apenas as regras de negócio, mas também a interação do usuário com a interface do sistema. Dessa forma, será possível obter uma aplicação ainda mais robusta e confiável.

Bibliografia

FATEC ARARAS. Faculdade de Tecnologia de Araras. Disponível em:

<https://fatecararas.cps.sp.gov.br/>. Acesso em: 05 jun. 2026.

FLUTTER. Flutter Documentation. Disponível em: <https://docs.flutter.dev/>. Acesso em: 05 jun. 2026.

FLUTTER. Testing Flutter Apps. Disponível em: <https://docs.flutter.dev/testing>. Acesso em: 05 jun. 2026.

DART. Dart Testing Documentation. Disponível em: <https://dart.dev/testing>. Acesso em: 05 jun. 2026.

SARAIVA JUNIOR, Orlando. ebook_qualidade_teste_software. GitHub. Disponível em:

https://github.com/orlandosaraivajr/ebook_qualidade_teste_software. Acesso em: 05 jun. 2026.

Pytest

Victor Manoel Martins
victormartinsworkspace@gmail.com
victor.martins3@aluno.cps.sp.gov.br

Resumo

Este e-book apresenta a utilização da biblioteca Pytest, uma das ferramentas mais populares para automação de testes na linguagem Python. O objetivo principal é demonstrar como a implementação de testes automatizados contribui para a qualidade do software, aumentando a confiabilidade do código, facilitando a identificação de falhas e apoiando práticas modernas de desenvolvimento, como o Test-Driven Development (TDD). A abordagem adotada combina a fundamentação teórica dos testes de software com a exploração prática dos recursos oferecidos pelo Pytest, por meio da construção de exemplos e cenários reais de validação. Ao longo do material, são apresentados conceitos essenciais, como a criação de funções de teste, o uso de assertivas simplificadas, a aplicação de fixtures, a parametrização de testes e a geração de relatórios. Os resultados evidenciam que o Pytest oferece uma solução flexível, intuitiva e escalável para o desenvolvimento de testes automatizados, sendo amplamente utilizado tanto em ambientes acadêmicos quanto corporativos. Conclui-se que sua adoção contribui para a padronização dos processos de teste, promove maior segurança durante a manutenção do código e fortalece a cultura de qualidade no desenvolvimento de software.

Palavras-chave: pytest, testes automatizados, Python, qualidade de software.

Introdução

A qualidade de software é um dos pilares fundamentais no desenvolvimento de sistemas modernos, exigindo práticas que garantam confiabilidade, manutenção contínua e redução de falhas ao longo de todo o ciclo de vida do produto. Entre essas práticas, os testes unitários desempenham papel estratégico, pois permitem

validar pequenas partes do código de forma isolada, assegurando que cada componente execute corretamente sua função.

Nesse contexto, este e-book dedica-se ao estudo do Pytest, um dos frameworks de testes mais utilizados no ecossistema Python, reconhecido por sua simplicidade, flexibilidade e capacidade de tornar a escrita de testes mais intuitiva. O escopo do trabalho concentra-se na apresentação dos conceitos fundamentais relacionados aos testes automatizados, na análise das principais funcionalidades oferecidas pelo Pytest e na demonstração prática de sua aplicação em projetos de software.

O objetivo geral é compreender de que forma a automatização de testes contribui para a melhoria da qualidade do software e como o Pytest pode auxiliar nesse processo. Como objetivos específicos, destacam-se: identificar as características e vantagens da ferramenta, analisar sua estrutura de funcionamento e apresentar exemplos práticos que evidenciem sua utilização em diferentes cenários de desenvolvimento.

A escolha do tema justifica-se pela crescente adoção do Pytest na indústria de software, especialmente devido à sua sintaxe simplificada, ao suporte a recursos avançados e à facilidade de integração com processos modernos de desenvolvimento, como Integração Contínua (CI) e Desenvolvimento Orientado a Testes (TDD).

A metodologia utilizada envolveu pesquisa bibliográfica, análise da documentação oficial do framework e experimentação prática por meio da implementação de casos de teste em Python. A organização deste e-book segue três etapas principais: inicialmente, são apresentados os conceitos fundamentais dos testes automatizados e do Pytest; em seguida, são explorados seus principais recursos e exemplos de aplicação; por fim, são apresentadas as considerações finais, destacando os benefícios da ferramenta e possíveis direções para estudos futuros.

Visão Geral

O Pytest é um framework de testes para Python amplamente utilizado devido à sua simplicidade, flexibilidade e facilidade de uso. Diferentemente de abordagens mais tradicionais, o Pytest permite criar testes de forma direta, utilizando funções comuns da linguagem Python e assertivas nativas, tornando o código de teste mais legível e intuitivo.

Além da execução de testes unitários, o framework oferece suporte para testes de integração, parametrização de cenários, reutilização de recursos por meio de fixtures e geração de relatórios detalhados. Outro diferencial importante é a capacidade de descobrir automaticamente arquivos e funções de teste, reduzindo a necessidade de configurações adicionais.

Sua arquitetura extensível permite a utilização de plugins desenvolvidos pela comunidade, ampliando funcionalidades como medição de cobertura de código, execução paralela e integração com pipelines de Integração Contínua (CI). Dessa forma, o Pytest tornou-se uma das ferramentas mais adotadas para automação de testes no ecossistema Python.

Breve histórico

O Pytest foi criado por Holger Krekel no início dos anos 2000 com o objetivo de oferecer uma alternativa mais simples e produtiva aos frameworks de teste existentes na época. Inspirado nos conceitos de testes automatizados já consolidados na engenharia de software, o projeto buscou reduzir a complexidade da escrita e manutenção de testes.

Ao longo dos anos, o framework evoluiu significativamente, incorporando recursos avançados como parametrização de testes, fixtures reutilizáveis, sistema de plugins e integração com diversas ferramentas do ecossistema Python. Sua comunidade ativa contribuiu para o desenvolvimento de extensões que ampliaram ainda mais suas capacidades.

Atualmente, o Pytest é considerado um dos frameworks de testes mais populares da linguagem Python, sendo utilizado tanto em projetos acadêmicos quanto em aplicações corporativas de grande porte. Sua adoção crescente está relacionada à sua facilidade de uso, à clareza dos relatórios gerados e à eficiência no desenvolvimento de testes automatizados.

Ambiente de testes

No contexto do Pytest, o ambiente de testes corresponde ao conjunto de recursos e práticas que permitem criar, organizar e executar verificações automatizadas de forma eficiente. Entre os principais elementos desse ambiente, destacam-se:

- Arquivos de teste: normalmente organizados em módulos específicos, com nomes iniciados ou finalizados por "test", facilitando sua identificação automática pelo framework.
- Funções de teste: implementadas como funções comuns da linguagem Python, geralmente precedidas pelo prefixo `test_`, permitindo que o Pytest as reconheça automaticamente.
- Assertivas nativas: o framework utiliza a própria instrução `assert` do Python, tornando os testes mais simples e legíveis.
- Fixtures: mecanismo utilizado para preparar dados, objetos ou configurações necessárias para os testes, promovendo reutilização e organização.
- Parametrização: recurso que permite executar o mesmo teste com diferentes conjuntos de dados, reduzindo duplicação de código.
- Execução por linha de comando: realizada por meio do comando `pytest`, que executa automaticamente todos os testes encontrados no projeto.
- Relatórios de execução: apresentam informações detalhadas sobre testes aprovados, falhas, erros e estatísticas gerais de execução.

Esse ambiente simplifica o processo de automação de testes e possibilita sua integração com ferramentas de desenvolvimento, versionamento e integração contínua, tornando-o adequado para projetos de diferentes portes.

Principais Conceitos e Estruturas do pytest

Para simplificar a criação e manutenção de testes automatizados, o Pytest disponibiliza diversos recursos que tornam o desenvolvimento mais produtivo e organizado. Entre os principais, destacam-se:

- `assert` (assertivas): utilizado para verificar se um resultado corresponde ao esperado. Diferentemente de outros frameworks, o Pytest utiliza a própria instrução `assert` do Python, fornecendo mensagens de erro detalhadas quando uma validação falha.
- `Fixtures`: mecanismo responsável por preparar e disponibilizar recursos necessários para os testes, como objetos, conexões ou conjuntos de dados. As fixtures promovem a reutilização de código e facilitam a manutenção dos testes.
- `pytest.raises()`: recurso utilizado para validar exceções esperadas. Permite verificar se determinada operação gera o erro previsto, garantindo o tratamento adequado de situações excepcionais.
- `Parametrização` (`@pytest.mark.parametrize`): possibilita executar o mesmo teste com diferentes conjuntos de dados, reduzindo duplicação de código e ampliando a cobertura dos testes.
- `Markers` (`@pytest.mark`): permitem categorizar testes em grupos específicos, facilitando a seleção e execução de determinados conjuntos durante o desenvolvimento.
- `Test Discovery` (descoberta automática de testes): funcionalidade que identifica automaticamente arquivos e funções de teste no projeto, dispensando configurações complexas para sua execução.
- `Plugins`: extensões que adicionam funcionalidades extras ao framework, como geração de relatórios, análise de cobertura de código e execução paralela de testes.

Esses recursos tornam o Pytest uma ferramenta flexível e eficiente para a automação de testes, contribuindo para a qualidade, confiabilidade e manutenção do software.

Desenvolvimento

Para o desenvolvimento deste projeto, foi realizada uma divisão em duas etapas principais. A primeira corresponde à pesquisa bibliográfica e documental, com foco no estudo da documentação oficial do Pytest, buscando compreender seus conceitos, recursos e funcionamento. A segunda etapa consistiu na aplicação prática, envolvendo a implementação de um sistema bancário simples em Python acompanhado da criação de testes automatizados utilizando o framework Pytest.

Essa prática teve como objetivo validar funcionalidades específicas de uma conta bancária e demonstrar, de forma concreta, como os testes automatizados contribuem para a qualidade e confiabilidade do software. Foram explorados recursos importantes do Pytest, como assertivas, fixtures e validação de exceções.

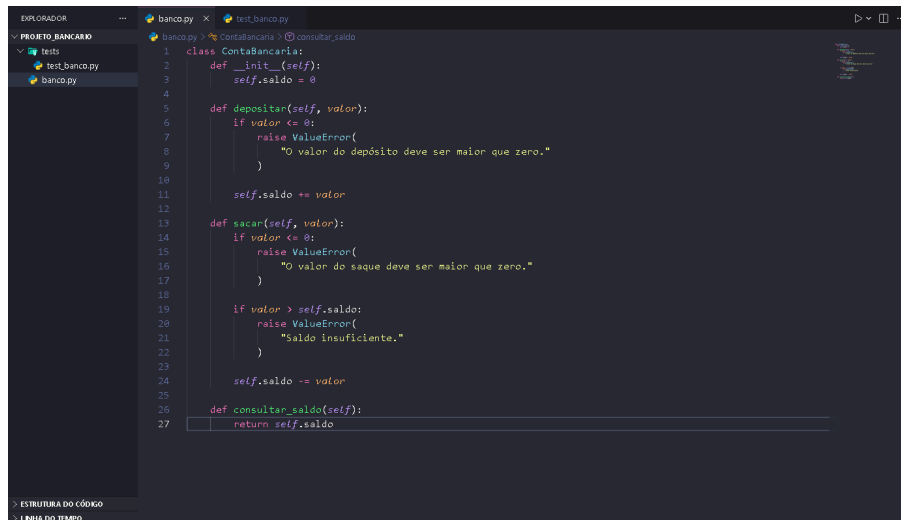
Seguindo o roteiro de demonstração, inicialmente foi criada a estrutura de diretórios do projeto para separar o código da aplicação dos arquivos de teste.

Principais Conceitos e Estruturas do pytest

```
projeto_bancario/  
|  
├── banco.py  
|  
└── tests/  
    └── test_banco.py
```

No arquivo banco.py foi implementada a classe responsável pelas operações bancárias básicas, incluindo depósito, saque e consulta de saldo.

Figura 1

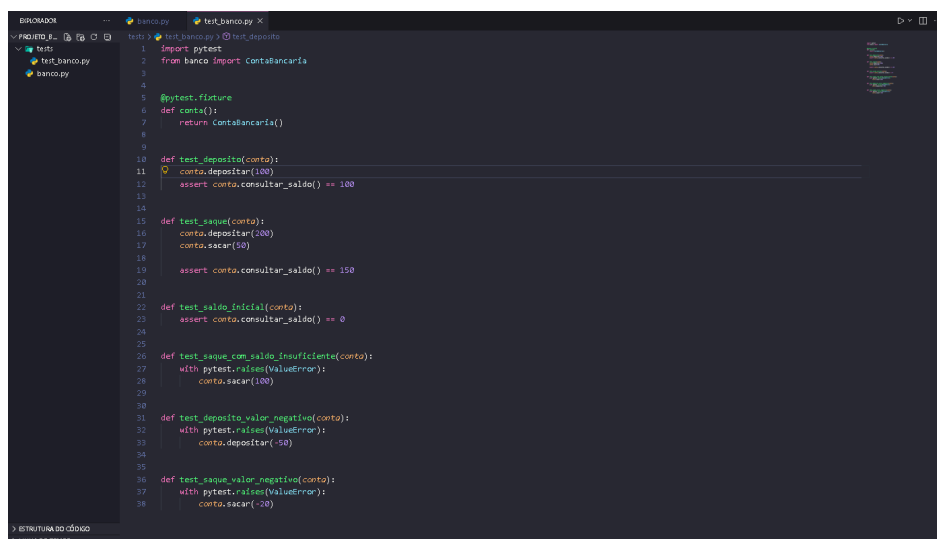


```
1 class ContaBancaria:
2     def __init__(self):
3         self.saldo = 0
4
5     def depositar(self, valor):
6         if valor <= 0:
7             raise ValueError(
8                 "O valor do depósito deve ser maior que zero."
9             )
10
11         self.saldo += valor
12
13     def sacar(self, valor):
14         if valor <= 0:
15             raise ValueError(
16                 "O valor do saque deve ser maior que zero."
17             )
18
19         if valor > self.saldo:
20             raise ValueError(
21                 "Saldo insuficiente."
22             )
23
24         self.saldo -= valor
25
26     def consultar_saldo(self):
27         return self.saldo
```

Fonte: próprio autor

Após a implementação da classe, foram criados testes automatizados para validar o comportamento das operações bancárias. Para facilitar a reutilização de código entre os testes, foi utilizada uma fixture responsável por criar uma nova instância da conta bancária antes de cada execução.

Figura 2



```
1 import pytest
2 from banco import ContaBancaria
3
4
5 @pytest.fixture
6 def conta():
7     return ContaBancaria()
8
9
10 def test_deposito(conta):
11     conta.depositar(100)
12     assert conta.consultar_saldo() == 100
13
14
15 def test_saque(conta):
16     conta.depositar(100)
17     conta.sacar(50)
18     assert conta.consultar_saldo() == 150
19
20
21 def test_saldo_inicial(conta):
22     assert conta.consultar_saldo() == 0
23
24
25 def test_saque_com_saldo_insuficiente(conta):
26     with pytest.raises(ValueError):
27         conta.sacar(100)
28
29
30 def test_deposito_valor_negativo(conta):
31     with pytest.raises(ValueError):
32         conta.depositar(-50)
33
34
35 def test_saque_valor_negativo(conta):
36     with pytest.raises(ValueError):
37         conta.sacar(-20)
```

Fonte: próprio autor

Os testes desenvolvidos verificam diferentes cenários de utilização do sistema. Foram avaliadas operações válidas, como depósitos e saques, além de situações excepcionais, como tentativas de saque sem saldo suficiente e movimentações com valores negativos.

Para permitir que o Pytest encontre automaticamente os testes, foi adotada a seguinte convenção:

- Os arquivos de teste inicia com o prefixo test_;
- As funções de teste inicia com o prefixo test_;
- Os testes foram organizados dentro da pasta tests;
- A execução foi realizada por meio do comando:

pytest -v

Além dos cenários de sucesso, foram implementados testes utilizando `pytest.raises()` para validar o tratamento correto de exceções. Também foi utilizado o recurso de fixtures, permitindo a reutilização da configuração inicial da conta bancária entre os diferentes casos de teste.

Os resultados obtidos demonstraram que o Pytest oferece uma forma simples e eficiente de automatizar testes em aplicações Python, contribuindo para a redução de erros, aumento da confiabilidade do código e maior facilidade de manutenção do software.

Figura 3

```
26 def test_saque_com_saldo_insuficiente(conta):
27     with pytest.raises(ValueError):
28         conta.sacar(100)
```

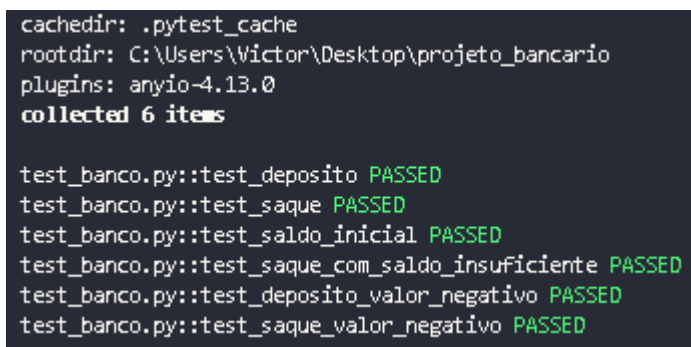
Fonte: próprio autor

Para demonstrar a capacidade do framework em validar situações de erro, foi implementado um teste utilizando o recurso `pytest.raises()`. Nesse cenário, a aplicação tenta realizar um saque sem saldo suficiente ou um depósito com valor inválido, verificando se a exceção esperada é lançada corretamente. Esse tipo de

teste é importante para garantir que as regras de negócio sejam respeitadas e que comportamentos inadequados sejam tratados pelo sistema.

Ao executar o comando de teste, todos os casos implementados são identificados automaticamente pelo Pytest e executados em sequência. Ao final da execução, o framework apresenta um relatório contendo informações sobre os testes aprovados, falhas encontradas e possíveis erros.

Figura 4



```
cachedir: .pytest_cache
rootdir: C:\Users\Victor\Desktop\projeto_bancario
plugins: anyio-4.13.0
collected 6 items

test_banco.py::test_deposito PASSED
test_banco.py::test_saque PASSED
test_banco.py::test_saldo_inicial PASSED
test_banco.py::test_saque_com_saldo_insuficiente PASSED
test_banco.py::test_deposito_valor_negativo PASSED
test_banco.py::test_saque_valor_negativo PASSED
```

Fonte: próprio autor

As saídas apresentadas pelo Pytest podem ser classificadas da seguinte forma:

- PASSED — o teste foi executado com sucesso e o resultado obtido corresponde ao esperado;
- FAILED — uma assertiva falhou, indicando que o resultado obtido é diferente do resultado esperado;
- ERROR — ocorreu uma exceção inesperada durante a execução do teste ou durante a configuração do ambiente de teste;
- SKIPPED — o teste foi ignorado intencionalmente por alguma configuração definida pelo desenvolvedor.

É importante destacar que, além dos recursos explorados neste projeto, o Pytest disponibiliza diversas funcionalidades avançadas que ampliam significativamente sua aplicação em cenários profissionais. Entre elas, destacam-se as fixtures parametrizadas, que permitem reutilizar configurações em múltiplos testes; a parametrização de testes, que possibilita executar o mesmo teste com diferentes conjuntos de dados; os markers, utilizados para categorizar e selecionar

grupos específicos de testes; e o amplo ecossistema de plugins, que adiciona funcionalidades como geração de relatórios detalhados, medição de cobertura de código e execução paralela.

Outro recurso bastante utilizado é o sistema de mocking, que pode ser integrado ao Pytest por meio da biblioteca `unittest.mock` ou de plugins específicos. Essa técnica permite simular dependências externas, como bancos de dados, APIs ou serviços de terceiros, tornando os testes mais rápidos, previsíveis e independentes do ambiente externo.

Esses recursos tornam o Pytest uma ferramenta moderna, flexível e altamente escalável, adequada tanto para projetos acadêmicos quanto para aplicações corporativas de grande porte, contribuindo para a qualidade, confiabilidade e manutenção contínua do software.

Considerações Finais

O estudo realizado permitiu compreender a relevância dos testes automatizados como parte essencial da garantia de qualidade em software. A utilização do framework Pytest demonstrou que a criação de testes de forma simples e organizada, aliada ao uso de assertivas, fixtures e validação de exceções, contribui significativamente para a identificação rápida e confiável de falhas durante o desenvolvimento.

A prática desenvolvida por meio do sistema bancário possibilitou observar o funcionamento do framework em cenários reais de validação. Os testes implementados permitiram verificar operações como depósitos, saques e consultas de saldo, além de validar o comportamento da aplicação diante de situações excepcionais, como tentativas de movimentação com valores inválidos ou saldo insuficiente. Dessa forma, foi possível evidenciar a importância dos testes automatizados na prevenção de erros e na manutenção da qualidade do código.

Como perspectiva para trabalhos futuros, podem ser explorados recursos mais avançados do Pytest, como parametrização de testes, mocking de dependências externas, análise de cobertura de código e integração com ferramentas de Integração Contínua e Entrega Contínua (CI/CD). Essas práticas contribuem para aumentar a confiabilidade das aplicações e aproximar o processo de desenvolvimento das exigências encontradas no mercado de software.

Por fim, o aprendizado obtido ao longo deste trabalho contribuiu para uma melhor compreensão dos conceitos de testes automatizados e de sua aplicação prática em projetos Python. O Pytest mostrou-se uma ferramenta moderna, acessível e eficiente, capaz de auxiliar tanto estudantes quanto profissionais na construção de aplicações mais seguras, estáveis e alinhadas às boas práticas da Engenharia de Software.

Bibliografia

PYTEST DEVELOPMENT TEAM. *Pytest Documentation*. Disponível em: <https://docs.pytest.org/>. Acesso em: 04 junho. 2026.

PYTHON SOFTWARE FOUNDATION. *Python Documentation*. Disponível em: <https://docs.python.org/3/>. Acesso em: 04 junho. 2026.

SARAIVA JUNIOR, Orlando. *ebook_qualidade_teste_software*. GitHub. Disponível em: https://github.com/orlandosaraivajr/ebook_qualidade_teste_software. Acesso em: 04 junho. 2026.

Kubernetes como Ferramenta de Garantia de Qualidade de Software

Matheus de Oliveira Matias Silva

matheus@matiasit.com

matheus.silva92@aluno.cps.sp.gov.br

Resumo

O Kubernetes é uma plataforma open-source de orquestração de contêineres que, além de automatizar o deployment e o gerenciamento de aplicações em escala, oferece mecanismos nativos que contribuem diretamente para a qualidade do software. Este capítulo explora como recursos como isolamento por namespaces, políticas de rede (ClusterIP, NodePort e LoadBalancer), probes de disponibilidade (liveness e readiness) e escalabilidade automática (Horizontal Pod Autoscaler e escalamento de nós) se traduzem em garantias de segurança, confiabilidade e disponibilidade ao longo do ciclo de vida do software. A metodologia adotada consiste na análise prática das funcionalidades do Kubernetes aplicadas ao SDLC, demonstrando como a plataforma atua como um mecanismo ativo de verificação e manutenção da qualidade. Os resultados mostram que a adoção do Kubernetes não apenas otimiza a infraestrutura, mas impõe padrões arquiteturais que elevam a qualidade intrínseca das aplicações, tornando-o uma ferramenta relevante para a Qualidade e Testes de Software.

Palavras-chave: Kubernetes. Qualidade de Software. DevOps. Escalabilidade. Segurança de Rede.

Introdução

A qualidade de software é frequentemente associada a práticas de teste como testes unitários, de integração, análise estática (SAST) e análise dinâmica (DAST). No entanto, a infraestrutura sobre a qual o software opera é igualmente determinante para sua confiabilidade, segurança e disponibilidade. Nesse contexto, o Kubernetes — originalmente criado pelo Google e cedido à Cloud Native Computing Foundation (CNCF) em 2014 — emerge como uma plataforma de orquestração de contêineres que, de forma nativa, implementa controles que correspondem a requisitos clássicos de qualidade de software.

O objetivo deste capítulo é demonstrar como o Kubernetes pode ser compreendido e utilizado como uma ferramenta de garantia de qualidade, analisando quatro pilares fundamentais: isolamento e segmentação de rede por namespaces, comunicação controlada entre serviços por meio de tipos de Service, verificação de saúde dos pods via probes, e escalabilidade automática por meio do Horizontal Pod Autoscaler (HPA) e do escalonamento de nós nos pools do cluster.

A justificativa para esta abordagem reside no fato de que a qualidade de software não se encerra no código-fonte. Um software de alta qualidade deve ser seguro, disponível, escalável e resiliente — atributos que o Kubernetes contribui diretamente para garantir. Este capítulo foi desenvolvido com base em estudo prático e análise da documentação oficial do Kubernetes, explorando sua aplicação em ambientes reais de produção, inclusive no contexto profissional do autor.

O capítulo está organizado da seguinte forma: após esta introdução, a seção de Desenvolvimento apresenta os conceitos fundamentais do Kubernetes e sua relação com o SDLC, discute os mecanismos de rede e isolamento, explora a verificação de saúde dos serviços e aborda a escalabilidade automática. As Considerações Finais sintetizam as contribuições do Kubernetes para a qualidade de software e indicam perspectivas de trabalhos futuros.

Desenvolvimento

O Kubernetes e o SDLC

O Kubernetes (K8s) é um sistema de orquestração de contêineres que automatiza o deployment, o escalonamento e o gerenciamento de aplicações containerizadas. No contexto do Software Development Life Cycle (SDLC), o Kubernetes atua principalmente nas fases de implantação, operação e manutenção, sendo um componente central em pipelines de CI/CD modernos. Sua arquitetura é baseada em um cluster composto por um plano de controle (control plane) e nós de trabalho (worker nodes), nos quais os pods — a menor unidade executável do Kubernetes — são agendados e executados (KUBERNETES, 2024).

A relação entre Kubernetes e qualidade de software se estabelece pelo fato de a plataforma transformar requisitos não funcionais — como disponibilidade, segurança e escalabilidade — em configurações declarativas e verificáveis. Assim, garantias que antes dependiam exclusivamente de processos operacionais manuais passam a ser codificadas como parte do próprio sistema, aproximando infraestrutura e qualidade de software.

Namespaces e Isolamento: Qualidade por Segmentação

Os namespaces são mecanismos de virtualização do Kubernetes que permitem dividir um cluster em ambientes lógicos isolados. Em uma abordagem orientada a qualidade, é comum criar namespaces distintos para cada ambiente do SDLC: desenvolvimento (dev), homologação (staging) e produção (production). Esse isolamento garante que defeitos introduzidos no ambiente de desenvolvimento não contaminem o ambiente produtivo, reduzindo o risco de incidentes e facilitando a reprodução de defeitos em ambientes controlados (KUBERNETES, 2024b).

Além do isolamento lógico, os namespaces permitem a aplicação de cotas de recursos (ResourceQuota) e limites de objetos (LimitRange), assegurando que nenhum serviço consuma recursos de forma indiscriminada. Esse controle é essencial para a qualidade em ambientes multi-tenant, nos quais múltiplos times compartilham a mesma infraestrutura. Do ponto de vista de testes, namespaces facilitam a execução de testes de integração end-to-end em ambientes efêmeros, criados e destruídos automaticamente durante o pipeline de CI/CD.

Tipos de Service e Segurança de Rede: Controle do Tráfego como Qualidade

O Kubernetes define diferentes tipos de Service para controlar como as aplicações são expostas: ClusterIP, NodePort, LoadBalancer e ExternalName. Cada tipo representa um nível diferente de exposição e, consequentemente, um nível diferente de superfície de ataque. O ClusterIP, tipo padrão, expõe o serviço apenas internamente ao cluster, tornando-o inacessível externamente. Esse modelo é adequado para serviços de back-end e bancos de dados, que nunca devem ser expostos diretamente à internet (KUBERNETES, 2024c).

O tipo LoadBalancer cria um balanceador de carga externo — geralmente provido pelo provedor de nuvem — que distribui o tráfego entre as réplicas do serviço e garante alta disponibilidade. Essa distribuição de carga é um atributo direto de qualidade, pois impede que uma única instância se torne gargalo e ponto único de falha. Do ponto de vista de segurança, combinar LoadBalancer com NetworkPolicies permite definir, de forma declarativa, quais pods podem se comunicar entre si, criando um modelo de menor privilégio de rede (principle of least privilege) que reduz o impacto de possíveis vulnerabilidades.

As NetworkPolicies do Kubernetes funcionam como um firewall distribuído em nível de pod, permitindo especificar regras de ingresso (ingress) e egresso (egress) com base em seletores de namespace e de labels. Em termos de qualidade, essa capacidade se assemelha a testes de segurança de rede automatizados: ao declarar que apenas o Serviço A pode acessar o Banco de Dados B, qualquer desvio dessa política é bloqueado em tempo de execução, funcionando como uma verificação contínua da arquitetura de segurança.

Liveness e Readiness Probes: Verificação de Saúde em Tempo de Execução

As probes são um dos mecanismos mais diretamente relacionados à qualidade de software em tempo de execução. O Kubernetes oferece três tipos: liveness probe, readiness probe e startup probe. A liveness probe verifica se um contêiner está funcionando corretamente; caso contrário, o Kubernetes reinicia o pod automaticamente. Esse mecanismo atua como um watchdog automático, garantindo a auto-recuperação do serviço diante de falhas como deadlocks ou corrupção de estado interno (KUBERNETES, 2024).

A readiness probe, por outro lado, verifica se um contêiner está pronto para receber tráfego. Enquanto a probe retorna falha, o pod é removido do pool de endpoints do Service correspondente, garantindo que requisições de usuários nunca sejam roteadas para instâncias ainda em inicialização ou em degradação. Essa característica é especialmente relevante em cenários de deployment contínuo (rolling update), onde novas versões da aplicação são implantadas de forma gradual sem interrupção do serviço.

Do ponto de vista da qualidade, as probes implementam na prática o conceito de “test in production” de forma segura: em vez de confiar apenas em testes realizados anteriormente ao deployment, o Kubernetes verifica continuamente, em intervalos configuráveis, se a aplicação satisfaz os critérios de saúde definidos pela equipe de desenvolvimento. Cada probe pode ser implementada via requisição HTTP, comando de shell ou conexão TCP, oferecendo flexibilidade para diferentes tipos de aplicação.

Escalabilidade Automática: HPA e Escalamento de Nós como Vetor de Qualidade

A escalabilidade é um dos atributos de qualidade previstos na norma ISO/IEC 25010 como parte da característica de eficiência de desempenho. O Kubernetes implementa escalabilidade automática em dois níveis complementares: no nível de pod, por meio do Horizontal Pod Autoscaler (HPA), e no nível de nó, por meio do Cluster Autoscaler ou do Node Autoprovisioner (NAP) em ambientes gerenciados como o Google Kubernetes Engine (GKE) (GOOGLE, 2024).

O HPA monitora métricas definidas pelo operador — tipicamente uso de CPU e memória, mas também métricas personalizadas via Metrics Server ou adapters externos — e ajusta automaticamente o número de réplicas de um Deployment ou StatefulSet dentro de limites pré-configurados (minReplicas e maxReplicas). Esse mecanismo garante que a aplicação mantenha seu desempenho mesmo sob picos de carga inesperados, sem a necessidade de intervenção humana.

O escalamento de nós complementa o HPA ao garantir que a capacidade do cluster acompanhe o crescimento da demanda. Quando o HPA aumenta o número de réplicas e o cluster não possui recursos suficientes, o Cluster Autoscaler adiciona novos nós ao pool correspondente.

Em períodos de baixa demanda, nós ociosos são removidos, reduzindo custos. Esse ciclo de escalonamento inteligente, quando associado a testes de carga no pipeline de CI/CD, permite validar os limites da aplicação antes que ela chegue ao ambiente produtivo, caracterizando uma prática avançada de engenharia de confiabilidade de site (SRE).

Kubernetes no Pipeline de CI/CD: Qualidade como Código

A integração do Kubernetes em pipelines de CI/CD como o GitHub Actions ou o Cloud Build permite tratar a qualidade da infraestrutura como código (Infrastructure as Code). Por meio de ferramentas como Kustomize, Helm e manifestos YAML versionados, toda a configuração do cluster passa pelo mesmo processo de revisão de código (code review) que o código-fonte da aplicação. Assim, mudanças na configuração de NetworkPolicies, probes ou HPA passam por validação automática antes de serem aplicadas ao cluster (REDHAT, 2024).

Complementarmente, ferramentas como kubesecc, kube-bench e OPA Gatekeeper permitem realizar análise estática (SAST) dos manifestos Kubernetes, identificando configurações inseguras como containers rodando como root, imagens sem tag fixa ou ausência de limites de recursos. Essa camada de verificação transforma o Kubernetes em um ponto de aplicação de políticas de segurança no próprio pipeline, unindo os conceitos de SAST ao domínio de configuração de infraestrutura.

Considerações Finais

Este capítulo demonstrou que o Kubernetes, embora não seja uma ferramenta de teste no sentido tradicional, é um componente essencial da qualidade de software em ambientes modernos baseados em contêineres. Seus mecanismos de isolamento por namespaces, controle de rede por tipos de Service e NetworkPolicies, verificação de saúde por liveness e readiness probes, e escalabilidade automática por HPA e Cluster Autoscaler implementam, de forma declarativa e verificável, atributos de qualidade previstos na norma ISO/IEC 25010.

A principal contribuição do Kubernetes para a qualidade de software reside na transferência de verificações de qualidade do ambiente de teste para o ambiente de produção, criando um ciclo contínuo de validação que complementa as abordagens clássicas de teste. Ao tratar a infraestrutura como código e integrá-la ao pipeline de CI/CD, as equipes de desenvolvimento passam a ter visível, rastreada e auditada toda a configuração que impacta a qualidade do produto final.

Como perspectiva de trabalho futuro, sugere-se a expansão deste estudo com a implementação de ferramentas de análise estática de manifestos Kubernetes (como kube-bench e OPA Gatekeeper) integradas ao pipeline de CI/CD, bem como a avaliação de service meshes como

Istio e Linkerd como camada adicional de segurança e observabilidade. Essas abordagens ampliariam significativamente o escopo da garantia de qualidade provida pelo ecossistema Kubernetes.

Bibliografia

GOOGLE. Google Kubernetes Engine: Cluster Autoscaler. Disponível em: <https://cloud.google.com/kubernetes-engine/docs/concepts/cluster-autoscaler>. Acesso em: 14 jun. 2026.

KUBERNETES. Kubernetes Documentation: Overview. Disponível em: <https://kubernetes.io/docs/concepts/overview/>. Acesso em: 14 jun. 2026.

KUBERNETES. Kubernetes Documentation: Namespaces. Disponível em: <https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/>. Acesso em: 14 jun. 2026.

KUBERNETES. Kubernetes Documentation: Service. Disponível em: <https://kubernetes.io/docs/concepts/services-networking/service/>. Acesso em: 14 jun. 2026.

KUBERNETES. Kubernetes Documentation: Configure Liveness, Readiness and Startup Probes. Disponível em: <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>. Acesso em: 14 jun. 2026.

KUBERNETES. Kubernetes Documentation: Horizontal Pod Autoscaling. Disponível em: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>. Acesso em: 14 jun. 2026.

REDHAT. What is CI/CD? Red Hat. Disponível em: <https://www.redhat.com/en/topics/devops/what-is-ci-cd>. Acesso em: 14 jun. 2026.

SARAIVA JUNIOR, Orlando. ebook_qualidade_teste_software. GitHub. Disponível em: https://github.com/orlandosaraivajr/ebook_qualidade_teste_software. Acesso em: 14 jun. 2026.

Selenium no mapa dos Quadrantes de Teste Ágil: onde ele mora e onde costuma visitar

Wilson Geraldo Donizeti Pereira Júnior
willsf2021@gmail.com
wilson.pereira3@aluno.cps.gov.br

Resumo

Automatizar testes virou item obrigatório em qualquer equipe de software que entregue com frequência, e o Selenium aparece com naturalidade nessa conversa quando o assunto é teste de aplicações web. O problema é que, na maioria das vezes, a ferramenta é apresentada apenas pela ótica técnica, sem que se discuta onde ela realmente se encaixa dentro de uma estratégia de qualidade. Este capítulo propõe um caminho diferente: analisar o Selenium com Python utilizando como referência o modelo dos Quadrantes de Teste Ágil, popularizado por Lisa Crispin e Janet Gregory a partir da ideia original de Brian Marick. A metodologia combina pesquisa bibliográfica, leitura da documentação oficial e a construção de um caso prático de automação envolvendo um fluxo de login no site público the-internet.herokuapp.com. Os resultados mostram que o Selenium tem residência fixa no Quadrante 2, voltado a testes funcionais que apoiam a equipe, mas que também visita os outros três quadrantes em cenários específicos, como verificações em pipeline de integração contínua, apoio a testes exploratórios e medições simples de tempo de carregamento. A discussão fecha apontando trade-offs e situações em que o uso da ferramenta vale ou não vale a pena.

Palavras-chaves: Selenium. Python. Quadrantes de Teste Ágil. Automação de Testes.

Introdução

Imagine a seguinte cena, que acontece em muitas equipes de desenvolvimento. A entrega está pronta, o time fez os ajustes de última hora e a pessoa responsável pelos testes abre uma planilha com cinquenta cenários para validar manualmente antes do deploy. Funciona? Funciona. Só que na semana seguinte chega uma nova feature, depois um bug fix, depois uma refatoração, e aquela mesma planilha volta a ser executada do zero. Em algum momento, alguém pula um passo, outro alguém esquece de testar um navegador específico e o problema vai parar em produção.

Aqui mora o ponto que motiva qualquer conversa séria sobre automação de testes. A automação não está sendo discutida porque virou tendência, também não tem a ver com tirar pessoas do processo de teste. A questão é mais simples: existe um conjunto de verificações que a equipe não consegue mais executar manualmente em tempo hábil, e alguém precisa repetir essas

verificações com confiabilidade. Quando essa dor aparece, ferramentas como o Selenium passam a fazer sentido.

Só que, na prática, é comum o Selenium ser apresentado de uma forma que ajuda pouco quem está começando. Mostram o comando de instalação, abrem um navegador automatizado, fazem uma busca no Google e encerram o assunto. A pergunta que fica sem resposta é a mais importante: em que momento da estratégia de qualidade essa ferramenta encaixa? Sem responder isso, qualquer time corre o risco de automatizar coisas que não precisariam ser automatizadas ou, pior, deixar de automatizar exatamente o que daria mais retorno.

Para tentar responder essa pergunta de forma mais estruturada, este capítulo apoia-se em um modelo bastante conhecido na literatura de testes ágeis: os Quadrantes de Teste Ágil, organizados originalmente por Brian Marick e detalhados por Lisa Crispin e Janet Gregory no livro *Agile Testing* (CRISPIN; GREGORY, 2009). O modelo divide os tipos de teste em quatro grandes regiões, levando em conta dois critérios: se o teste apoia a equipe ou critica o produto, e se o foco é o negócio ou a tecnologia. É um mapa simples, mas que ajuda muito a posicionar ferramentas e decisões.

A proposta aqui é usar esse mapa para localizar o Selenium. A tese de fundo é que ele tem um endereço principal, o Quadrante 2, mas costuma visitar os outros três em situações específicas. Compreender essa distinção evita dois erros comuns: usar Selenium onde uma ferramenta mais leve resolveria, ou rejeitar Selenium em cenários onde ele encaixaria perfeitamente.

O capítulo está organizado da seguinte forma. Primeiro, discute-se o problema que motiva o uso de ferramentas como o Selenium e por que olhar para ele apenas pela parte técnica costuma ser insuficiente. Em seguida, os Quadrantes de Teste Ágil são apresentados de forma direta, sem acadêmiquês. Depois, mostra-se onde o Selenium mora e em quais quadrantes ele costuma visitar. A seção seguinte traz um caso prático, com código em Python, executando um fluxo de login em um site público preparado para receber esse tipo de teste. O capítulo encerra com os erros comuns observados em projetos reais de automação Selenium e uma discussão sobre quando a ferramenta vale a pena, quando não vale, e o que esperar dela.

Desenvolvimento

O problema antes da ferramenta

Olha só uma situação que aparece com bastante frequência em equipes que começaram a automatizar testes recentemente. Alguém ouve falar de Selenium, instala a biblioteca, abre um navegador automatizado, faz um clique aqui, um clique ali, e em uma semana o time já tem trinta

scripts rodando. Em três meses, são duzentos. Em seis meses, ninguém mais sabe direito o que cada script testa, qual é prioritário, qual está quebrado por bug real e qual está quebrado porque o front-end mudou de cor.

Aqui aparece a verdadeira dor. Automatizar é razoavelmente fácil. Automatizar com critério é outro patamar. Sem um modelo mental claro, a equipe acaba investindo tempo em scripts que não geram retorno, deixa de cobrir áreas críticas e produz uma suíte de testes lenta, frágil e cara de manter. Quando o sistema cresce, esse detalhe passa a importar muito.

É nesse ponto que entra o valor de um mapa. Antes de escrever a primeira linha de código com Selenium, vale a pena responder a uma pergunta: que tipo de teste eu quero automatizar e por quê? Os Quadrantes de Teste Ágil oferecem exatamente esse mapa.

Os Quadrantes de Teste Ágil em uma página

O modelo foi proposto inicialmente por Brian Marick em 2003 e ganhou notoriedade quando Lisa Crispin e Janet Gregory o utilizaram como espinha dorsal do livro Agile Testing (CRISPIN; GREGORY, 2009). A ideia central é simples: testes podem ser classificados a partir de dois eixos.

O primeiro eixo trata da intenção do teste. Ele apoia a equipe durante o desenvolvimento, ajudando a construir o produto certo? Ou ele critica o produto pronto, buscando descobrir problemas que escaparam? Essa diferença muda completamente quem escreve o teste, quando escreve e com que objetivo.

O segundo eixo trata do foco do teste. Ele se preocupa com o ponto de vista do negócio, validando regras e fluxos do usuário? Ou ele se preocupa com aspectos técnicos internos, como integrações, performance e segurança?

Cruzando esses dois eixos, surgem quatro quadrantes.

Quadrante 1 — Tecnologia, apoia a equipe. Aqui moram os testes unitários, testes de componentes e testes de integração de baixo nível. São testes rápidos, escritos pelo time de desenvolvimento e executados a todo momento. JUnit, pytest unitário, NUnit e xUnit vivem aqui. Pense neles como o cinto de segurança do programador: estão sempre apertados, mas só fazem diferença quando algo dá errado.

Quadrante 2 — Negócio, apoia a equipe. São os testes funcionais, testes de aceitação, testes de história e protótipos. Eles validam se o sistema entrega o que foi combinado com a área de negócio. Costumam ser escritos em uma linguagem mais próxima do usuário e exercitam a aplicação como um todo, geralmente passando pela interface. Ferramentas como Cucumber, Robot Framework e o próprio Selenium ocupam esse espaço.

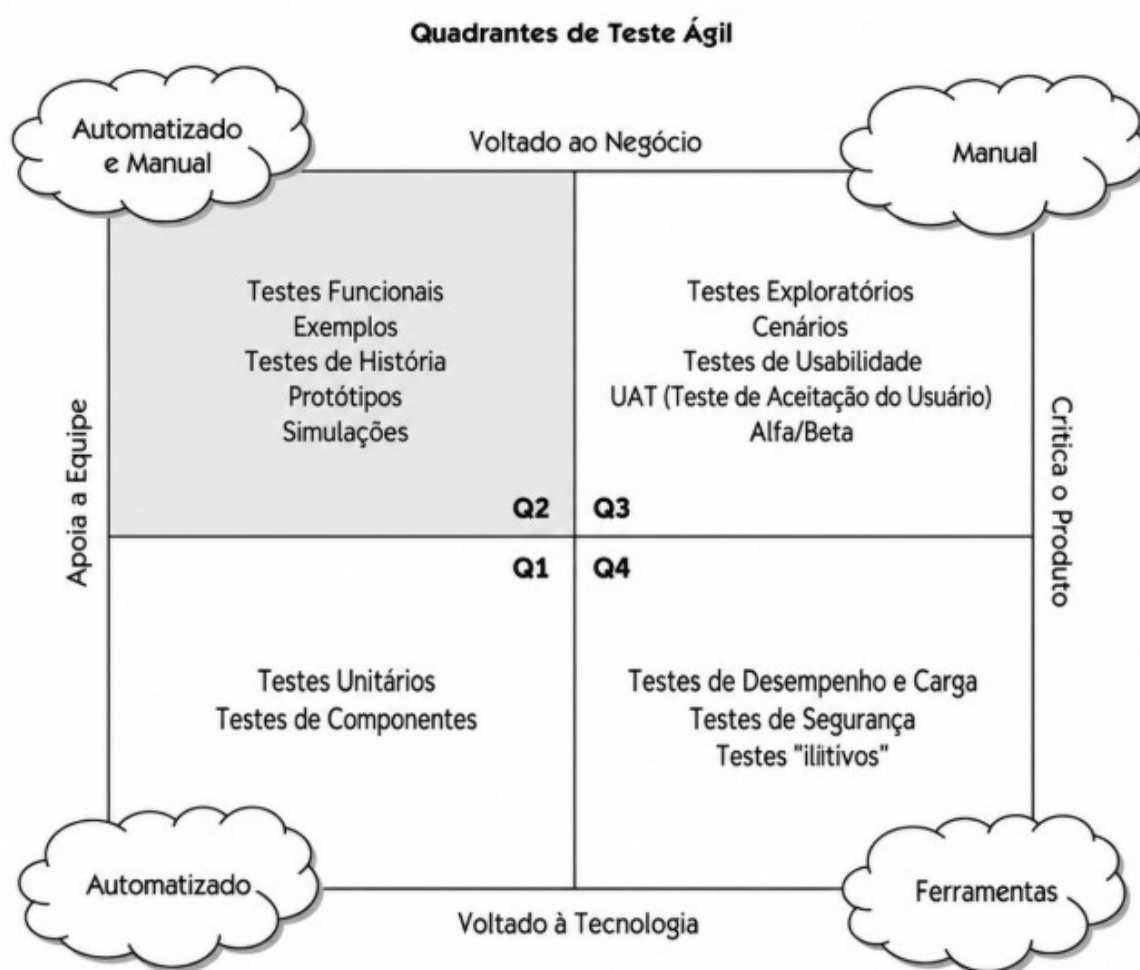
Quadrante 3 — Negócio, critica o produto. Testes exploratórios, testes de usabilidade,

testes de aceitação do usuário e testes alfa e beta. Aqui o objetivo é olhar para o produto pronto com olhos de quem vai usar, e isso costuma exigir gente, intuição e contexto. Automatizar inteiramente o Q3 é difícil, e nem deveria ser o objetivo.

Quadrante 4 — Tecnologia, critica o produto. Testes de performance, testes de carga, testes de segurança, testes de acessibilidade e os famosos testes de “-idades” (escalabilidade, confiabilidade, manutenibilidade). Ferramentas como JMeter, Gatling, OWASP ZAP e Lighthouse vivem aqui.

A figura a seguir traz uma representação simplificada dessa divisão.

Figura 1 – Quadrantes de Teste Ágil



Fonte: adaptado de Crispin e Gregory (2009).

O endereço do Selenium: Quadrante 2

Com o mapa em mãos, agora dá para localizar o Selenium com mais precisão. A casa dele é o Quadrante 2.

Faz sentido quando se olha para a natureza da ferramenta. Selenium foi criado para

automatizar interações com navegadores reais, simulando o que um usuário faria: abrir uma página, preencher um campo, clicar em um botão, validar uma mensagem. Esse tipo de verificação acompanha o caminho que uma pessoa real percorreria, e o objetivo é confirmar que o sistema entrega aquilo que o negócio combinou.

Imagine uma loja virtual. O dono do produto diz: quando o usuário adicionar um item ao carrinho, aplicar o cupom DESCONTO10 e finalizar a compra, o valor cobrado precisa estar com 10% de desconto e o pedido precisa aparecer no histórico. Esse cenário descreve uma regra de negócio, atravessa toda a aplicação e termina em uma interface gráfica. Selenium consegue automatizar esse fluxo do começo ao fim, executando contra o sistema rodando em um navegador real, exatamente como um cliente faria.

Esse é o tipo de teste que o Quadrante 2 representa, e é o tipo de teste em que Selenium brilha. Ele apoia a equipe porque o resultado da execução guia decisões durante o desenvolvimento. E ele tem foco no negócio porque o que está sendo validado é uma regra falada na linguagem do produto, e não um detalhe interno do código.

As visitas aos outros quadrantes

O Selenium tem residência fixa no Quadrante 2, e isso não significa que ele fique restrito a esse espaço. Em projetos reais, é comum que a ferramenta apareça em situações que tecnicamente pertencem a outros quadrantes. Vale a pena olhar cada um desses casos.

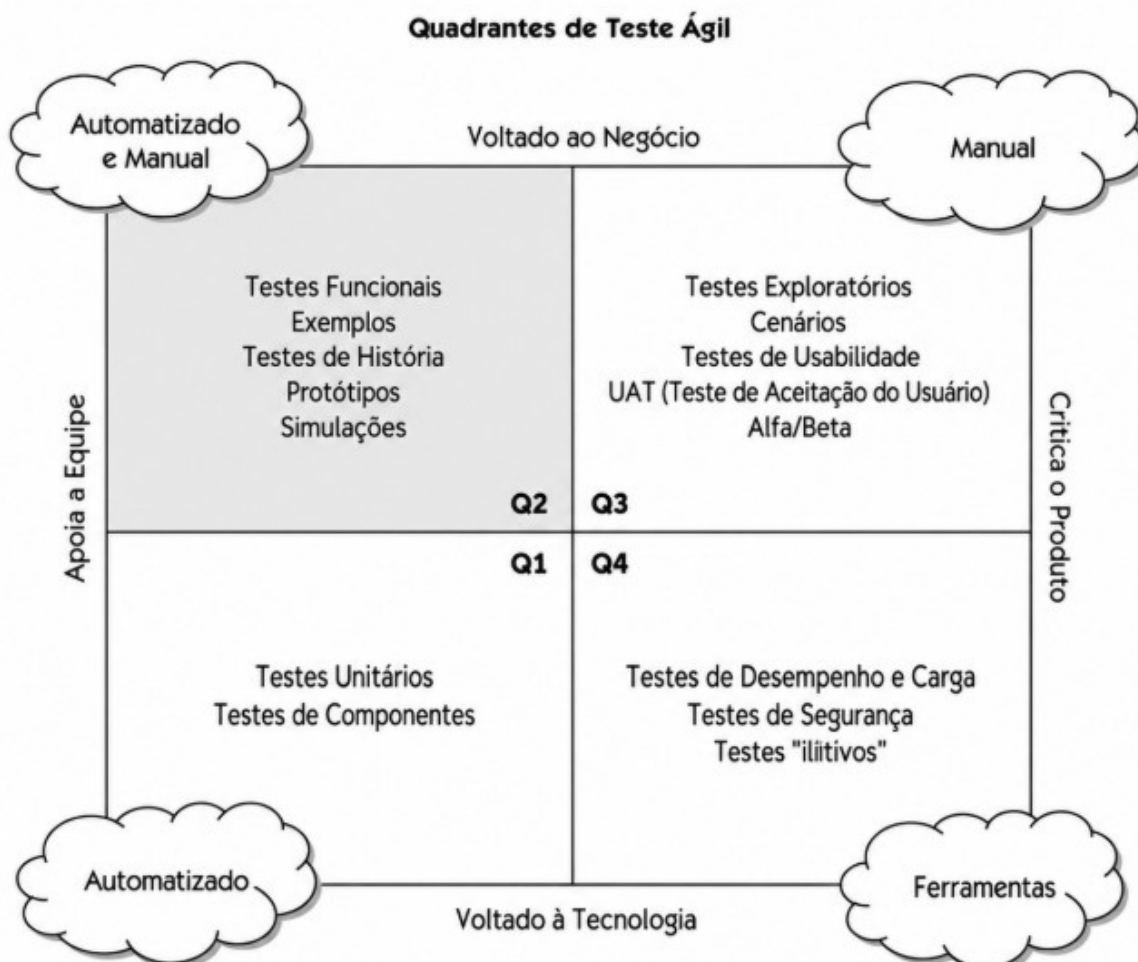
Visita ao Quadrante 1. O Q1 é o território dos testes técnicos que apoiam a equipe. Em tese, esse espaço é de bibliotecas de teste unitário. Só que existe um tipo de uso do Selenium que se aproxima desse perfil: os smoke tests rodando dentro de pipelines de integração contínua. Aqui o objetivo é confirmar que a aplicação subiu corretamente após o deploy, que a tela de login carrega, que o menu principal aparece. São verificações pequenas, rápidas e técnicas, executadas a cada commit. O Selenium acaba sendo usado como uma sonda automática no ambiente, mais perto da intenção do Q1 do que do Q2 clássico.

Visita ao Quadrante 3. O Q3 é território de testes manuais que criticam o produto, e a primeira reação é dizer que o Selenium não tem nada a fazer ali. Em projetos reais, a história fica um pouco diferente. O Selenium IDE, extensão de navegador que grava interações do usuário e gera scripts automaticamente, costuma ser usado como apoio em testes exploratórios. Um testador percorre um fluxo, grava o que fez, e quando encontra um bug consegue replicar o caminho exato sem precisar redigitar tudo. Também aparece em UAT, quando o time precisa documentar com precisão o passo a passo que o usuário final executou para chegar em determinado erro. Não chega a ser automação no sentido completo, e o Selenium dá apoio ao trabalho manual desse quadrante.

Visita ao Quadrante 4. O Q4 é onde vivem performance, segurança e acessibilidade. Selenium não é uma ferramenta de carga, isso precisa ficar claro. Para simular cinco mil usuários simultâneos, a escolha certa é JMeter, Gatling ou similar. Só que o Selenium consegue fazer algumas medições técnicas pontuais e bastante úteis. Ele pode injetar JavaScript na página para consultar a Navigation Timing API e medir quanto tempo o navegador levou para carregar a página completa, do primeiro byte ao DOM pronto. Pode ser combinado com a biblioteca axe-core para rodar verificações automáticas de acessibilidade durante a execução dos testes funcionais (DEQUE SYSTEMS, 2025). Em pipelines mais avançados, pode ainda alimentar dashboards de qualidade com tempos de resposta medidos do lado do cliente. Não substitui as ferramentas dedicadas do Q4, e oferece uma camada adicional de verificação que pega problemas que escapariam de outra forma.

A figura a seguir resume essas visitas.

Figura 2 – Selenium e os quadrantes que ele visita



Fonte: adaptado de Crispin e Gregory (2009).

Caso prático: login automatizado no the-internet.herokuapp

Para sair do plano teórico, vale a pena exercitar o Selenium em um caso concreto. Escolheu-

se aqui o site público the-internet.herokuapp.com, mantido pelo autor Dave Haefner especificamente para servir de cenário de prática para automação de testes. Ele oferece páginas estáveis, sem CAPTCHA e sem políticas de bloqueio, o que evita os problemas comuns ao tentar automatizar sites comerciais.

O cenário escolhido é o de login. A página /login do site espera as credenciais tomsmith e SuperSecretPassword!. Quando o login é bem-sucedido, a aplicação redireciona para uma página de área segura e exibe uma mensagem de confirmação. Quando o login falha, ela exibe uma mensagem de erro. O objetivo da automação é validar os dois caminhos.

Passo 1 — Preparar o ambiente.

Antes de qualquer linha de código, o ambiente precisa estar configurado. Python instalado, virtualenv criada e as dependências baixadas:

```
python -m venv venv
source venv/bin/activate    # no Windows: venv\Scripts\activate
pip install selenium webdriver-manager pytest
```

A biblioteca webdriver-manager resolve um problema chato: ela baixa e gerencia automaticamente o driver compatível com a versão do navegador instalado, evitando a necessidade de baixar e atualizar o ChromeDriver manualmente.

Passo 2 — Estruturar o teste seguindo Page Object Model.

Em vez de escrever tudo dentro de um único arquivo, o código foi separado em duas partes: uma classe representando a página de login (Page Object) e o arquivo de teste em si. Esse padrão isola os detalhes da interface em um lugar só, o que facilita a manutenção quando o front-end muda.

```
# login_page.py
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class LoginPage:
    URL = "https://the-internet.herokuapp.com/login"

    def __init__(self, driver, timeout=10):
        self.driver = driver
        self.wait = WebDriverWait(driver, timeout)

    def abrir(self):
        self.driver.get(self.URL)
        return self

    def preencher_credenciais(self, usuario, senha):
        self.driver.find_element(By.ID, "username").send_keys(usuario)
        self.driver.find_element(By.ID, "password").send_keys(senha)
        return self

    def confirmar(self):
        botao = self.driver.find_element(
            By.CSS_SELECTOR, "button[type='submit']"
        )
        botao.click()
        return self

    def mensagem_flash(self):
        elemento = self.wait.until(
            EC.presence_of_element_located((By.ID, "flash"))
        )
        return elemento.text.strip()
```

Passo 3 — Escrever o teste.

```
# test_login.py
import pytest
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
from login_page import LoginPage

@pytest.fixture
def driver():
    chrome = webdriver.Chrome(
        service=Service(ChromeDriverManager().install())
    )
    yield chrome
    chrome.quit()

def test_login_com_sucesso(driver):
    pagina = LoginPage(driver)
    pagina.abrir().preencher_credenciais(
        "tomsmith", "SuperSecretPassword!"
    ).confirmar()
    assert "You logged into a secure area" in pagina.mensagem_flash()

def test_login_com_senha_invalida(driver):
    pagina = LoginPage(driver)
    pagina.abrir().preencher_credenciais(
        "tomsmith", "senha_errada"
    ).confirmar()
    assert "Your password is invalid" in pagina.mensagem_flash()
```

Perceba que o teste em si ficou bastante limpo. Ele descreve o cenário em termos próximos da linguagem do negócio: abrir a página, preencher credenciais, confirmar e validar a mensagem. Os detalhes técnicos sobre como cada elemento é encontrado na página ficaram dentro da classe `LoginPage`. Esse arranjo segue a recomendação do Quadrante 2, com testes que falam a língua do negócio e exercitam o sistema todo.

Passo 4 — Executar e coletar evidências.

A execução é feita com pytest:

```
pytest test_login.py -v
```

Figura 3 – Execução dos testes no terminal

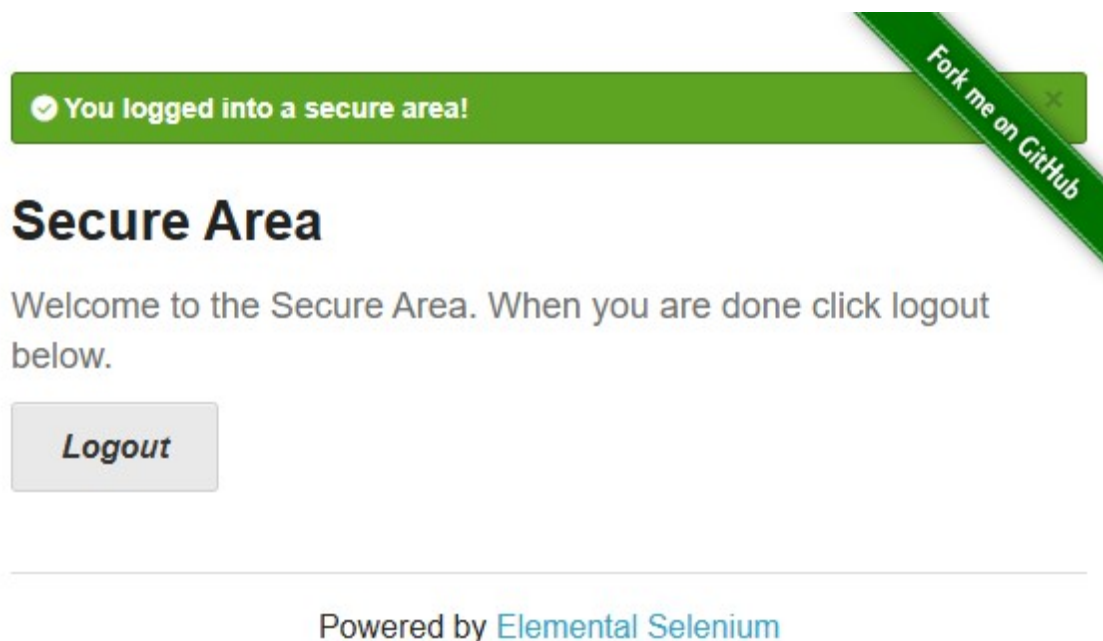
```
(.venv) PS C:\Users\Melina\Desktop\PythonProject1> pytest test_login.py -v
===== test session starts =====
platform win32 -- Python 3.13.1, pytest-9.1.0, pluggy-1.6.0 -- C:\Users\Melina\Desktop\PythonProject1\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Melina\Desktop\PythonProject1
collected 2 items

test_login.py::test_login_com_sucesso PASSED [ 50%]
test_login.py::test_login_com_senha_invalida PASSED [100%]

===== 2 passed in 14.31s =====
```

Fonte: próprio autor.

Figura 4 – Tela da área segura após login bem-sucedido



Fonte: próprio autor.

Passo 5 — Visita ao Quadrante 4: medir o tempo de carregamento.

Para ilustrar uma das visitas do Selenium a outro quadrante, dá para acrescentar uma medição simples de performance front-end usando a Navigation Timing API. Esse dado pertence ao Q4, e é coletado de dentro do mesmo teste do Q2.

```
def test_tempo_de_carga_login(driver):
    pagina = LoginPage(driver)
    pagina.abrir()
    tempo_ms = driver.execute_script("""
        const t = performance.timing;
        return t.loadEventEnd - t.navigationStart;
    """)
    print(f"Tempo de carregamento: {tempo_ms} ms")
    assert tempo_ms < 5000, "Página demorou mais que 5s para carregar"
```

Aqui o Selenium está executando uma função JavaScript dentro do navegador para consultar uma métrica técnica de performance. Não é o caso de uso clássico da ferramenta, e resolve uma necessidade real sem exigir a introdução de outro stack.

Erros comuns em projetos de automação Selenium

Em projetos reais, alguns problemas aparecem com frequência grande o bastante para virarem padrão. Vale conhecer cada um antes de cair neles.

Uso abusivo de `time.sleep`. A tentação de colocar `time.sleep(5)` em todo lugar é grande, especialmente no começo. O problema é que esse comando trava o teste por um tempo fixo, mesmo quando o elemento já está pronto, e quebra o teste quando o elemento demora um pouco mais. A solução correta é usar esperas explícitas com `WebDriverWait` combinado com `expected_conditions`, que esperam exatamente até a condição desejada ocorrer, com um tempo máximo de tolerância.

Locators frágeis. Selecionar elementos por XPath absoluto ou por classes CSS geradas automaticamente é um caminho garantido para uma suíte de testes que quebra a cada deploy. Quando possível, dar preferência a `id`, depois `data-testid` (atributo criado especificamente para testes), depois CSS selectors estáveis. XPath fica como último recurso.

Falta de Page Object Model. Espalhar `driver.find_element(...)` pelo código de teste funciona para dez linhas. Para mil linhas, vira pesadelo de manutenção. Centralizar os elementos e ações de cada página em uma classe própria mantém o teste limpo e isola o impacto de mudanças no front-end.

Testes acoplados a dados de produção. Quando o teste depende de “o pedido 12345 que existe na base”, ele vai falhar no dia em que esse pedido for arquivado. Idealmente, o teste deve criar seus próprios dados ou rodar contra uma base controlada.

Testes que rodam só na máquina do desenvolvedor. Uma suíte automatizada que não roda no pipeline de CI tem o valor que o tempo da próxima refatoração permitir. Investir em rodar Selenium dentro do CI, em modo headless e idealmente com Selenium Grid ou um serviço como BrowserStack, multiplica o retorno.

Quando vale a pena, quando não vale

Depois de mapear onde o Selenium se encaixa e como ele costuma falhar, fica mais fácil tomar a decisão de usar a ferramenta. Alguns cenários onde Selenium tende a entregar bom retorno: aplicações web tradicionais, com fluxos relativamente estáveis, cuja validação manual está consumindo tempo demais da equipe. Sistemas em que é importante testar contra navegadores reais. Casos em que a regra de negócio só pode ser validada de ponta a ponta, atravessando front, back e banco.

Por outro lado, existem cenários em que outras ferramentas resolvem melhor. Para testar APIs sem interface, Selenium é exagero. Pytest com requests ou Postman dão conta com muito menos custo. Para testar performance pesada, o lugar é JMeter ou Gatling. Para testar aplicações mobile nativas, Appium ou Espresso são os caminhos. Para testar componentes isolados de front-end, Jest ou Vitest costumam ser mais rápidos. E em SPAs muito modernas, com muita assincronia, ferramentas como Cypress ou Playwright frequentemente oferecem uma experiência de desenvolvimento mais agradável.

Selenium funciona melhor quando o time tem clareza de que vai usar uma ferramenta de automação de UI e está disposto a investir em estrutura: Page Object, esperas explícitas, integração com CI e disciplina de manutenção. Sem essa estrutura, o que era para ser uma rede de proteção vira uma rede de tropeços.

Considerações Finais

A pergunta que abriu o capítulo seguiu sem resposta direta até aqui: em que momento da estratégia de qualidade o Selenium se encaixa? A resposta passa por entender que ele tem uma casa, o Quadrante 2, onde apoia a equipe validando regras de negócio através de fluxos completos na interface. E passa também por entender que ele visita os outros quadrantes em situações específicas, atuando como sonda em pipelines de CI no Q1, como apoio a testes manuais no Q3 e como coletor de métricas técnicas pontuais no Q4.

Esse enquadramento muda a forma de pensar a ferramenta. Em vez de tentar automatizar tudo com Selenium ou descartá-lo por completo em favor de alternativas mais novas, o ideal é reconhecer onde ele rende mais e onde rende menos. Para fluxos funcionais de aplicações web tradicionais, em equipes que automatizam com critério, o retorno costuma compensar. Para outros tipos de teste, a escolha sensata é combinar Selenium com ferramentas mais especializadas.

Vale registrar um último ponto. Nenhum framework de testes substitui o pensamento crítico do time. Um mapa, como o dos Quadrantes de Teste Ágil, ajuda a fazer escolhas mais conscientes, e não decide pelo profissional. O Selenium continua sendo uma ferramenta gratuita, madura, com comunidade ativa e suporte a múltiplas linguagens, e essas características explicam por que ele segue relevante mais de vinte anos depois de criado. Saber onde ele mora e onde ele visita é o que separa um projeto de automação que escala de um projeto que vira gargalo.

Bibliografia

CRISPIN, Lisa; GREGORY, Janet. Agile Testing: A Practical Guide for Testers and Agile Teams. Boston: Addison-Wesley, 2009.

DEQUE SYSTEMS. axe-core: Accessibility engine for automated testing. GitHub. Disponível em: <https://github.com/dequelabs/axe-core>. Acesso em: 15 jun. 2026.

FATEC ARARAS. Faculdade de Tecnologia de Araras. Disponível em: <https://fatecararas.cps.sp.gov.br/>. Acesso em: 15 jun. 2026.

HAEFFNER, Dave. The Internet: a site for testing automation. Disponível em: <https://the-internet.herokuapp.com/>. Acesso em: 15 jun. 2026.

MARICK, Brian. Exploration through example: Agile Testing Directions. 2003. Disponível em: <http://www.exampler.com/old-blog/2003/08/22.html>. Acesso em: 15 jun. 2026.

PYTEST. Pytest Documentation. Disponível em: <https://docs.pytest.org/en/stable/>. Acesso em: 15 jun. 2026.

SARAIVA JUNIOR, Orlando. ebook_qualidade_teste_software. GitHub. Disponível em: https://github.com/orlandosaraivajr/ebook_qualidade_teste_software. Acesso em: 15 jun. 2026.

SELENIUM. The Selenium Browser Automation Project. Disponível em: <https://www.selenium.dev/documentation/>. Acesso em: 15 jun. 2026.

WEBDRIVER MANAGER. webdriver-manager for Python. GitHub. Disponível em: https://github.com/SergeyPirogov/webdriver_manager. Acesso em: 15 jun. 2026.

Robot Framework

Gabriel de Oliveira Souza
gbrel860@gmail.com
Gabriel.souza78@aluno.cps.sp.gov.br

Resumo

Este documento apresenta de forma prática e detalhada o uso da ferramenta Robot Framework, explorando como ela revoluciona a automação de testes através de uma abordagem baseada em palavras-chave (*Keyword-Driven Testing*). O foco é demonstrar como essa ferramenta, construída em Python, facilita o empacotamento da lógica de testes, permitindo que eles sejam executados de forma consistente em qualquer ambiente, garantindo um ciclo de desenvolvimento mais ágil e confiável. Metodologicamente, desenvolveu-se um script de testes parametrizado contendo três cenários matemáticos executados em ambiente de nuvem através do Google Colab, simulando validações de regras de negócio. Os resultados alcançados comprovam a eficácia do framework na detecção imediata de erros por meio de asserções rigorosas e na geração automatizada de relatórios visuais e arquivos de log estruturados. Conclui-se que a centralização de dados em variáveis reduz drasticamente o débito técnico e o tempo gasto com manutenção corretiva de software.

Palavras-chaves: Robot Framework. Automação de Testes. Python. Fatec Araras.

Introdução

O Robot Framework é muito mais do que apenas uma ferramenta de automação; ele é um ecossistema completo e genérico projetado para lidar com Testes de Aceitação e Automação de Processos Robóticos (RPA). Sendo uma estrutura de código aberto (*Open Source*), ele permite que empresas e desenvolvedores criem soluções de automação sem os altos custos de licenciamento de ferramentas proprietárias. No cenário contemporâneo do desenvolvimento multiplataforma, a velocidade das entregas exige que a validação de software deixe de ser um gargalo manual e passe a ser um ativo estratégico, integrado diretamente ao ciclo de vida do projeto.

A grande vantagem do Robot está na sua capacidade de abstração. Ele atua como uma camada inteligente que fica entre o testador e o código complexo. Em vez de lidar diretamente com os detalhes técnicos de como um navegador clica em um botão, gerencia requisições assíncronas ou manipula o DOM (*Document Object Model*), o Robot permite que o usuário foque estritamente na regra de negócio. Ele ajuda a empacotar e isolar toda a lógica de teste em unidades modulares, garantindo que o comportamento do software possa ser validado de forma consistente em qualquer ambiente seja na

máquina local de um desenvolvedor, em um servidor de Integração Contínua (CI/CD) ou até em ambientes de nuvem efêmeros, como o Google Colab.

Essa abordagem mitiga drasticamente a ocorrência de falsos positivos nos testes, pois o isolamento do runtime garante que fatores externos não corrompam os resultados das asserções. Isso elimina as “configurações manuais exaustivas” que geralmente atrasam projetos de software e geram atrito entre as equipes de desenvolvimento e garantia de qualidade (QA). Ao padronizar a forma como os testes são escritos e executados, o Robot Framework garante que a qualidade do software não seja um processo subjetivo ou dependente do humor do avaliador, mas sim uma etapa técnica reproduzível, mensurável, auditável e altamente escalável.

Origem e Evolução

A ferramenta surgiu em 2005, dentro da Nokia Networks, idealizada por Pekka Klärck em sua tese de mestrado. O desafio original era criar uma forma de automatizar sistemas complexos de telecomunicações que pudesse ser compreendida tanto por desenvolvedores seniores quanto por engenheiros de outras áreas e gestores de produto. A palavra-chave aqui sempre foi comunicação: diminuir o abismo técnico entre quem entende do negócio e quem escreve o código.

Em 2008, o código foi liberado publicamente como *Open Source*, e hoje é mantido ativamente pela *Robot Framework Foundation*, uma organização sem fins lucrativos apoiada por diversas empresas globais de tecnologia. Sua arquitetura central é extremamente leve e extensível, baseada no conceito de plug-ins. Essa característica modular permite que o núcleo do framework permaneça intacto ao longo dos anos, enquanto sua capacidade de expansão via bibliotecas externas permite que ele seja acoplado a quase qualquer tecnologia atual, desde testes de APIs REST até interações complexas com inteligência artificial e visão computacional.

O Impacto da Automação no Ciclo de Vida do Software

Para compreender a relevância de frameworks como o Robot, é necessário analisar o impacto financeiro e operacional dos defeitos de software (*bugs*). De acordo com a teoria clássica da Engenharia de Software, o custo para corrigir um erro aumenta exponencialmente à medida que o sistema avança pelas etapas do ciclo de desenvolvimento. Um erro de cálculo detectado na fase de requisitos custa uma fração mínima se comparado ao mesmo erro descoberto pelo cliente final em um ambiente de produção.

Nesse contexto, o Robot Framework atua como uma ferramenta de *Shift-Left Testing*, uma prática que visa antecipar as rotinas de teste para o início do ciclo de desenvolvimento. Ao automatizar tarefas repetitivas, como a execução de cálculos matemáticos e fluxos de navegação, o framework liberta o capital humano para focar em testes exploratórios complexos e na arquitetura do sistema. O resultado direto para as organizações é a redução do *Time-to-Market* (tempo de lançamento no

mercado) e o estabelecimento de uma cultura de entrega contínua baseada em dados reais e métricas de cobertura de código irrefutáveis.

Desenvolvimento

Por que utilizar o Robot Framework?

Muitas vezes, falhas em testes de software acontecem porque a comunicação entre o que o teste faz em nível de código e o que a equipe de negócios ou gerência entende é falha. O Robot Framework mitiga esse problema de desalinhamento de expectativas através de diversos pilares técnicos que sustentam sua arquitetura:

1. **Abstração por Keywords (Palavras-chave):** Ao contrário de scripts complexos escritos em linguagens de programação puras (onde o fluxo se perde em meio a loops, condicionais e seletores complexos), o Robot utiliza palavras que descrevem ações humanas em linguagem natural (ex: "Fazer Login", "Verificar Saldo", "Calcular Valor Com Desconto"). Essa camada de abstração cria uma DSL (*Domain Specific Language*) que serve como documentação viva do projeto. Isso torna o ambiente de testes consistente, reduz o tempo de treinamento de novos membros e facilita drasticamente a manutenção do código a longo prazo.
2. **Agnosticismo de Tecnologia (Multiplataforma):** Como o foco do desenvolvimento atual é a entrega de soluções multiplataforma, o Robot se destaca por sua capacidade de interagir com diferentes camadas de uma aplicação usando exatamente a mesma sintaxe básica. É possível testar interfaces Web, aplicativos Mobile (Android e iOS), serviços de APIs REST/GraphQL e até integrações diretas em Bancos de Dados relacionais e não-relacionais. O desenvolvedor não precisa aprender um framework novo para cada tecnologia; ele mantém a estrutura lógica e altera apenas a biblioteca de suporte específica do alvo do teste.
3. **Relatórios e Evidências Nativas:** Um dos maiores diferenciais do Robot Framework é que ele não depende de plug-ins de terceiros para a geração de métricas. Ele gera automaticamente três arquivos essenciais e complementares imediatamente após a execução de qualquer comando no terminal: o log.html (focado no detalhamento técnico passo a passo para o desenvolvedor realizar o *debugging*), o report.html (um painel visual executivo com estatísticas de sucesso e falha, gráficos de pizza e resumos para apresentação gerencial) e o output.xml (que armazena os dados brutos estruturados, permitindo a integração nativa com esteiras automatizadas de CI/CD e ferramentas de análise de código).

Conceitos Fundamentais do Framework

Para o correto desenvolvimento de scripts de automação, é indispensável compreender a taxonomia e os componentes modulares que integram o ecossistema do Robot Framework:

- **Keywords (Palavras-chave):** São os blocos fundamentais de construção de qualquer fluxo de automação. Elas encapsulam sequências de comandos complexos em termos legíveis e reutilizáveis. O ecossistema divide-as em duas categorias: as *Built-in keywords* (funções nativas do núcleo do framework, responsáveis por interações básicas, manipulação de strings e operações lógicas) e as *Custom keywords* (criadas sob demanda pelo próprio desenvolvedor para encapsular regras matemáticas complexas, fluxos de autenticação específicos ou reaproveitar códigos Python customizados).
- **Libraries (Bibliotecas):** São módulos de extensão escritos em Python ou Java que adicionam novas capacidades e drivers ao núcleo do Robot Framework. A *SeleniumLibrary*, por exemplo, é a biblioteca responsável por prover a ponte de comunicação entre o script e as APIs dos navegadores web, permitindo o controle completo de elementos do DOM, preenchimento de formulários e simulação precisa de comportamentos de navegação humana.
- **Test Cases (Casos de Teste):** Trata-se da declaração ordenada, lógica e cronológica de ações e palavras-chave projetadas para validar se um comportamento de software atende aos requisitos de negócio estabelecidos. Cada caso de teste é estruturado de forma a isolar o escopo do cenário, utilizando dados centralizados na seção de variáveis e aplicando funções de asserção para definir se o status final da execução será classificado como sucesso ou como uma falha controlada para fins de auditoria de qualidade.

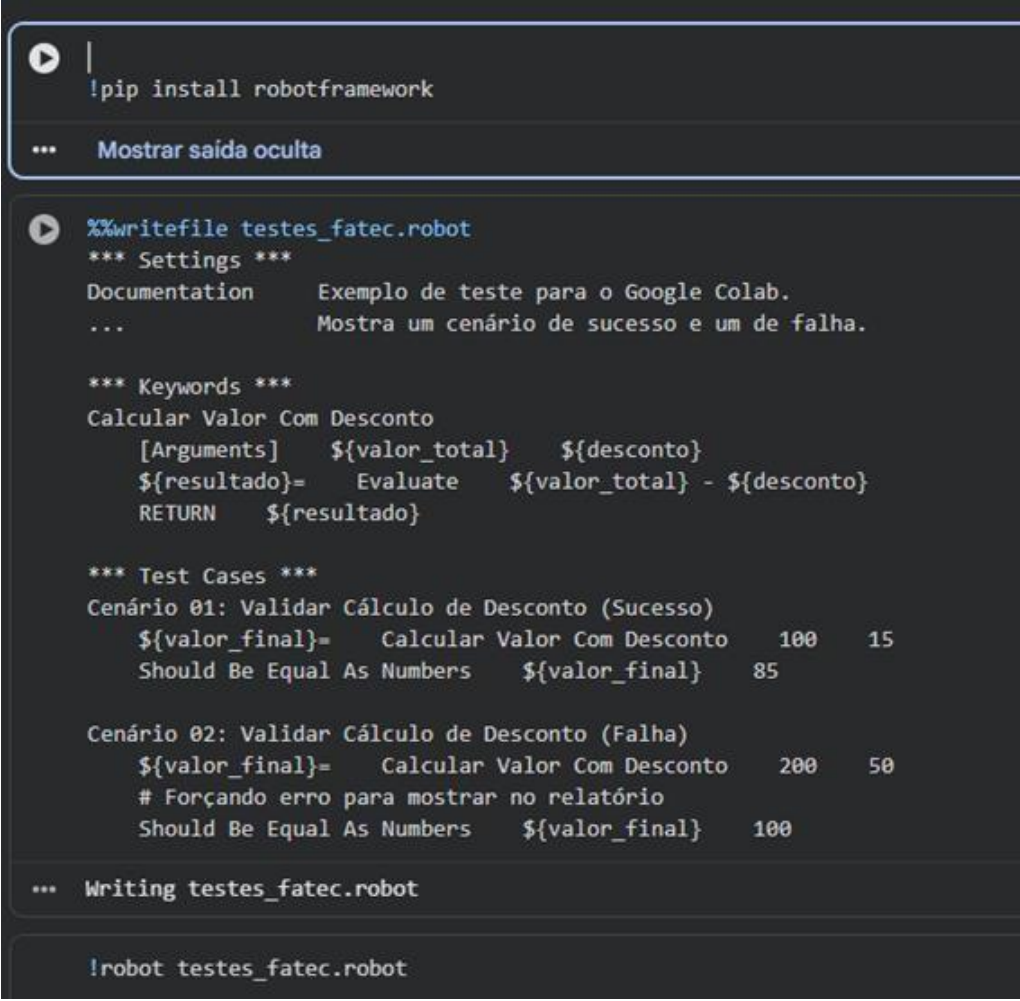
Instalação e Configuração Passo a Passo

A engenharia por trás do provisionamento do ambiente do Robot Framework foi projetada para garantir a reprodutibilidade em diferentes sistemas operacionais. A instalação é feita de forma estritamente modular utilizando o gerenciador de pacotes oficial do Python, o pip. O comando de instalação injeta o núcleo do robotframework e suas respectivas bibliotecas de forma isolada, o que garante a portabilidade do ambiente de testes tanto em sistemas baseados em Windows quanto em distribuições Linux ou ambientes efêmeros baseados em contêineres na nuvem.

No contexto do desenvolvimento local, a configuração do Ambiente Integrado de Desenvolvimento (IDE) desempenha um papel crítico na produtividade do desenvolvedor de QA. Ao utilizar o *Visual Studio Code*, a instalação da extensão oficial *Robot Framework Language Server* provê recursos indispensáveis para o fluxo de trabalho profissional, tais como o realce de sintaxe (*syntax highlighting*), ferramentas de autocompletar baseadas no contexto das bibliotecas importadas, validação de indentação e mecanismos de depuração visual que aceleram a identificação de erros de escrita antes mesmo da execução do script no terminal.

Por fim, a validação final para a execução de testes em aplicações reais exige a configuração dos componentes de comunicação de baixo nível, conhecidos como *WebDrivers* (como o ChromeDriver para o Google Chrome ou GeckoDriver para o Mozilla Firefox). Para mitigar os

problemas clássicos de quebra de testes causados pelo descompasso de versões entre o navegador local e o driver, a engenharia de automação moderna adota o uso de gerenciadores inteligentes, como a biblioteca webdriver-manager. Esse componente automatiza o download, a checagem de versão e a injeção do binário correto em tempo de execução, preparando o ambiente multiplataforma de forma automatizada e livre de intervenções manuais propensas a erros.



```
!pip install robotframework

... Mostrar saída oculta

%%writefile testes_fatec.robot
*** Settings ***
Documentation      Exemplo de teste para o Google Colab.
...               Mostra um cenário de sucesso e um de falha.

*** Keywords ***
Calcular Valor Com Desconto
    [Arguments]    ${valor_total}    ${desconto}
    ${resultado}=  Evaluate    ${valor_total} - ${desconto}
    RETURN        ${resultado}

*** Test Cases ***
Cenário 01: Validar Cálculo de Desconto (Sucesso)
    ${valor_final}=    Calcular Valor Com Desconto    100    15
    Should Be Equal As Numbers    ${valor_final}    85

Cenário 02: Validar Cálculo de Desconto (Falha)
    ${valor_final}=    Calcular Valor Com Desconto    200    50
    # Forçando erro para mostrar no relatório
    Should Be Equal As Numbers    ${valor_final}    100

... Writing testes_fatec.robot

!robot testes_fatec.robot
```

Fonte: próprio autor

A imagem ilustra o ambiente prático de testes automatizados orquestrado diretamente na nuvem através do ecossistema do Google Colab. Na primeira célula de execução apresentada, observa-se o uso do comando `!pip install robotframework`, que realiza a instalação dinâmica e o provisionamento do núcleo do framework no runtime efêmero da máquina virtual.

Logo em seguida, o script utiliza o comando mágico Jupyter `%%writefile testes_fatec.robot` para criar e estruturar fisicamente o arquivo de testes dentro do diretório do sistema. A anatomia do código gerado divide-se claramente em três blocos organizacionais:

- **Seção Settings:** Onde se define a documentação descritiva do escopo dos testes executados na suíte.
- **Seção Keywords:** Onde a palavra-chave customizada Calcular Valor Com Desconto é programada. Ela recebe os argumentos `{valor_total}` e `{desconto}` e invoca a biblioteca interna Evaluate para processar a expressão aritmética de subtração por meio do interpretador Python integrado, retornando o `{resultado}` encapsulado.
- **Seção Test Cases:** Onde estão declarados os dois primeiros cenários de validação da regra de negócio da aplicação. No **Cenário 01 (Sucesso)**, o framework calcula o desconto de 15 sobre o valor base de 100 e aplica a asserção aritmética Should Be Equal As Numbers para validar se o resultado real é idêntico à expectativa de 85. No **Cenário 02 (Falha)**, realiza-se o cálculo de um valor base de 200 com desconto de 50 (cujo resultado real é 150), mas insere-se deliberadamente uma asserção de expectativa incorreta de 100. Essa falha controlada é inserida com o objetivo técnico de auditar o rigor do framework e forçar a geração de evidências detalhadas de erro no relatório de saída gerado pelo comando de execução final! `robot testes_fatec.robot`.

Considerações Finais

O Robot Framework prova que a automação de alta performance não precisa ser um "bicho de sete cabeças" ou um privilégio de especialistas em codificação profunda. Ao utilizar contêineres de lógica através de *Keywords* e ambientes isolados via bibliotecas modulares, conseguimos criar um ciclo de testes muito mais confiável, rápido e, acima de tudo, compreensível para toda a equipe, desde desenvolvedores até gestores de produtos. A democratização dos testes automatizados, proporcionada pela sintaxe baseada em palavras-chave, redefine o papel da garantia de qualidade dentro das fábricas de software modernas.

Neste trabalho, a execução prática utilizando a infraestrutura do Google Colab demonstrou que a portabilidade e o agnosticismo tecnológico do framework são fundamentais para ambientes ágeis. A capacidade de simular cenários de sucesso e, primordialmente, orquestrar falhas controladas para auditar a precisão dos relatórios provou que a ferramenta é um escudo contra a regressão de *bugs* em sistemas complexos. O comportamento detalhado evidenciado nos arquivos de saída elimina o empirismo no diagnóstico de falhas, transformando dados brutos de execução em conhecimento técnico acionável e imediato para a equipe de engenharia de software.

Este material serve como um guia estratégico para quem deseja elevar o nível de seus projetos, garantindo que qualquer software, seja um simples script de automação em Python ou uma aplicação multiplataforma complexa e distribuída, funcione exatamente como o planejado, de forma padronizada, profissional e auditável através de relatórios automáticos. Recomenda-se, para trabalhos

futuros, a integração deste pipeline de testes automatizados com ferramentas de orquestração contínua (CI/CD) e a expansão do escopo para a validação de APIs REST com comportamento assíncrono, consolidando de forma definitiva a automação no cerne da arquitetura de desenvolvimento.

Bibliografia

[1] ROBOT FRAMEWORK. *Robot Framework User Guide*. Versão 7.4. Disponível em:

<https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html>. Acesso em:

25 mai. 2026.

[2] ROBOT FRAMEWORK. *SeleniumLibrary Documentation*. Disponível em:

<https://robotframework.org/SeleniumLibrary/SeleniumLibrary.html>. Acesso em: 25 mai. 2026.

Pytest na Prática: Marks, Fixtures e Cobertura em uma API

Kalliel Marcos Pinheiro
kallielmpinheiro2004@gmail.com
kalliel.pinheiro@aluno.cps.sp.gov.br

Resumo

Neste capítulo, abordaremos a implementação de testes com pytest visando garantir a qualidade do software, desde testes unitários até validações em nível de integração. Começaremos com uma introdução ao universo de testes, suas particularidades e taxonomia, avançando para o pytest como ferramenta facilitadora e seus plugins, que o tornam ainda mais poderoso. O objetivo final é demonstrar a viabilidade de testar uma API real e mostrar como escrever testes pode ser simples, natural e longe de ser uma mera duplicação de código.

Palavras-chaves: pytest, qualidade do software, testes unitários, API.

Introdução

Os testes são uma parte fundamental quando falamos de um sistema coeso e confiável. O pytest surge com uma missão simples, como descrito em sua própria documentação: "fácil de escrever, pequeno, reutilizável e escalável quando a complexidade aparecer".

Partindo para uma teoria básica de testes, Gerard Meszaros, em seu livro *xUnit Test Patterns: Refactoring Test Code*, divide a estrutura de um teste em 4 etapas:

1. Setup — preparação do ambiente e dos dados necessários para o teste;
2. Exercise — execução da unidade de código que está sendo testada;
3. Verify — verificação se o resultado obtido corresponde ao esperado;
4. Teardown — limpeza e desalocação dos recursos utilizados.

Kent Beck, por sua vez, propõe uma divisão similar em 3 fases comumente conhecida como Arrange, Act, Assert (AAA). A principal diferença em relação ao modelo de Meszaros está na ausência de uma etapa explícita de teardown, que no modelo de Beck tende a ser absorvida pelo próprio ambiente de testes ou por mecanismos automáticos de limpeza.

A taxonomia dos testes funciona como uma forma de classificar os testes de acordo com seu escopo e propósito. Na primeira camada temos os testes de unidade, que validam o comportamento

de uma unidade isolada de código, geralmente uma função ou método sem dependências externas. São os testes mais rápidos, mais numerosos e formam a base da pirâmide de testes.

Figura 1 – Exemplo de teste estruturado nas 4 fases de Meszaros

```
def test_add_item_to_cart():  
    # Setup  
    cart = []  
    item = {"product": "Notebook", "price": 3500.00, "qty": 1}  
  
    # Exercise  
    cart.append(item)  
    total = sum(i["price"] * i["qty"] for i in cart)  
  
    # Verify  
    assert len(cart) == 1  
    assert cart[0]["product"] == "Notebook"  
    assert total == 3500.00  
  
    # Teardown  
    cart.clear()
```

Fonte: Elaborado pelo autor (2026).

Para começar a utilizar o pytest, é necessário adicioná-lo ao projeto por meio de um gerenciador de dependências. As opções mais comuns no ecossistema Python são o pip, gerenciador nativo da linguagem, e o uv, uma alternativa moderna focada em velocidade:

- pip install -U pytest
- uv add pytest

Com o pytest instalado, dois recursos se destacam como pilares da ferramenta: os **marks** (marcadores) e as **fixtures**, cada um com propósitos bem definidos.

Os marks permitem categorizar e controlar a execução dos testes, é possível marcar um teste como lento, como dependente de um serviço externo, ou simplesmente pulá-lo sob determinadas condições. Já as fixtures são o mecanismo central de preparação de contexto no pytest: permitem criar, reutilizar e encerrar recursos compartilhados entre testes de forma declarativa, substituindo com elegância o manual Setup e Teardown que vimos anteriormente.

Desenvolvimento

Para ilustrar os conceitos na prática, desenvolvemos uma API simples utilizando Python, Django e Django Ninja para construir uma API REST com 4 endpoints que simulam transações bancárias. O pytest será a ferramenta responsável por garantir a qualidade do software ao longo de toda a suíte de testes. Nas próximas seções, aplicaremos de forma progressiva e didática os conceitos abordados até aqui, marks e fixtures, cobrindo desde a validação das regras de negócio até

o comportamento esperado de cada endpoint.

Figura 2 – Modelagem da Transação

```
class Transaction(models.Model):
    You, yesterday | 1 author (You)
    class Type(models.TextChoices):
        DEPOSIT = "deposit", "Depósito"
        WITHDRAWAL = "withdrawal", "Saque"
        TRANSFER = "transfer", "Transferência"
        PAYMENT = "payment", "Pagamento"

    You, yesterday | 1 author (You)
    class Status(models.TextChoices):
        PENDING = "pending", "Pendente"
        PROCESSING = "processing", "Processando"
        COMPLETED = "completed", "Concluída"
        FAILED = "failed", "Falhou"
        REVERSED = "reversed", "Estornada"

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)

    type = models.CharField(max_length=20, choices=Type.choices)
    status = models.CharField(
        max_length=20, choices=Status.choices, default=Status.PENDING
    )

    amount = models.DecimalField(max_digits=18, decimal_places=2)
    currency = models.CharField(max_length=3, default="BRL")

    source_account = models.CharField(max_length=34, blank=True)
    destination_account = models.CharField(max_length=34, blank=True)

    description = models.CharField(max_length=255, blank=True)

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    processed_at = models.DateTimeField(null=True, blank=True)

    You, yesterday | 1 author (You)
    class Meta:
        ordering = ["-created_at"]

    def __str__(self):
        return (
            f"{self.get_type_display()} {self.amount} {self.currency} ({self.status})"
        )
```

Fonte: Elaborado pelo autor (2026).

Conforme observado na Figura 2, este é o ponto de atuação das duas primeiras fases do ciclo de testes. No Setup, preparamos o ambiente e os dados necessários no contexto da nossa API, isso

significa definir a model que representa uma transação bancária, estabelecendo os campos, tipos e restrições que serão a base de todos os testes. No Exercise, utilizamos esses dados para construir os cenários que alimentarão os testes nas seções seguintes.

Figura 3 – Endpoints da Transação

```
@router.post("/transactions", response={201: TransactionOut})
def create_transaction(request, payload: TransactionIn):
    """cria 1 t -> registra uma nova transação."""
    transaction = Transaction.objects.create(**payload.dict())
    return 201, transaction

@router.get("/transactions", response=List[TransactionOut])
def list_transactions(request):
    """get n t -> lista todas as transações."""
    return Transaction.objects.all()

@router.get("/transactions/{transaction_id}", response=TransactionOut)
def get_transaction(request, transaction_id: UUID):
    """get 1 t -> busca uma transação pelo id."""
    return get_object_or_404(Transaction, id=transaction_id)

@router.delete("/transactions/{transaction_id}", response={204: None})
def delete_transaction(request, transaction_id: UUID):
    """deleta 1 t -> remove uma transação pelo id."""
    transaction = get_object_or_404(Transaction, id=transaction_id)
    transaction.delete()
    return 204, None
```

Fonte: Elaborado pelo autor (2026).

Conforme ilustrado na Figura 3, estes são os 4 endpoints que servirão de palco para os testes ao longo das próximas seções. Por meio deles, demonstraremos na prática como escrever testes eficazes, como identificar código morto, trechos que nunca são alcançados durante a execução, como analisar a cobertura linha a linha e como evoluir a qualidade do software de forma mensurável

Figura 4 – Conftest.py

```
import pytest
from django.test import Client
from payments.models import Transaction

BASE_URL = "/api/router_payment/transactions"

@pytest.fixture
def client():
    return Client()

@pytest.fixture
def base_url():
    return BASE_URL

@pytest.fixture
def transaction_payload():
    return {
        "type": Transaction.Type.DEPOSIT,
        "amount": "150.00",
        "currency": "BRL",
        "source_account": "12345678901234567890123456789012",
        "destination_account": "09876543210987654321098765432109",
        "description": "Depósito de teste",
    }

@pytest.fixture
def transaction(db, transaction_payload):
    return Transaction.objects.create(
        type=transaction_payload["type"],
        amount=transaction_payload["amount"],
        currency=transaction_payload["currency"],
        source_account=transaction_payload["source_account"],
        destination_account=transaction_payload["destination_account"],
        description=transaction_payload["description"],
    )
```

Fonte: Elaborado pelo autor (2026).

Conforme ilustrado na Figura 4, temos o nosso conftest.py. À primeira vista pode parecer simples, mas é aqui que reside um dos recursos mais poderosos do pytest no dia a dia: as fixtures.

Para entender seu valor, imagine uma suíte com 30 endpoints onde boa parte deles compartilha a mesma lógica de preparação, criar um cliente autenticado, inicializar o banco de dados, configurar headers. Sem o conftest.py, essa lógica seria repetida em cada arquivo de teste, violando diretamente o princípio DRY (Don't Repeat Yourself). É exatamente aqui que o conftest.py brilha.

Ele atua como um repositório central de fixtures, disponibilizando-as automaticamente para todos os testes do projeto sem nenhuma importação explícita. Ao invés de repetir a mesma lógica 30 vezes, ela é definida uma única vez e injetada por contexto, ou seja, passada como parâmetro de função para o teste que a solicitar. O teste sabe que aquela lógica funciona, mas não precisa saber como ela é construída, respeitando o princípio da responsabilidade única.

Figura 5 – Teste do Suite de Criação

```
@pytest.mark.create
class TestCreateTransaction:
    @pytest.mark.smoke # marcador de método: caminho feliz crítico
    def test_cria_transacao_retorna_201(self, client, base_url, transaction_payload):
        response = client.post(
            base_url, data=transaction_payload, content_type="application/json"
        )

        assert response.status_code == 201

    def test_cria_transacao_persiste_no_banco(
        self, client, base_url, transaction_payload
    ):
        assert Transaction.objects.count() == 0

        client.post(base_url, data=transaction_payload, content_type="application/json")

        assert Transaction.objects.count() == 1

    def test_cria_transacao_retorna_dados_enviados(
        self, client, base_url, transaction_payload
    ):
        response = client.post(
            base_url, data=transaction_payload, content_type="application/json"
        )

        body = response.json()
        assert body["type"] == transaction_payload["type"]
        assert body["amount"] == transaction_payload["amount"]
        assert body["currency"] == transaction_payload["currency"]
        # status default definido no model
        assert body["status"] == Transaction.Status.PENDING
        assert "id" in body
```

Fonte: Elaborado pelo autor (2026).

Conforme ilustrado na Figura 5, observamos o uso combinado de dois marks. O primeiro, `@pytest.mark.create`, atua no nível da classe e categoriza todos os testes do grupo como testes de criação, ou seja, testes que envolvem persistência em banco de dados e, portanto, possuem um

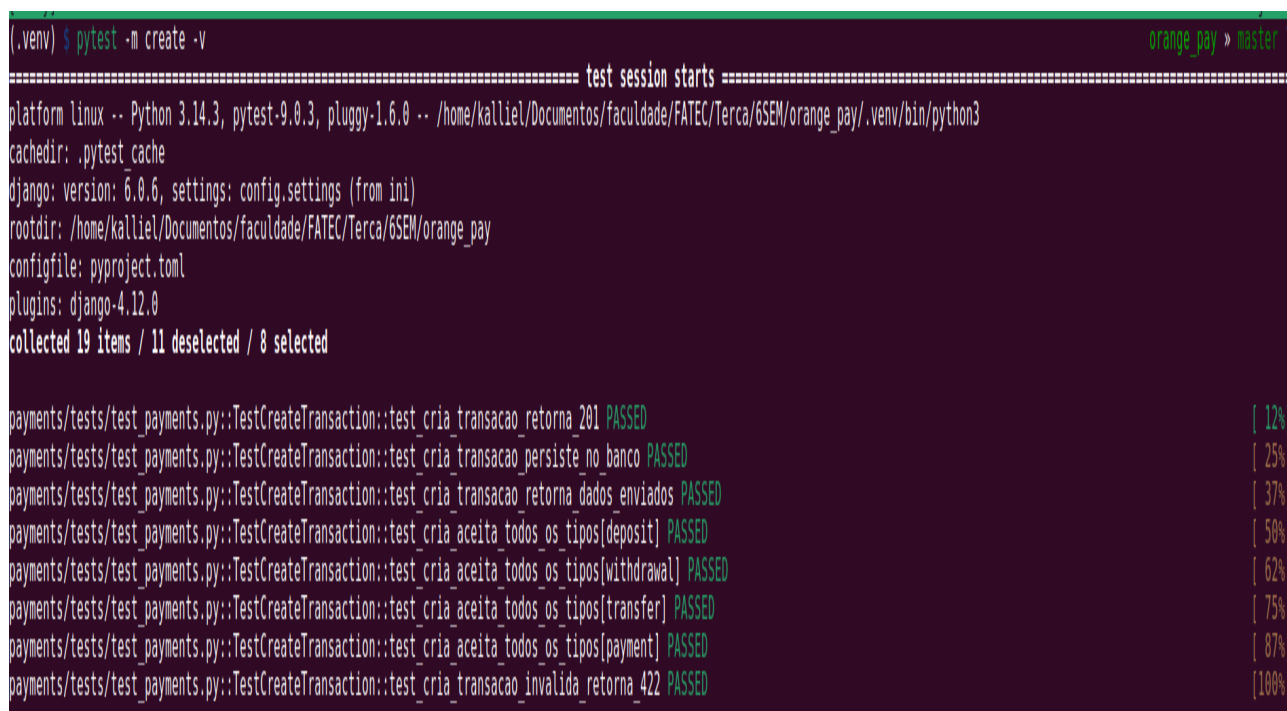
ponto crítico que precisa ser validado com atenção.

Dentro dessa classe, temos um segundo nível com `@pytest.mark.smoke`, aplicado ao método `test_cria_transacao_retorna_201`. O smoke test representa o caminho feliz, o fluxo principal e mais crítico da funcionalidade, aquele que, se quebrar, indica que algo fundamental está errado. É o primeiro teste que deve passar antes de qualquer outro ser executado.

Essa combinação de marks em dois níveis, classe e método, é uma das grandes vantagens do pytest: ela permite executar subconjuntos específicos da suíte de forma cirúrgica. Por exemplo:

- `pytest -m create`
roda apenas o caminho feliz de toda a suíte
- `pytest -m smoke`
roda apenas o caminho feliz dos testes de criação
- `pytest -m "create and smoke"`

Figura 6 – Execução dos testes



```
(.venv) $ pytest -m create -v
===== test session starts =====
platform linux -- Python 3.14.3, pytest-9.0.3, pluggy-1.6.0 -- /home/kalliel/Documentos/faculdade/FATEC/Terca/6SEM/orange_pay/.venv/bin/python3
cachedir: .pytest cache
django: version: 6.0.6, settings: config.settings (from ini)
rootdir: /home/kalliel/Documentos/faculdade/FATEC/Terca/6SEM/orange_pay
configfile: pyproject.toml
plugins: django-4.12.0
collected 19 items / 11 deselected / 8 selected

payments/tests/test_payments.py::TestCreateTransaction::test_cria_transacao_retorna_201 PASSED [ 12%]
payments/tests/test_payments.py::TestCreateTransaction::test_cria_transacao_persiste_no_banco PASSED [ 25%]
payments/tests/test_payments.py::TestCreateTransaction::test_cria_transacao_retorna_dados_enviados PASSED [ 37%]
payments/tests/test_payments.py::TestCreateTransaction::test_cria_aceita_todos_os_tipos[deposit] PASSED [ 50%]
payments/tests/test_payments.py::TestCreateTransaction::test_cria_aceita_todos_os_tipos[withdrawal] PASSED [ 62%]
payments/tests/test_payments.py::TestCreateTransaction::test_cria_aceita_todos_os_tipos[transfer] PASSED [ 75%]
payments/tests/test_payments.py::TestCreateTransaction::test_cria_aceita_todos_os_tipos[payment] PASSED [ 87%]
payments/tests/test_payments.py::TestCreateTransaction::test_cria_transacao_invalida_retorna_422 PASSED [100%]
```

Fonte: Elaborado pelo autor (2026).

Conforme ilustrado na Figura 6, podemos constatar que ao todo temos 19 testes, dos quais apenas 8 foram executados, exatamente aqueles marcados com a flag `create`. Isso demonstra na prática o poder dos marks: com um simples filtro na linha de comando, é possível isolar um subconjunto específico da suíte e garantir que apenas os testes relevantes sejam executados em determinado momento.

Esse comportamento abre caminho para estratégias mais robustas de qualidade, como a

criação de cronjobs ou pipelines automatizadas que garantam que o mínimo de testes passe antes de aceitar um pull request ou promover um build para produção, evitando que regressões silenciosas causem transtornos em ambiente produtivo.

A seguir, apresentaremos um complemento natural a essa abordagem: a cobertura de testes com o `pytest-cov`. Diferente dos recursos que vimos até aqui, o `pytest-cov` não é nativo do `pytest`, mas sim um plugin que se integra ao seu ecossistema e nos permite enxergar, linha a linha, quais partes do código estão sendo exercitadas pelos testes e quais permanecem invisíveis a eles

Para instalar o `pytest-cov`, basta adicioná-lo ao projeto pelo gerenciador de dependências de sua preferência:

- `pip install pytest-cov`
- `uv add pytest-cov`

Com o plugin instalado, a cobertura é gerada junto à execução dos testes com a flag `-cov`:

cobertura do módulo `payments`

- `pytest --cov=payments`

cobertura com relatório linha a linha no terminal

- `pytest --cov=payments --cov-report=term-missing`

cobertura com relatório em HTML — navegável no browser

- `pytest --cov=payments --cov-report=html`

O relatório `term-missing` é especialmente útil no dia a dia: ele exibe diretamente no terminal quais linhas do código não foram tocadas por nenhum teste, permitindo identificar com precisão os pontos cegos da suíte.

Figura 7 – Cobertura dos testes



Fonte: Elaborado pelo autor (2026).

Conforme ilustrado na Figura 7, observamos o relatório gerado pelo comando `pytest --cov=payments --cov-report=html`. No relatório HTML, a leitura da cobertura é visual e intuitiva:

- Linhas verdes: foram executadas por ao menos um teste;
- Linhas parcialmente cobertas (pontilhadas ou amareladas): foram executadas, mas possuem ramificações não exercitadas, como um `if` cujo caminho alternativo nunca foi testado;
- Linhas vermelhas: não foram tocadas por nenhum teste, indicando código morto ou

fluxos completamente ignorados pela suíte.

No caso do nosso `test_payments.py`, o relatório aponta 100% de cobertura — todas as 85 instruções foram executadas, sem nenhuma linha descoberta. Um resultado que reflete a atenção dedicada a cobrir os diferentes cenários de cada endpoint.

Vale ressaltar que não nos aprofundaremos em todos os plugins disponíveis no ecossistema do `pytest`, pois cada um deles renderia um capítulo à parte. O objetivo aqui foi apresentar o `pytest-cov` como um complemento natural à suíte de testes, oferecendo visibilidade sobre o que está sendo testado e, mais importante, sobre o que não está.

Considerações Finais

Após essa leitura, é natural que surja o entusiasmo de sair implementando testes em massa e isso é ótimo. Porém, vale ressaltar: testes não são uma bala de prata. Eles não eliminam bugs por si só, nem garantem um sistema perfeito. O que garantem, quando bem aplicados, é visibilidade, confiança e previsibilidade, e isso faz toda a diferença no ciclo de vida de um software.

Qualidade de software é um conjunto de boas práticas, e os testes são uma peça importante desse conjunto, não a única. A dica que carrego no meu dia a dia como desenvolvedor e que deixo aqui é simples: foque na base. Boas práticas e fundamentos que funcionam há décadas continuam funcionando porque foram testados à exaustão pela indústria. Domine-os antes de correr para as novidades.

Este capítulo poderia ter sido escrito com qualquer outra ferramenta do ecossistema de testes. A escolha pelo `pytest` foi intencional, não por ser a mais moderna ou a mais completa, mas por refletir aquilo em que acredito que realmente funciona na prática. A base sólida sempre será mais valiosa do que a ferramenta da moda.

Abaixo deixo algumas fontes ricas em conteúdo que estudei, usei, aprovei e comprovei na prática, e que recomendo para quem quiser se aprofundar além do que foi abordado aqui. Todas estão devidamente listadas nas referências ao final do e-book.

Bibliografia

PYTEST. Documentation. Disponível em: <https://docs.pytest.org/en/stable/>. Acesso em: 11 nov. 2024.cesso em: 11 jun. 2026.

MENDES, Eduardo. O mínimo que você deveria saber sobre testes unitários – #30diasdepython.

YouTube, 2024. Disponível em: <https://youtu.be/pZvhZ-Lr-PE>. Acesso em: 11 jun. 2026.

MENDES, Eduardo. Pytest: uma introdução – Live de Python #167. YouTube, 2021. Disponível em: <https://www.youtube.com/live/MjQCvJmc31A>. Acesso em: 11 jun. 2026.

MENDES, Eduardo. Pytest fixtures – Live de Python #168. YouTube, 2021. Disponível em: https://www.youtube.com/live/sidi9Z_IkLU. Acesso em: 11 jun. 2026.

PINHEIRO, Kalliel. orange_pay: repositório de referência para testes com pytest. GitHub, 2026. Disponível em: https://github.com/Kallielmpinheiro/orange_pay. Acesso em: 11 jun. 2026.

Testes Automatizados no Laravel com PHPUnit

Marco Antonio Amorim Filho

marcoamorim877@gmail.com

Resumo

Este e-book apresenta a utilização do PHPUnit no contexto do framework Laravel, uma das soluções mais populares para automação de testes em aplicações desenvolvidas na linguagem PHP. O objetivo principal é demonstrar como a implementação de testes automatizados contribui para a qualidade do software, aumentando a confiabilidade do código, facilitando a identificação de falhas e apoiando práticas modernas de desenvolvimento, como o Test-Driven Development (TDD) e a Integração Contínua (CI). A abordagem adotada combina a fundamentação teórica dos testes de software com a exploração prática dos recursos oferecidos pelo Laravel em conjunto com o PHPUnit, por meio da construção de exemplos e cenários reais de validação. Ao longo do material, são apresentados conceitos essenciais, como a criação de funções de teste, o uso de assertivas, a aplicação da trait RefreshDatabase para isolamento do banco de dados, o uso de factories e a validação de exceções por meio do expectException. Os resultados evidenciam que o ecossistema Laravel oferece uma solução integrada, intuitiva e escalável para o desenvolvimento de testes automatizados, sendo amplamente utilizado tanto em ambientes acadêmicos quanto corporativos. Conclui-se que sua adoção contribui para a padronização dos processos de teste, promove maior segurança durante a manutenção do código e fortalece a cultura de qualidade no desenvolvimento de software.

Palavras-chave: *PHPUnit, Laravel, testes automatizados, PHP, qualidade de software.*

Introdução

A qualidade de software é um dos pilares fundamentais no desenvolvimento de sistemas modernos, exigindo práticas que garantam confiabilidade, manutenção contínua e redução de falhas ao longo de todo o ciclo de vida do produto. Entre

essas práticas, os testes automatizados desempenham papel estratégico, pois permitem validar pequenas partes do código de forma isolada e, ao mesmo tempo, verificar o comportamento integrado de toda a aplicação, assegurando que cada componente execute corretamente sua função.

Nesse contexto, este e-book dedica-se ao estudo dos testes automatizados no Laravel, framework PHP amplamente adotado no mercado, com utilização do PHPUnit como ferramenta de execução de testes. O Laravel é reconhecido por oferecer uma estrutura completa e pronta para uso, na qual o ambiente de testes já vem configurado por padrão desde a criação do projeto. O escopo do trabalho concentra-se na apresentação dos conceitos fundamentais relacionados aos testes automatizados, na análise das principais funcionalidades oferecidas pelo Laravel em conjunto com o PHPUnit e na demonstração prática de sua aplicação em um projeto de software.

O objetivo geral é compreender de que forma a automatização de testes contribui para a melhoria da qualidade do software e como o Laravel auxilia nesse processo. Como objetivos específicos, destacam-se: identificar as características e vantagens da ferramenta, analisar sua estrutura de funcionamento e apresentar exemplos práticos que evidenciem sua utilização em diferentes cenários de desenvolvimento.

A escolha do tema justifica-se pela ampla adoção do Laravel na indústria de software brasileira e internacional, especialmente devido à sua sintaxe expressiva, ao suporte nativo a recursos de teste, à facilidade de integração com processos modernos de desenvolvimento, como Integração Contínua (CI) e Desenvolvimento Orientado a Testes (TDD), e à robustez do ecossistema PHP.

A metodologia utilizada envolveu pesquisa bibliográfica, análise da documentação oficial do framework Laravel e do PHPUnit, e experimentação prática por meio da implementação de casos de teste em PHP. A organização deste e-book segue três etapas principais: inicialmente, são apresentados os conceitos fundamentais dos testes automatizados e do ambiente de testes do Laravel; em seguida, são explorados seus principais recursos e exemplos de aplicação; por fim, são apresentadas as considerações finais, destacando os benefícios da ferramenta e possíveis direções para estudos futuros.

Visão Geral

O Laravel é um framework PHP amplamente utilizado no desenvolvimento de aplicações web devido à sua sintaxe expressiva, à completude de seus recursos e à facilidade de uso. Ele oferece, de forma nativa, um ambiente completo de testes baseado no PHPUnit, permitindo criar testes de forma direta, utilizando funções e classes da linguagem PHP e assertivas fornecidas tanto pelo PHPUnit quanto pelas extensões específicas do Laravel, tornando o código de teste mais legível e intuitivo.

Além da execução de testes unitários, o ecossistema do Laravel oferece suporte para testes de integração, testes HTTP simulados, testes envolvendo banco de dados, mocking de serviços externos e até mesmo testes de navegador por meio do pacote Laravel Dusk. Outro diferencial importante é a capacidade de descobrir automaticamente arquivos e funções de teste, reduzindo a necessidade de configurações adicionais.

Sua arquitetura extensível permite a utilização de pacotes desenvolvidos pela comunidade, ampliando funcionalidades como medição de cobertura de código, execução paralela e integração com pipelines de Integração Contínua (CI). Dessa forma, o Laravel tornou-se uma das ferramentas mais adotadas para o desenvolvimento de aplicações PHP com cobertura de testes automatizados.

Breve histórico

O PHPUnit foi criado por Sebastian Bergmann em 2001 com inspiração no JUnit, framework de testes para a linguagem Java. Seu objetivo era oferecer uma alternativa robusta para a automação de testes em aplicações PHP, em um momento em que essa prática ainda era pouco difundida no ecossistema. Ao longo dos anos, o PHPUnit consolidou-se como a principal ferramenta de testes da linguagem PHP.

O Laravel, por sua vez, foi criado por Taylor Otwell em 2011 com o objetivo de oferecer um framework PHP mais expressivo, produtivo e moderno. Desde suas primeiras versões, o framework incorporou o PHPUnit como dependência padrão, fornecendo um ambiente de testes pré-configurado em cada novo projeto. Esta integração nativa entre Laravel e PHPUnit tornou a prática de testes automatizados acessível a desenvolvedores iniciantes e avançados.

Atualmente, o Laravel é considerado um dos frameworks PHP mais populares do mundo, sendo utilizado tanto em projetos acadêmicos quanto em aplicações corporativas de grande porte. Sua adoção crescente está relacionada à facilidade de uso, à clareza dos relatórios gerados pelo PHPUnit, à eficiência no desenvolvimento de testes automatizados e ao suporte mais recente também a sintaxes modernas como a do framework Pest, que oferece uma camada de escrita mais expressiva sobre o próprio PHPUnit.

Ambiente de testes

No contexto do Laravel, o ambiente de testes corresponde ao conjunto de recursos e práticas que permitem criar, organizar e executar verificações automatizadas de forma eficiente. Entre os principais elementos desse ambiente, destacam-se:

- **Arquivos de teste:** normalmente organizados em módulos específicos dentro das pastas `tests/Unit` e `tests/Feature`, com nomes finalizados pelo sufixo "Test", facilitando sua identificação automática pelo framework.
- **Funções de teste:** implementadas como métodos de classe, geralmente precedidas pelo prefixo `test_`, permitindo que o PHPUnit as reconheça automaticamente.
- **Assertivas:** o framework utiliza métodos assertivos fornecidos pelo PHPUnit, como `assertEquals`, `assertTrue`, `assertStatus` e `assertDatabaseHas`, tornando os testes mais simples e legíveis.
- **Factories:** mecanismo utilizado para gerar dados falsos de forma rápida e padronizada, preparando o ambiente necessário para os testes e promovendo reutilização.
- **RefreshDatabase:** trait disponibilizada pelo Laravel que envolve cada teste em uma transação de banco de dados, executando rollback automático ao final, mantendo o ambiente sempre limpo.
- **Execução por linha de comando:** realizada por meio do comando `php artisan test`, que executa automaticamente todos os testes encontrados no projeto e apresenta relatórios formatados.

- **Relatórios de execução:** apresentam informações detalhadas sobre testes aprovados, falhas, erros e estatísticas gerais de execução, com saída colorida no terminal.

Esse ambiente simplifica o processo de automação de testes e possibilita sua integração com ferramentas de desenvolvimento, versionamento e integração contínua, tornando-o adequado para projetos de diferentes portes.

Principais Conceitos e Estruturas do PHPUnit no Laravel

Para simplificar a criação e manutenção de testes automatizados, o Laravel disponibiliza, em conjunto com o PHPUnit, diversos recursos que tornam o desenvolvimento mais produtivo e organizado. Entre os principais, destacam-se:

- **Assertivas:** utilizadas para verificar se um resultado corresponde ao esperado. Métodos como `assertEquals`, `assertTrue`, `assertFalse`, `assertCount` e `assertNull` validam diferentes tipos de condições e fornecem mensagens de erro detalhadas quando uma validação falha.
- **Assertivas HTTP:** métodos específicos para verificar respostas de requisições simuladas, como `assertStatus`, `assertOk`, `assertSee` e `assertRedirect`, permitindo testar rotas e controllers de forma simples.
- **Assertivas de banco de dados:** métodos como `assertDatabaseHas`, `assertDatabaseMissing`, `assertDatabaseCount`, `assertModelExists` e `assertModelMissing` permitem validar o estado dos dados persistidos durante os testes.
- **expectException:** recurso utilizado para validar exceções esperadas. Permite verificar se determinada operação gera o erro previsto, garantindo o tratamento adequado de situações excepcionais.
- **Trait RefreshDatabase:** incluída em uma classe de teste, faz com que cada método de teste seja envolvido em uma transação e revertido ao final, mantendo o banco em estado conhecido entre testes sem afetar dados reais.
- **Factories:** classes responsáveis por gerar instâncias de modelos com dados fictícios, permitindo criar registros completos em poucas linhas e reduzindo duplicação de código nos testes.

- **Mocking:** recurso que permite substituir comportamentos reais de serviços externos por simulações, como `Cache::expects`, `Mail::fake` e `Queue::fake`, isolando o teste de dependências externas.
- **Test Discovery (descoberta automática de testes):** funcionalidade que identifica automaticamente arquivos e classes de teste no projeto, dispensando configurações complexas para sua execução.
- **Filtragem de testes:** por meio da opção `--filter`, é possível executar apenas um subconjunto específico dos testes, agilizando o desenvolvimento de funcionalidades isoladas.

Esses recursos tornam o Laravel uma plataforma flexível e eficiente para a automação de testes, contribuindo para a qualidade, confiabilidade e manutenção do software.

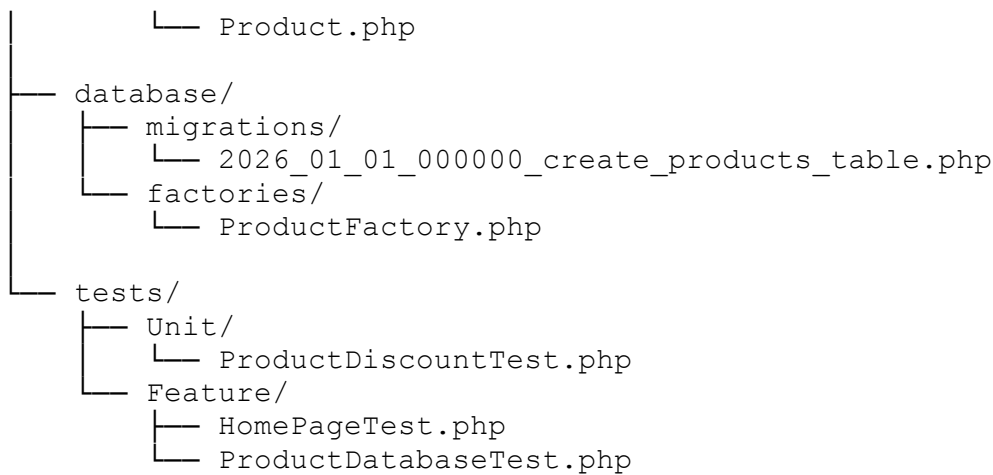
Desenvolvimento

Para o desenvolvimento deste projeto, foi realizada uma divisão em duas etapas principais. A primeira corresponde à pesquisa bibliográfica e documental, com foco no estudo da documentação oficial do Laravel e do PHPUnit, buscando compreender seus conceitos, recursos e funcionamento. A segunda etapa consistiu na aplicação prática, envolvendo a implementação de um sistema simples de gestão de produtos em PHP acompanhado da criação de testes automatizados utilizando o ambiente de testes do Laravel.

Essa prática teve como objetivo validar funcionalidades específicas de um modelo de produtos e demonstrar, de forma concreta, como os testes automatizados contribuem para a qualidade e confiabilidade do software. Foram explorados recursos importantes do Laravel em conjunto com o PHPUnit, como assertivas, factories, a trait `RefreshDatabase` e validação de exceções.

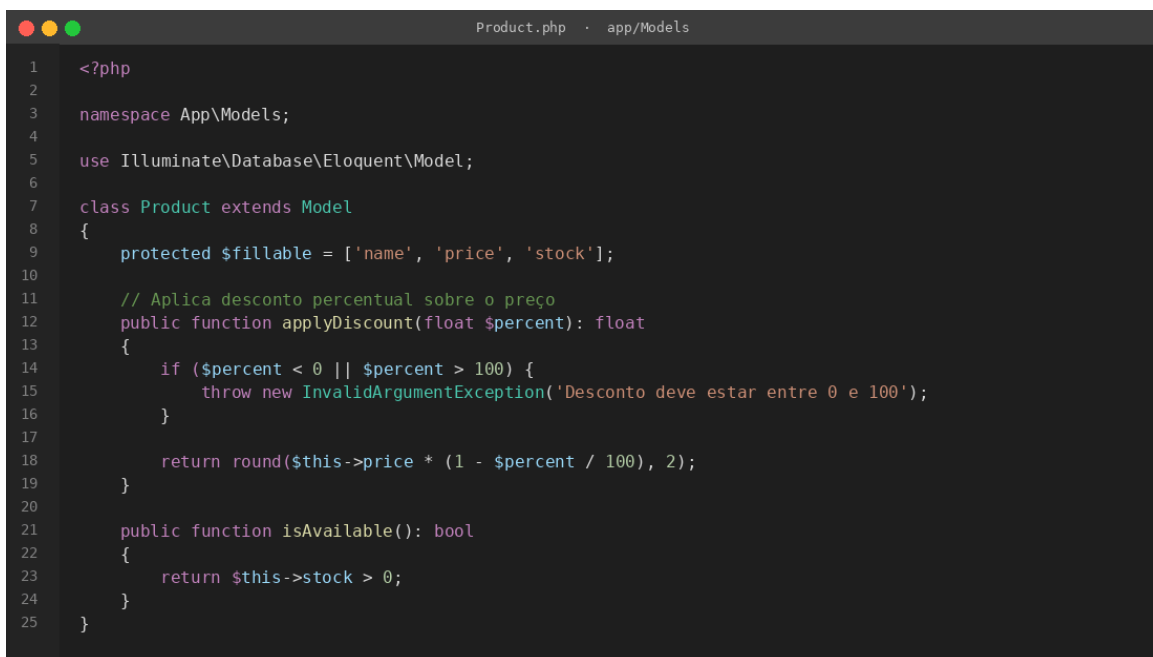
Seguindo o roteiro de demonstração, inicialmente foi criada a estrutura de diretórios do projeto para separar o código da aplicação dos arquivos de teste, conforme a organização padrão sugerida pelo próprio framework:

```
projeto_produtos/  
├── app/  
│   └── Models/  
└── tests/
```



No arquivo `Product.php` foi implementada a classe responsável pelas operações de um produto, incluindo a aplicação de descontos percentuais sobre o preço, com validação de regras de negócio, e a verificação de disponibilidade em estoque.

Figura 1



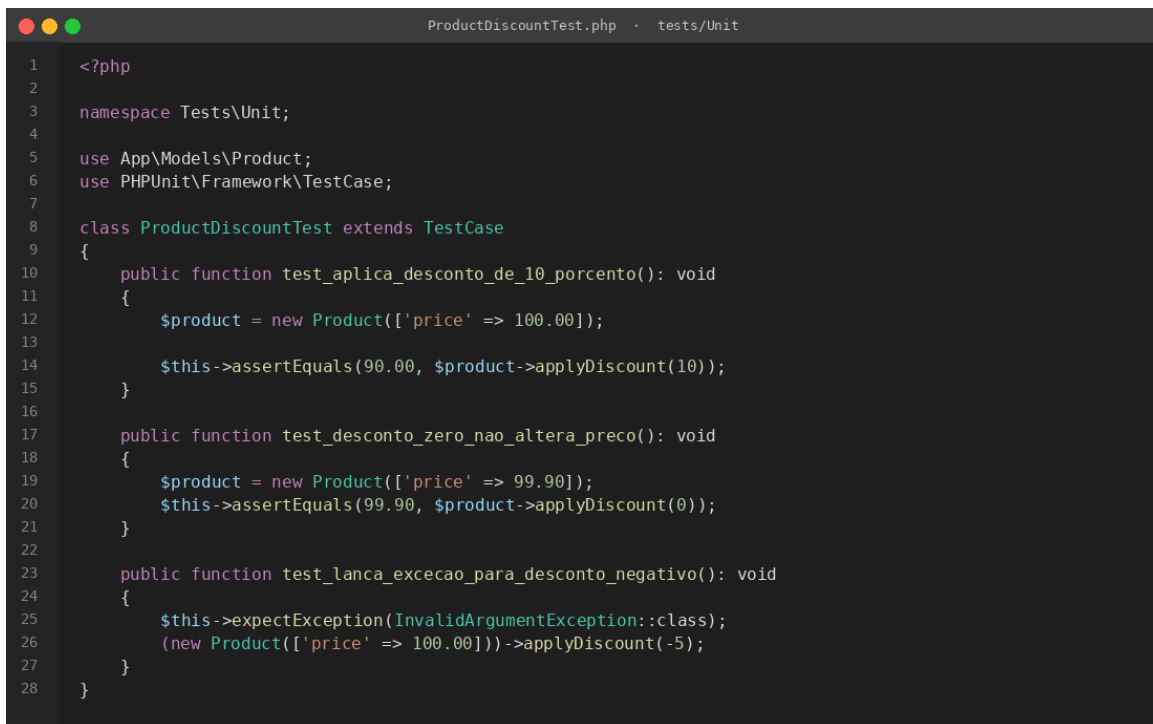
```

1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6
7  class Product extends Model
8  {
9      protected $fillable = ['name', 'price', 'stock'];
10
11      // Aplica desconto percentual sobre o preço
12      public function applyDiscount(float $percent): float
13      {
14          if ($percent < 0 || $percent > 100) {
15              throw new InvalidArgumentException('Desconto deve estar entre 0 e 100');
16          }
17
18          return round($this->price * (1 - $percent / 100), 2);
19      }
20
21      public function isAvailable(): bool
22      {
23          return $this->stock > 0;
24      }
25  }

```

Fonte: próprio autor

Após a implementação da classe, foram criados testes automatizados para validar o comportamento do modelo de produtos. Os testes verificam diferentes cenários, incluindo aplicação de desconto válido, desconto igual a zero e lançamento de exceção para valores inválidos. A organização do código de teste segue o padrão esperado pelo PHPUnit, com classe que estende `TestCase` e métodos com prefixo `test_`.

Figura 2

```
1  <?php
2
3  namespace Tests\Unit;
4
5  use App\Models\Product;
6  use PHPUnit\Framework\TestCase;
7
8  class ProductDiscountTest extends TestCase
9  {
10     public function test_aplica_desconto_de_10_porcento(): void
11     {
12         $product = new Product(['price' => 100.00]);
13
14         $this->assertEquals(90.00, $product->applyDiscount(10));
15     }
16
17     public function test_desconto_zero_nao_altera_preco(): void
18     {
19         $product = new Product(['price' => 99.90]);
20         $this->assertEquals(99.90, $product->applyDiscount(0));
21     }
22
23     public function test_lanca_excecao_para_desconto_negativo(): void
24     {
25         $this->expectException(InvalidArgumentException::class);
26         (new Product(['price' => 100.00]))->applyDiscount(-5);
27     }
28 }
```

Fonte: próprio autor

Os testes desenvolvidos verificam diferentes cenários de utilização do sistema. Foram avaliadas operações válidas, como o cálculo correto do desconto, além de situações excepcionais, como tentativas de aplicação de descontos negativos ou superiores a cem por cento, que devem resultar no lançamento de uma exceção.

Para permitir que o Laravel encontre automaticamente os testes, foi adotada a seguinte convenção:

- Os arquivos de teste possuem o sufixo Test em seu nome (por exemplo, ProductDiscountTest.php);
- As funções de teste iniciam com o prefixo test_;
- Os testes foram organizados nas pastas tests/Unit e tests/Feature;
- A execução foi realizada por meio do comando:

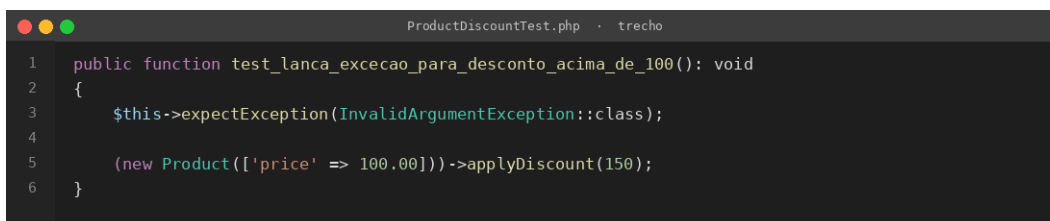
```
php artisan test
```

Além dos cenários de sucesso, foram implementados testes utilizando o método expectException para validar o tratamento correto de exceções. Também foi utilizada a trait RefreshDatabase nos testes que envolvem o banco de dados,

permitindo a reutilização da configuração inicial e o isolamento entre os diferentes casos de teste.

Os resultados obtidos demonstraram que o Laravel oferece uma forma simples e eficiente de automatizar testes em aplicações PHP, contribuindo para a redução de erros, aumento da confiabilidade do código e maior facilidade de manutenção do software.

Figura 3

A imagem mostra uma janela de editor de código com o título "ProductDiscountTest.php · trecho". O código é o seguinte:

```
1 public function test_lanca_excecao_para_desconto_acima_de_100(): void
2 {
3     $this->expectException(InvalidArgumentException::class);
4
5     (new Product(['price' => 100.00]))->applyDiscount(150);
6 }
```

Fonte: próprio autor

Para demonstrar a capacidade do framework em validar situações de erro, foi implementado um teste utilizando o método `expectException`. Nesse cenário, a aplicação tenta aplicar um desconto superior a cem por cento sobre o produto, verificando se a exceção esperada (`InvalidArgumentException`) é lançada corretamente. Esse tipo de teste é importante para garantir que as regras de negócio sejam respeitadas e que comportamentos inadequados sejam tratados pelo sistema.

Ao executar o comando de teste, todos os casos implementados são identificados automaticamente pelo Laravel e executados em sequência. Ao final da execução, o framework apresenta um relatório contendo informações sobre os testes aprovados, falhas encontradas e possíveis erros.

Figura 4


```

Terminal - php artisan test

$ php artisan test

PASS Tests\Unit\ProductDiscountTest
✓ aplica desconto de 10 por cento 0.04s
✓ aplica desconto de 50 por cento 0.01s
✓ desconto zero nao altera preco 0.01s
✓ lanca excecao para desconto negativo 0.01s
✓ lanca excecao para desconto acima de 100 0.01s

PASS Tests\Feature\HomePageTest
✓ pagina inicial retorna status 200 0.18s
✓ pagina inicial tem texto laravel 0.02s
✓ rota inexistente retorna 404 0.03s

PASS Tests\Feature\ProductDatabaseTest
✓ pode criar um produto no banco 0.12s
✓ pode contar produtos criados 0.04s
✓ banco comeca vazio em cada teste 0.03s
✓ produto deletado some do banco 0.05s

Tests: 12 passed (24 assertions)
Duration: 0.61s

```

Fonte: próprio autor

As saídas apresentadas pelo Laravel em conjunto com o PHPUnit podem ser classificadas da seguinte forma:

- **PASS:** o teste foi executado com sucesso e o resultado obtido corresponde ao esperado;
- **FAIL:** uma assertiva falhou, indicando que o resultado obtido é diferente do resultado esperado;
- **ERROR:** ocorreu uma exceção inesperada durante a execução do teste ou durante a configuração do ambiente de teste;
- **SKIPPED:** o teste foi ignorado intencionalmente por alguma configuração definida pelo desenvolvedor, geralmente por meio do método `markTestSkipped`.

É importante destacar que, além dos recursos explorados neste projeto, o Laravel disponibiliza diversas funcionalidades avançadas que ampliam significativamente sua aplicação em cenários profissionais. Entre elas, destacam-se as factories parametrizadas, que permitem reutilizar configurações em múltiplos testes; a utilização de provedores de dados via `@dataProvider`, que possibilita executar o mesmo teste com diferentes conjuntos de dados; e o amplo ecossistema de pacotes, que adiciona funcionalidades como geração de relatórios detalhados, medição de cobertura de código e execução paralela.

Outro recurso bastante utilizado é o sistema de mocking, integrado ao Laravel por meio das fachadas de teste (`Cache::expects`, `Mail::fake`, `Queue::fake`, `Notification::fake`, `Event::fake`, entre outras) e da biblioteca Mockery. Essa técnica permite simular dependências externas, como envio de e-mails, processamento de filas, chamadas a APIs ou serviços de terceiros, tornando os testes mais rápidos, previsíveis e independentes do ambiente externo.

Adicionalmente, o ecossistema do Laravel oferece o pacote Laravel Dusk, voltado para testes de navegador automatizados. Por meio dele, é possível simular um usuário real interagindo com a aplicação em um navegador Chrome controlado pelo ChromeDriver, permitindo verificar fluxos completos, como login, cadastro e preenchimento de formulários.

Esses recursos tornam o Laravel uma plataforma moderna, flexível e altamente escalável para o desenvolvimento de testes automatizados, adequada tanto para projetos acadêmicos quanto para aplicações corporativas de grande porte, contribuindo para a qualidade, confiabilidade e manutenção contínua do software.

Considerações Finais

O estudo realizado permitiu compreender a relevância dos testes automatizados como parte essencial da garantia de qualidade em software. A utilização do ambiente de testes do Laravel em conjunto com o PHPUnit demonstrou que a criação de testes de forma simples e organizada, aliada ao uso de assertivas, factories, da trait RefreshDatabase e da validação de exceções, contribui significativamente para a identificação rápida e confiável de falhas durante o desenvolvimento.

A prática desenvolvida por meio do sistema de gestão de produtos possibilitou observar o funcionamento do framework em cenários reais de validação. Os testes implementados permitiram verificar operações como o cálculo de descontos e a consulta de disponibilidade de estoque, além de validar o comportamento da aplicação diante de situações excepcionais, como tentativas de aplicação de descontos com valores inválidos. Dessa forma, foi possível evidenciar

a importância dos testes automatizados na prevenção de erros e na manutenção da qualidade do código.

Como perspectiva para trabalhos futuros, podem ser explorados recursos mais avançados do ecossistema Laravel, como testes HTTP de APIs RESTful, mocking avançado de dependências externas, testes de navegador com o Laravel Dusk, análise de cobertura de código por meio de pacotes específicos e integração com ferramentas de Integração Contínua e Entrega Contínua (CI/CD), como GitHub Actions e GitLab CI. Essas práticas contribuem para aumentar a confiabilidade das aplicações e aproximar o processo de desenvolvimento das exigências encontradas no mercado de software.

Por fim, o aprendizado obtido ao longo deste trabalho contribuiu para uma melhor compreensão dos conceitos de testes automatizados e de sua aplicação prática em projetos PHP utilizando o Laravel. O ambiente de testes do Laravel mostrou-se uma solução moderna, acessível e eficiente, capaz de auxiliar tanto estudantes quanto profissionais na construção de aplicações mais seguras, estáveis e alinhadas às boas práticas da Engenharia de Software.

Bibliografia

LARAVEL. *Laravel Documentation*. Disponível em: <https://laravel.com/docs/>. Acesso em: jun. 2026.

PHPUNIT. *PHPUnit Documentation*. Disponível em: <https://docs.phpunit.de/>. Acesso em: jun. 2026.

PHP DOCUMENTATION GROUP. *PHP Manual*. Disponível em: <https://www.php.net/manual/>. Acesso em: jun. 2026.

LARAVEL. *Laravel Dusk Documentation*. Disponível em: <https://laravel.com/docs/dusk>. Acesso em: jun. 2026.

Testes Automatizados no Laravel com PHPUnit

Marco Antonio Amorim Filho

marcoamorim877@gmail.com

Resumo

Este e-book apresenta a utilização do PHPUnit no contexto do framework Laravel, uma das soluções mais populares para automação de testes em aplicações desenvolvidas na linguagem PHP. O objetivo principal é demonstrar como a implementação de testes automatizados contribui para a qualidade do software, aumentando a confiabilidade do código, facilitando a identificação de falhas e apoiando práticas modernas de desenvolvimento, como o Test-Driven Development (TDD) e a Integração Contínua (CI). A abordagem adotada combina a fundamentação teórica dos testes de software com a exploração prática dos recursos oferecidos pelo Laravel em conjunto com o PHPUnit, por meio da construção de exemplos e cenários reais de validação. Ao longo do material, são apresentados conceitos essenciais, como a criação de funções de teste, o uso de assertivas, a aplicação da trait RefreshDatabase para isolamento do banco de dados, o uso de factories e a validação de exceções por meio do expectException. Os resultados evidenciam que o ecossistema Laravel oferece uma solução integrada, intuitiva e escalável para o desenvolvimento de testes automatizados, sendo amplamente utilizado tanto em ambientes acadêmicos quanto corporativos. Conclui-se que sua adoção contribui para a padronização dos processos de teste, promove maior segurança durante a manutenção do código e fortalece a cultura de qualidade no desenvolvimento de software.

Palavras-chave: PHPUnit, Laravel, testes automatizados, PHP, qualidade de software.

Introdução

A qualidade de software é um dos pilares fundamentais no desenvolvimento de sistemas modernos, exigindo práticas que garantam confiabilidade, manutenção contínua e redução de falhas ao longo de todo o ciclo de vida do produto. Entre essas práticas, os testes automatizados desempenham papel estratégico, pois permitem validar pequenas partes do código de forma isolada e, ao mesmo tempo, verificar o comportamento integrado de toda a aplicação, assegurando que cada componente execute corretamente sua função.

Nesse contexto, este e-book dedica-se ao estudo dos testes automatizados no Laravel, framework PHP amplamente adotado no mercado, com utilização do PHPUnit como ferramenta de execução de testes. O Laravel é reconhecido por oferecer uma estrutura completa e pronta para uso, na qual o ambiente de testes já vem configurado por padrão desde a criação do projeto. O escopo do

trabalho concentra-se na apresentação dos conceitos fundamentais relacionados aos testes automatizados, na análise das principais funcionalidades oferecidas pelo Laravel em conjunto com o PHPUnit e na demonstração prática de sua aplicação em um projeto de software.

O objetivo geral é compreender de que forma a automatização de testes contribui para a melhoria da qualidade do software e como o Laravel auxilia nesse processo. Como objetivos específicos, destacam-se: identificar as características e vantagens da ferramenta, analisar sua estrutura de funcionamento e apresentar exemplos práticos que evidenciem sua utilização em diferentes cenários de desenvolvimento.

A escolha do tema justifica-se pela ampla adoção do Laravel na indústria de software brasileira e internacional, especialmente devido à sua sintaxe expressiva, ao suporte nativo a recursos de teste, à facilidade de integração com processos modernos de desenvolvimento, como Integração Contínua (CI) e Desenvolvimento Orientado a Testes (TDD), e à robustez do ecossistema PHP.

A metodologia utilizada envolveu pesquisa bibliográfica, análise da documentação oficial do framework Laravel e do PHPUnit, e experimentação prática por meio da implementação de casos de teste em PHP. A organização deste e-book segue três etapas principais: inicialmente, são apresentados os conceitos fundamentais dos testes automatizados e do ambiente de testes do Laravel; em seguida, são explorados seus principais recursos e exemplos de aplicação; por fim, são apresentadas as considerações finais, destacando os benefícios da ferramenta e possíveis direções para estudos futuros.

Visão Geral

O Laravel é um framework PHP amplamente utilizado no desenvolvimento de aplicações web devido à sua sintaxe expressiva, à completude de seus recursos e à facilidade de uso. Ele oferece, de forma nativa, um ambiente completo de testes baseado no PHPUnit, permitindo criar testes de forma direta, utilizando funções e classes da linguagem PHP e assertivas fornecidas tanto pelo PHPUnit quanto pelas extensões específicas do Laravel, tornando o código de teste mais legível e intuitivo.

Além da execução de testes unitários, o ecossistema do Laravel oferece suporte para testes de integração, testes HTTP simulados, testes envolvendo banco de dados, mocking de serviços externos e até mesmo testes de navegador por meio do pacote Laravel Dusk. Outro diferencial importante é a capacidade de descobrir automaticamente arquivos e funções de teste, reduzindo a necessidade de configurações adicionais.

Sua arquitetura extensível permite a utilização de pacotes desenvolvidos pela comunidade, ampliando funcionalidades como medição de cobertura de código, execução paralela e integração

com pipelines de Integração Contínua (CI). Dessa forma, o Laravel tornou-se uma das ferramentas mais adotadas para o desenvolvimento de aplicações PHP com cobertura de testes automatizados.

Breve histórico

O PHPUnit foi criado por Sebastian Bergmann em 2001 com inspiração no JUnit, framework de testes para a linguagem Java. Seu objetivo era oferecer uma alternativa robusta para a automação de testes em aplicações PHP, em um momento em que essa prática ainda era pouco difundida no ecossistema. Ao longo dos anos, o PHPUnit consolidou-se como a principal ferramenta de testes da linguagem PHP.

O Laravel, por sua vez, foi criado por Taylor Otwell em 2011 com o objetivo de oferecer um framework PHP mais expressivo, produtivo e moderno. Desde suas primeiras versões, o framework incorporou o PHPUnit como dependência padrão, fornecendo um ambiente de testes pré-configurado em cada novo projeto. Esta integração nativa entre Laravel e PHPUnit tornou a prática de testes automatizados acessível a desenvolvedores iniciantes e avançados.

Atualmente, o Laravel é considerado um dos frameworks PHP mais populares do mundo, sendo utilizado tanto em projetos acadêmicos quanto em aplicações corporativas de grande porte. Sua adoção crescente está relacionada à facilidade de uso, à clareza dos relatórios gerados pelo PHPUnit, à eficiência no desenvolvimento de testes automatizados e ao suporte mais recente também a sintaxes modernas como a do framework Pest, que oferece uma camada de escrita mais expressiva sobre o próprio PHPUnit.

Ambiente de testes

No contexto do Laravel, o ambiente de testes corresponde ao conjunto de recursos e práticas que permitem criar, organizar e executar verificações automatizadas de forma eficiente. Entre os principais elementos desse ambiente, destacam-se:

- **Arquivos de teste:** normalmente organizados em módulos específicos dentro das pastas `tests/Unit` e `tests/Feature`, com nomes finalizados pelo sufixo “Test”, facilitando sua identificação automática pelo framework.
- **Funções de teste:** implementadas como métodos de classe, geralmente precedidas pelo prefixo `test_`, permitindo que o PHPUnit as reconheça automaticamente.
- **Assertivas:** o framework utiliza métodos assertivos fornecidos pelo PHPUnit, como `assertEquals`, `assertTrue`, `assertStatus` e `assertDatabaseHas`, tornando os testes mais simples e legíveis.

- **Factories:** mecanismo utilizado para gerar dados falsos de forma rápida e padronizada, preparando o ambiente necessário para os testes e promovendo reutilização.
- **RefreshDatabase:** trait disponibilizada pelo Laravel que envolve cada teste em uma transação de banco de dados, executando rollback automático ao final, mantendo o ambiente sempre limpo.
- **Execução por linha de comando:** realizada por meio do comando `php artisan test`, que executa automaticamente todos os testes encontrados no projeto e apresenta relatórios formatados.
- **Relatórios de execução:** apresentam informações detalhadas sobre testes aprovados, falhas, erros e estatísticas gerais de execução, com saída colorida no terminal.

Esse ambiente simplifica o processo de automação de testes e possibilita sua integração com ferramentas de desenvolvimento, versionamento e integração contínua, tornando-o adequado para projetos de diferentes portes.

Principais Conceitos e Estruturas do PHPUnit no Laravel

Para simplificar a criação e manutenção de testes automatizados, o Laravel disponibiliza, em conjunto com o PHPUnit, diversos recursos que tornam o desenvolvimento mais produtivo e organizado. Entre os principais, destacam-se:

- **Assertivas:** utilizadas para verificar se um resultado corresponde ao esperado. Métodos como `assertEquals`, `assertTrue`, `assertFalse`, `assertCount` e `assertNull` validam diferentes tipos de condições e fornecem mensagens de erro detalhadas quando uma validação falha.
- **Assertivas HTTP:** métodos específicos para verificar respostas de requisições simuladas, como `assertStatus`, `assertOk`, `assertSee` e `assertRedirect`, permitindo testar rotas e controllers de forma simples.
- **Assertivas de banco de dados:** métodos como `assertDatabaseHas`, `assertDatabaseMissing`, `assertDatabaseCount`, `assertModelExists` e `assertModelMissing` permitem validar o estado dos dados persistidos durante os testes.
- **expectException:** recurso utilizado para validar exceções esperadas. Permite verificar se determinada operação gera o erro previsto, garantindo o tratamento adequado de situações excepcionais.
- **Trait RefreshDatabase:** incluída em uma classe de teste, faz com que cada método de teste seja envolvido em uma transação e revertido ao final, mantendo o banco em estado conhecido entre testes sem afetar dados reais.

- **Factories:** classes responsáveis por gerar instâncias de modelos com dados fictícios, permitindo criar registros completos em poucas linhas e reduzindo duplicação de código nos testes.
- **Mocking:** recurso que permite substituir comportamentos reais de serviços externos por simulações, como `Cache::expects`, `Mail::fake` e `Queue::fake`, isolando o teste de dependências externas.
- **Test Discovery (descoberta automática de testes):** funcionalidade que identifica automaticamente arquivos e classes de teste no projeto, dispensando configurações complexas para sua execução.
- **Filtragem de testes:** por meio da opção `--filter`, é possível executar apenas um subconjunto específico dos testes, agilizando o desenvolvimento de funcionalidades isoladas.

Esses recursos tornam o Laravel uma plataforma flexível e eficiente para a automação de testes, contribuindo para a qualidade, confiabilidade e manutenção do software.

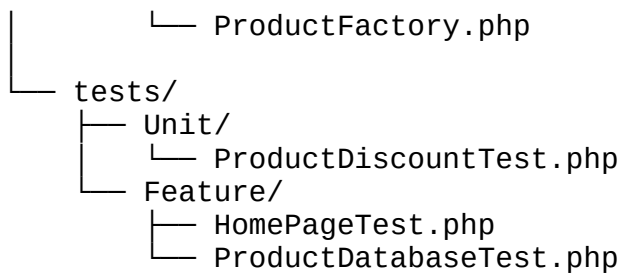
Desenvolvimento

Para o desenvolvimento deste projeto, foi realizada uma divisão em duas etapas principais. A primeira corresponde à pesquisa bibliográfica e documental, com foco no estudo da documentação oficial do Laravel e do PHPUnit, buscando compreender seus conceitos, recursos e funcionamento. A segunda etapa consistiu na aplicação prática, envolvendo a implementação de um sistema simples de gestão de produtos em PHP acompanhado da criação de testes automatizados utilizando o ambiente de testes do Laravel.

Essa prática teve como objetivo validar funcionalidades específicas de um modelo de produtos e demonstrar, de forma concreta, como os testes automatizados contribuem para a qualidade e confiabilidade do software. Foram explorados recursos importantes do Laravel em conjunto com o PHPUnit, como assertivas, factories, a trait `RefreshDatabase` e validação de exceções.

Seguindo o roteiro de demonstração, inicialmente foi criada a estrutura de diretórios do projeto para separar o código da aplicação dos arquivos de teste, conforme a organização padrão sugerida pelo próprio framework:

```
projeto_produtos/
├── app/
│   └── Models/
│       └── Product.php
├── database/
│   ├── migrations/
│   │   └── 2026_01_01_000000_create_products_table.php
│   └── factories/
```

No arquivo Product.php foi implementada a classe responsável pelas operações de um produto, incluindo a aplicação de descontos percentuais sobre o preço, com validação de regras de negócio, e a verificação de disponibilidade em estoque.

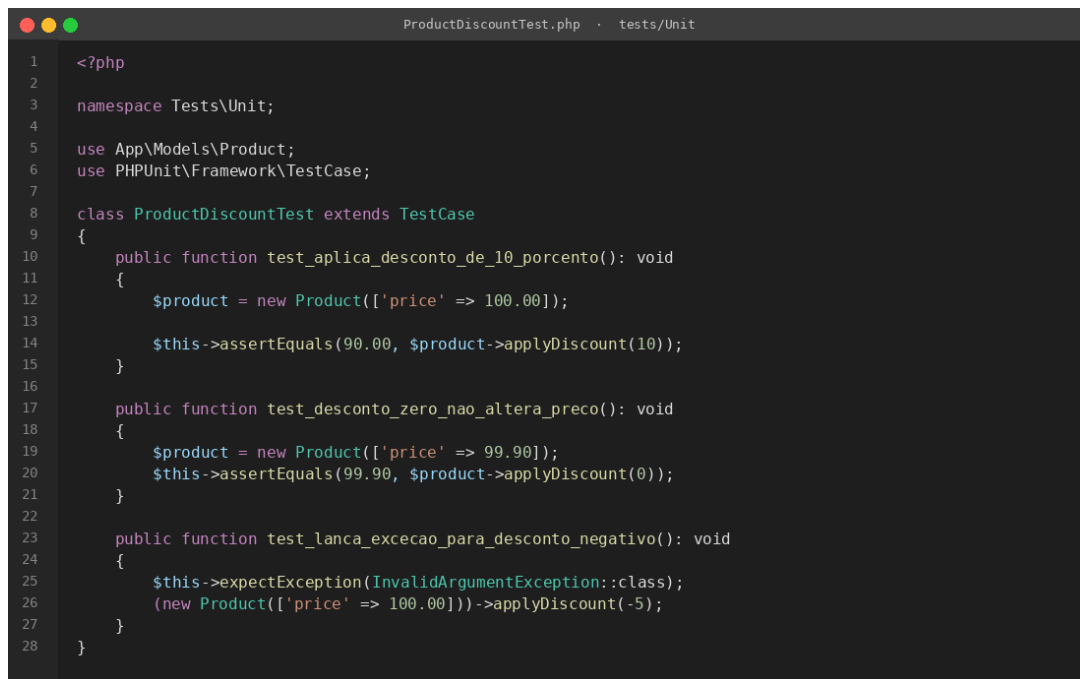
Figura 1 – Classe Product.php

```
1 <?php
2
3 namespace App\Models;
4
5 use Illuminate\Database\Eloquent\Model;
6
7 class Product extends Model
8 {
9     protected $fillable = ['name', 'price', 'stock'];
10
11     // Aplica desconto percentual sobre o preço
12     public function applyDiscount(float $percent): float
13     {
14         if ($percent < 0 || $percent > 100) {
15             throw new InvalidArgumentException('Desconto deve estar entre 0 e 100');
16         }
17
18         return round($this->price * (1 - $percent / 100), 2);
19     }
20
21     public function isAvailable(): bool
22     {
23         return $this->stock > 0;
24     }
25 }
```

Fonte: próprio autor

Após a implementação da classe, foram criados testes automatizados para validar o comportamento do modelo de produtos. Os testes verificam diferentes cenários, incluindo aplicação de desconto válido, desconto igual a zero e lançamento de exceção para valores inválidos. A organização do código de teste segue o padrão esperado pelo PHPUnit, com classe que estende TestCase e métodos com prefixo test_.

Figura 2 – Testes da classe Product (ProductDiscountTest.php)



```

1  <?php
2
3  namespace Tests\Unit;
4
5  use App\Models\Product;
6  use PHPUnit\Framework\TestCase;
7
8  class ProductDiscountTest extends TestCase
9  {
10     public function test_aplica_desconto_de_10_porcento(): void
11     {
12         $product = new Product(['price' => 100.00]);
13
14         $this->assertEquals(90.00, $product->applyDiscount(10));
15     }
16
17     public function test_desconto_zero_nao_altera_preco(): void
18     {
19         $product = new Product(['price' => 99.90]);
20         $this->assertEquals(99.90, $product->applyDiscount(0));
21     }
22
23     public function test_lanca_excecao_para_desconto_negativo(): void
24     {
25         $this->expectException(InvalidArgumentException::class);
26         (new Product(['price' => 100.00]))->applyDiscount(-5);
27     }
28 }

```

Fonte: próprio autor

Os testes desenvolvidos verificam diferentes cenários de utilização do sistema. Foram avaliadas operações válidas, como o cálculo correto do desconto, além de situações excepcionais, como tentativas de aplicação de descontos negativos ou superiores a cem por cento, que devem resultar no lançamento de uma exceção.

Para permitir que o Laravel encontre automaticamente os testes, foi adotada a seguinte convenção:

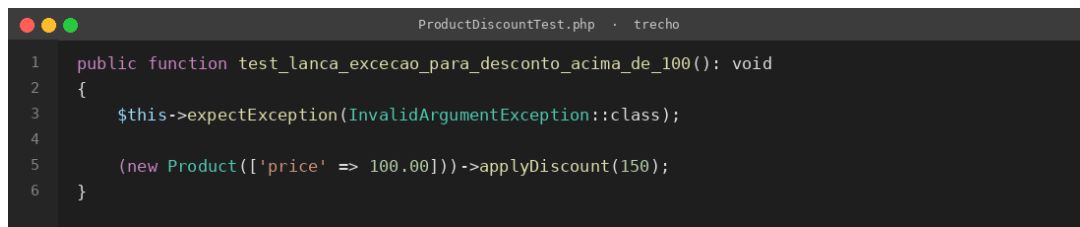
- Os arquivos de teste possuem o sufixo Test em seu nome (por exemplo, ProductDiscountTest.php);
- As funções de teste iniciam com o prefixo test_;
- Os testes foram organizados nas pastas tests/Unit e tests/Feature;
- A execução foi realizada por meio do comando:

```
php artisan test
```

Além dos cenários de sucesso, foram implementados testes utilizando o método expectException para validar o tratamento correto de exceções. Também foi utilizada a trait RefreshDatabase nos testes que envolvem o banco de dados, permitindo a reutilização da configuração inicial e o isolamento entre os diferentes casos de teste.

Os resultados obtidos demonstraram que o Laravel oferece uma forma simples e eficiente de automatizar testes em aplicações PHP, contribuindo para a redução de erros, aumento da confiabilidade do código e maior facilidade de manutenção do software.

Figura 3 – Teste de exceção para desconto acima de 100%



```

1 public function test_lanca_excecao_para_desconto_acima_de_100(): void
2 {
3     $this->expectException(InvalidArgumentException::class);
4
5     (new Product(['price' => 100.00]))->applyDiscount(150);
6 }

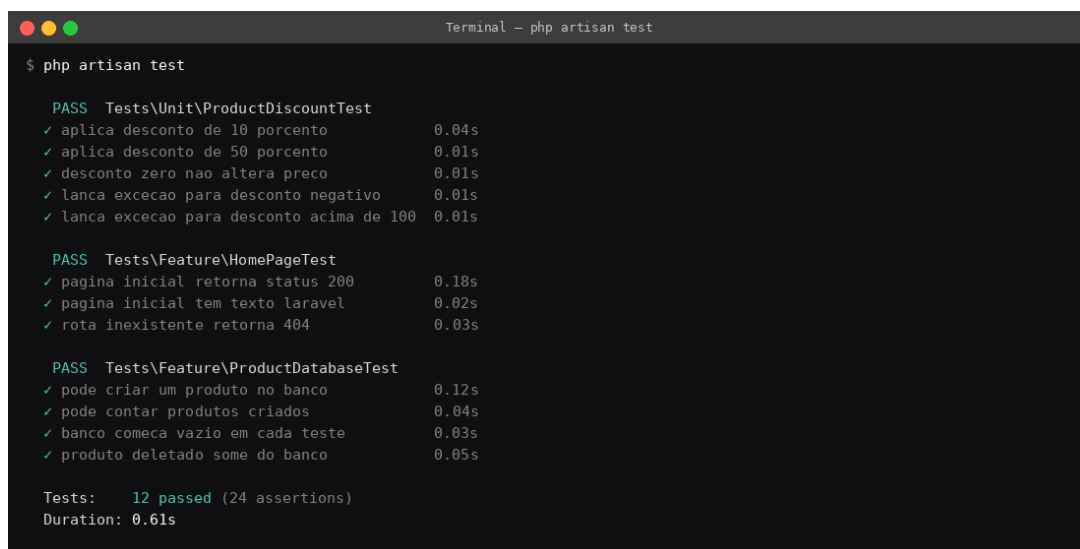
```

Fonte: próprio autor

Para demonstrar a capacidade do framework em validar situações de erro, foi implementado um teste utilizando o método `expectException`. Nesse cenário, a aplicação tenta aplicar um desconto superior a cem por cento sobre o produto, verificando se a exceção esperada (`InvalidArgumentException`) é lançada corretamente. Esse tipo de teste é importante para garantir que as regras de negócio sejam respeitadas e que comportamentos inadequados sejam tratados pelo sistema.

Ao executar o comando de teste, todos os casos implementados são identificados automaticamente pelo Laravel e executados em sequência. Ao final da execução, o framework apresenta um relatório contendo informações sobre os testes aprovados, falhas encontradas e possíveis erros.

Figura 4 – Relatório de execução do comando `php artisan test`



```

Terminal - php artisan test

$ php artisan test

PASS Tests\Unit\ProductDiscountTest
✓ aplica desconto de 10 por cento      0.04s
✓ aplica desconto de 50 por cento      0.01s
✓ desconto zero nao altera preco      0.01s
✓ lanca excecao para desconto negativo 0.01s
✓ lanca excecao para desconto acima de 100 0.01s

PASS Tests\Feature\HomePageTest
✓ pagina inicial retorna status 200    0.18s
✓ pagina inicial tem texto laravel     0.02s
✓ rota inexistente retorna 404         0.03s

PASS Tests\Feature\ProductDatabaseTest
✓ pode criar um produto no banco       0.12s
✓ pode contar produtos criados         0.04s
✓ banco comeca vazio em cada teste     0.03s
✓ produto deletado some do banco       0.05s

Tests: 12 passed (24 assertions)
Duration: 0.61s

```

Fonte: próprio autor

As saídas apresentadas pelo Laravel em conjunto com o PHPUnit podem ser classificadas da seguinte forma:

- **PASS:** o teste foi executado com sucesso e o resultado obtido corresponde ao esperado;
- **FAIL:** uma assertiva falhou, indicando que o resultado obtido é diferente do resultado esperado;
- **ERROR:** ocorreu uma exceção inesperada durante a execução do teste ou durante a configuração do ambiente de teste;

- **SKIPPED:** o teste foi ignorado intencionalmente por alguma configuração definida pelo desenvolvedor, geralmente por meio do método `markTestSkipped`.

É importante destacar que, além dos recursos explorados neste projeto, o Laravel disponibiliza diversas funcionalidades avançadas que ampliam significativamente sua aplicação em cenários profissionais. Entre elas, destacam-se as factories parametrizadas, que permitem reutilizar configurações em múltiplos testes; a utilização de provedores de dados via `@dataProvider`, que possibilita executar o mesmo teste com diferentes conjuntos de dados; e o amplo ecossistema de pacotes, que adiciona funcionalidades como geração de relatórios detalhados, medição de cobertura de código e execução paralela.

Outro recurso bastante utilizado é o sistema de mocking, integrado ao Laravel por meio das facades de teste (`Cache::expects`, `Mail::fake`, `Queue::fake`, `Notification::fake`, `Event::fake`, entre outras) e da biblioteca Mockery. Essa técnica permite simular dependências externas, como envio de e-mails, processamento de filas, chamadas a APIs ou serviços de terceiros, tornando os testes mais rápidos, previsíveis e independentes do ambiente externo.

Adicionalmente, o ecossistema do Laravel oferece o pacote Laravel Dusk, voltado para testes de navegador automatizados. Por meio dele, é possível simular um usuário real interagindo com a aplicação em um navegador Chrome controlado pelo ChromeDriver, permitindo verificar fluxos completos, como login, cadastro e preenchimento de formulários.

Esses recursos tornam o Laravel uma plataforma moderna, flexível e altamente escalável para o desenvolvimento de testes automatizados, adequada tanto para projetos acadêmicos quanto para aplicações corporativas de grande porte, contribuindo para a qualidade, confiabilidade e manutenção contínua do software.

Considerações Finais

O estudo realizado permitiu compreender a relevância dos testes automatizados como parte essencial da garantia de qualidade em software. A utilização do ambiente de testes do Laravel em conjunto com o PHPUnit demonstrou que a criação de testes de forma simples e organizada, aliada ao uso de assertivas, factories, da trait `RefreshDatabase` e da validação de exceções, contribui significativamente para a identificação rápida e confiável de falhas durante o desenvolvimento.

A prática desenvolvida por meio do sistema de gestão de produtos possibilitou observar o funcionamento do framework em cenários reais de validação. Os testes implementados permitiram verificar operações como o cálculo de descontos e a consulta de disponibilidade de estoque, além de validar o comportamento da aplicação diante de situações excepcionais, como tentativas de aplicação

de descontos com valores inválidos. Dessa forma, foi possível evidenciar a importância dos testes automatizados na prevenção de erros e na manutenção da qualidade do código.

Como perspectiva para trabalhos futuros, podem ser explorados recursos mais avançados do ecossistema Laravel, como testes HTTP de APIs RESTful, mocking avançado de dependências externas, testes de navegador com o Laravel Dusk, análise de cobertura de código por meio de pacotes específicos e integração com ferramentas de Integração Contínua e Entrega Contínua (CI/CD), como GitHub Actions e GitLab CI. Essas práticas contribuem para aumentar a confiabilidade das aplicações e aproximar o processo de desenvolvimento das exigências encontradas no mercado de software.

Por fim, o aprendizado obtido ao longo deste trabalho contribuiu para uma melhor compreensão dos conceitos de testes automatizados e de sua aplicação prática em projetos PHP utilizando o Laravel. O ambiente de testes do Laravel mostrou-se uma solução moderna, acessível e eficiente, capaz de auxiliar tanto estudantes quanto profissionais na construção de aplicações mais seguras, estáveis e alinhadas às boas práticas da Engenharia de Software.

Bibliografia

LARAVEL. Laravel Documentation. Disponível em: <https://laravel.com/docs/>. Acesso em: jun. 2026.

PHPUNIT. PHPUnit Documentation. Disponível em: <https://docs.phpunit.de/>. Acesso em: jun. 2026.

PHP DOCUMENTATION GROUP. PHP Manual. Disponível em: <https://www.php.net/manual/>. Acesso em: jun. 2026.

LARAVEL. Laravel Dusk Documentation. Disponível em: <https://laravel.com/docs/dusk>. Acesso em: jun. 2026.